

Multi-Modal Deep Learning

Lecture Series | Summer 2025

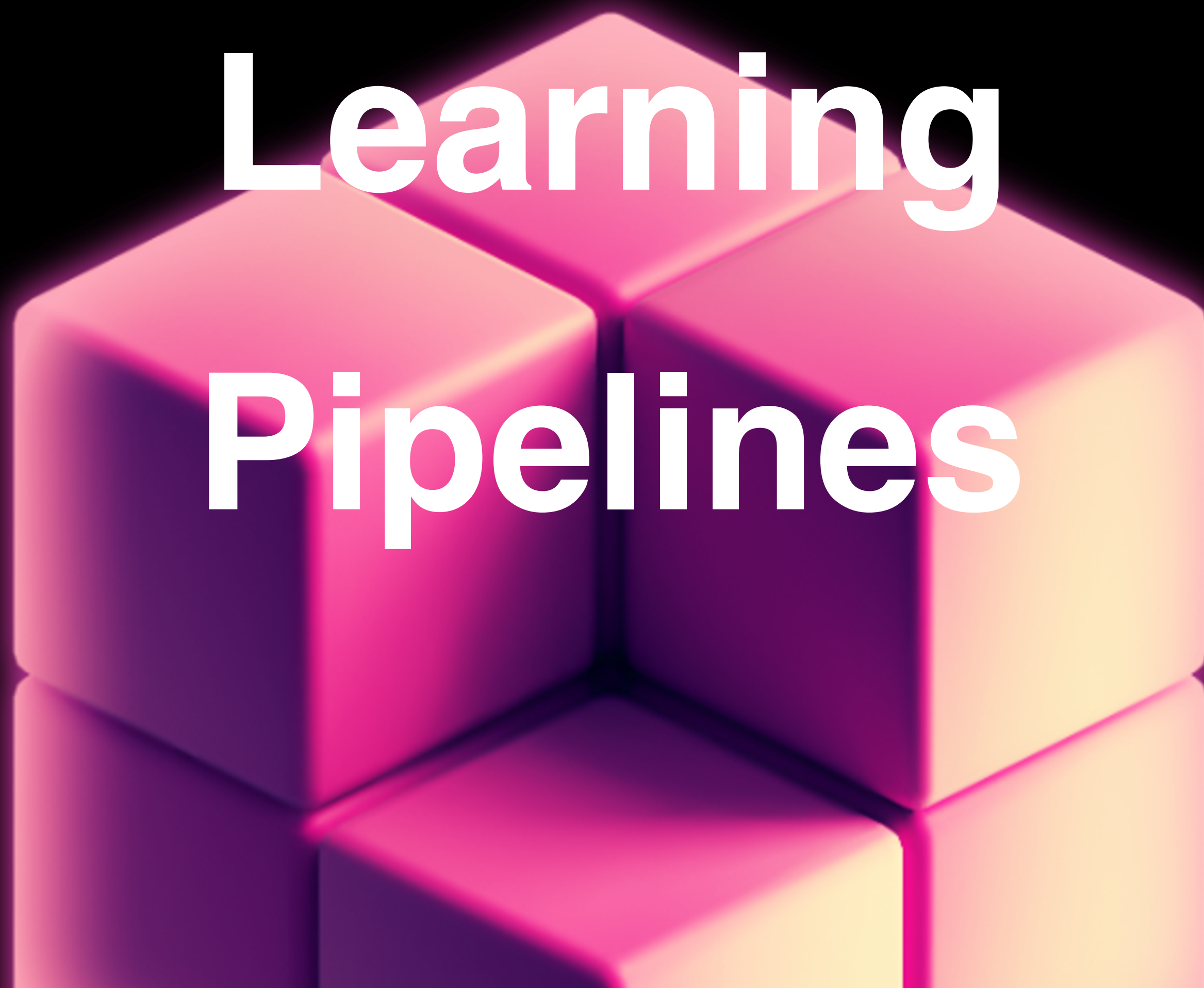
Institute for AI and Informatics in Medicine

01.07.2025 and 08.07.2025

Multi-Modal Deep

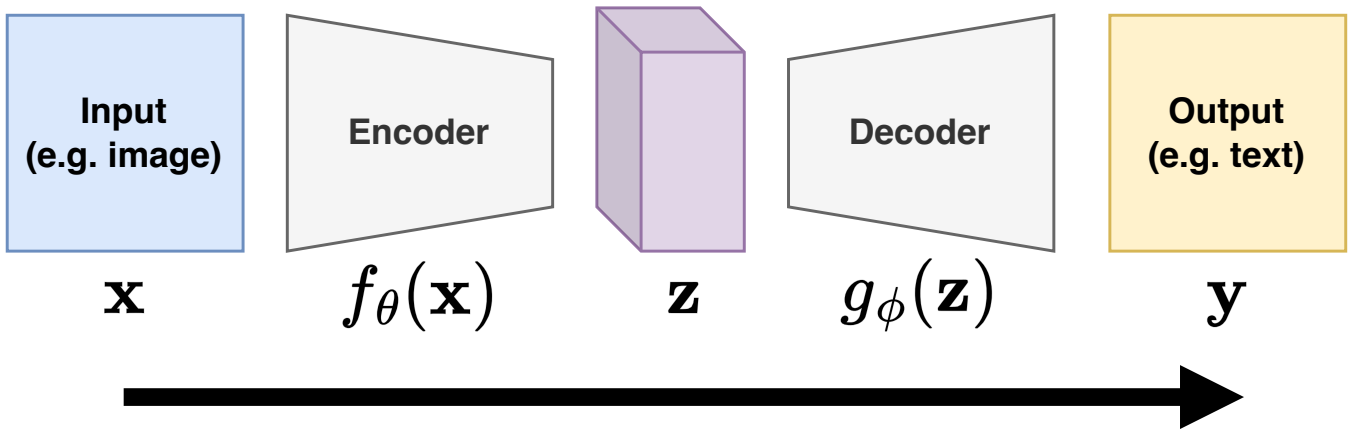
Learning

Pipelines

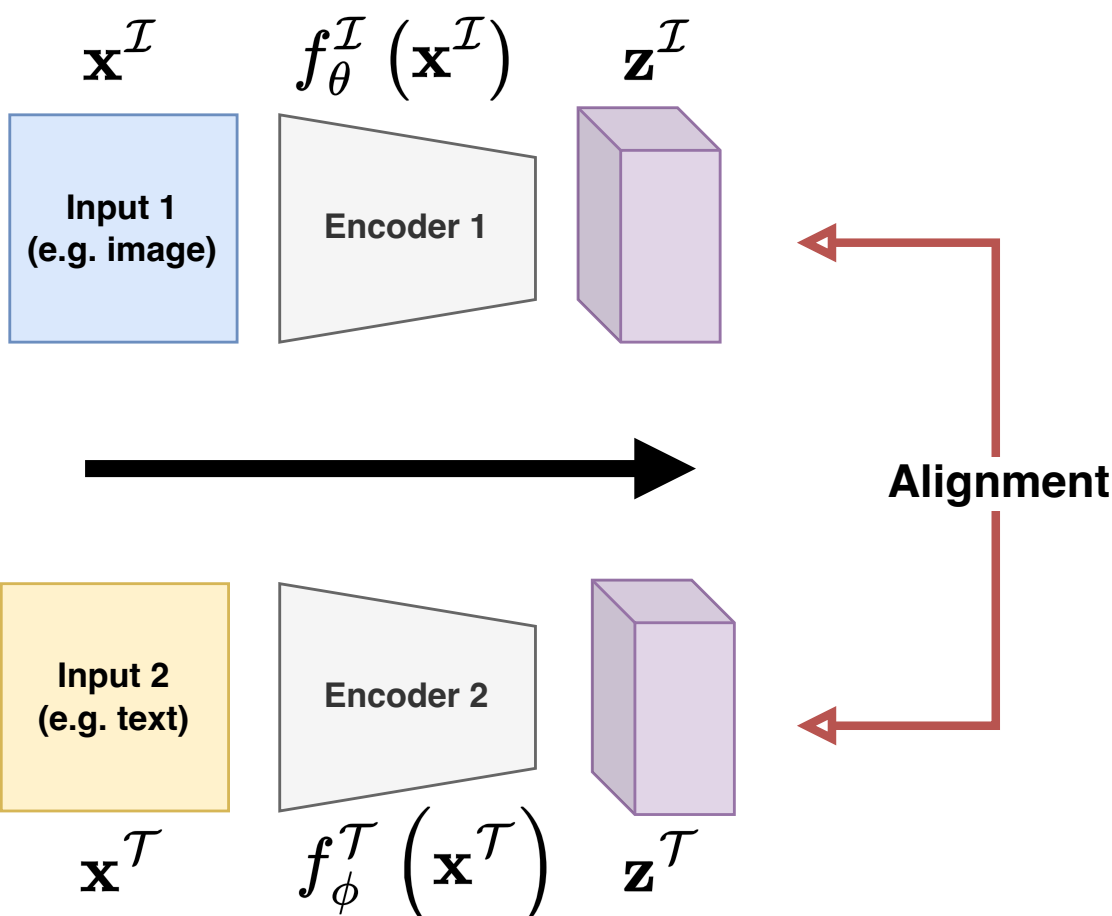


MULTI-MODAL DEEP LEARNING PIPELINES

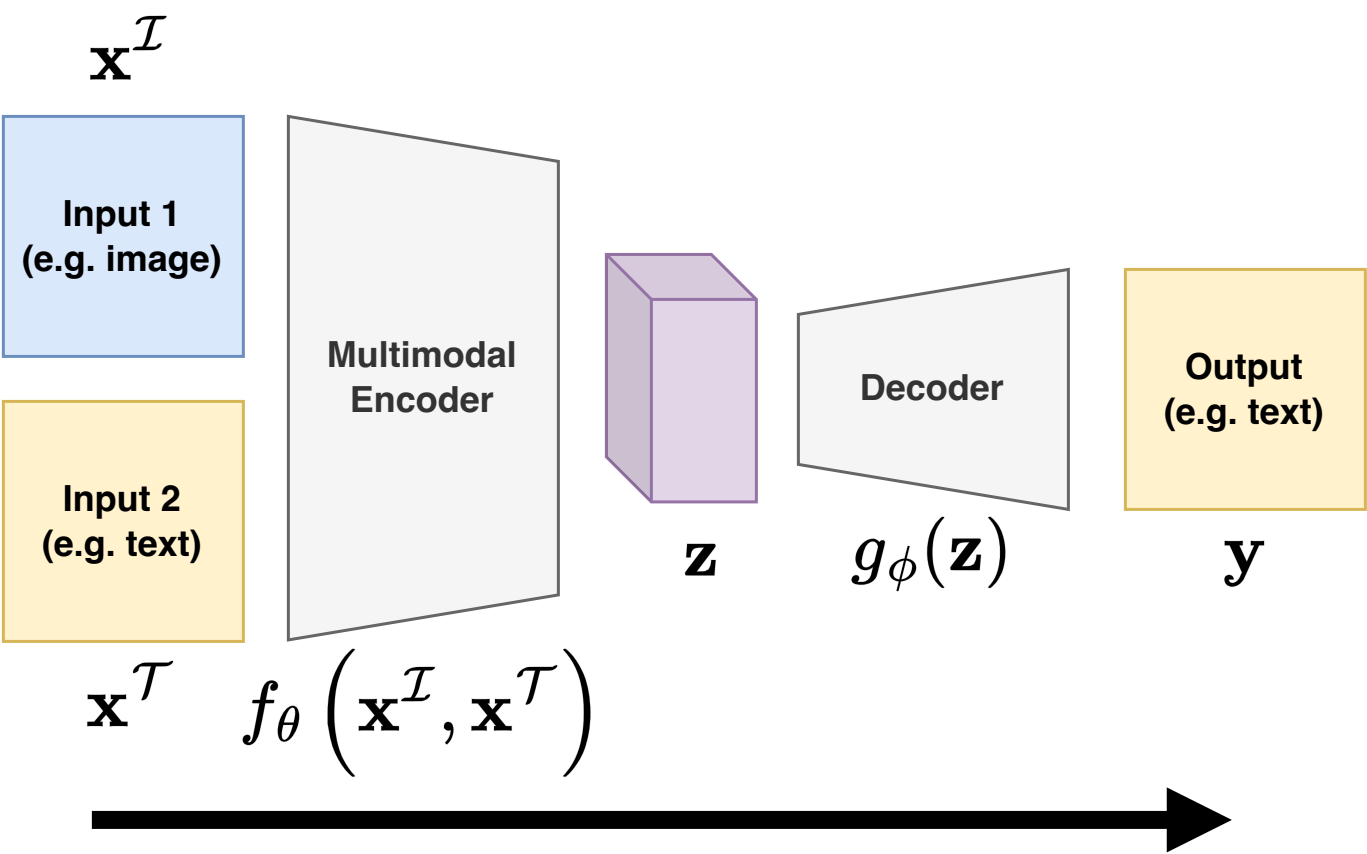
Translation



Alignment

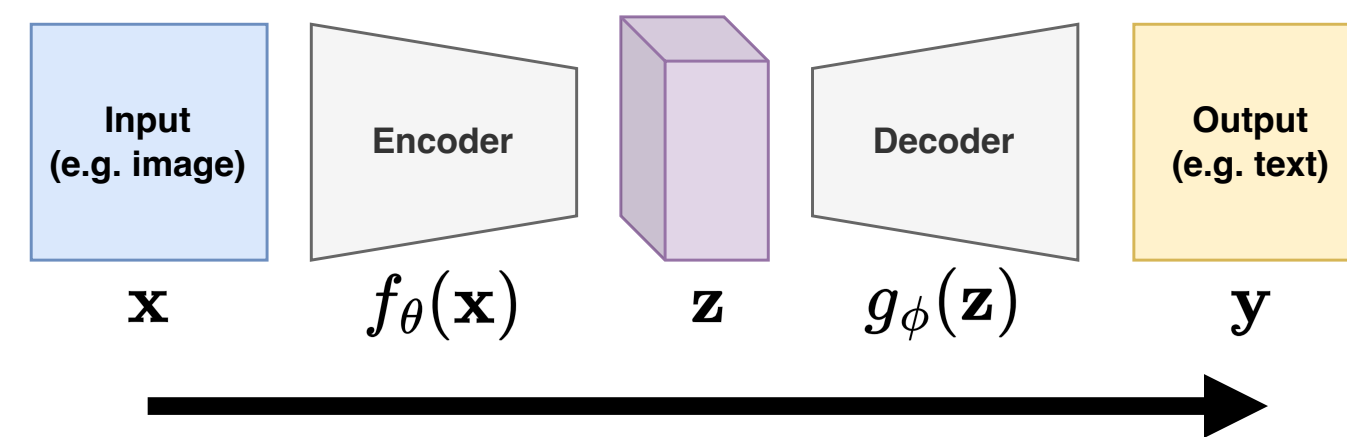


Fusion

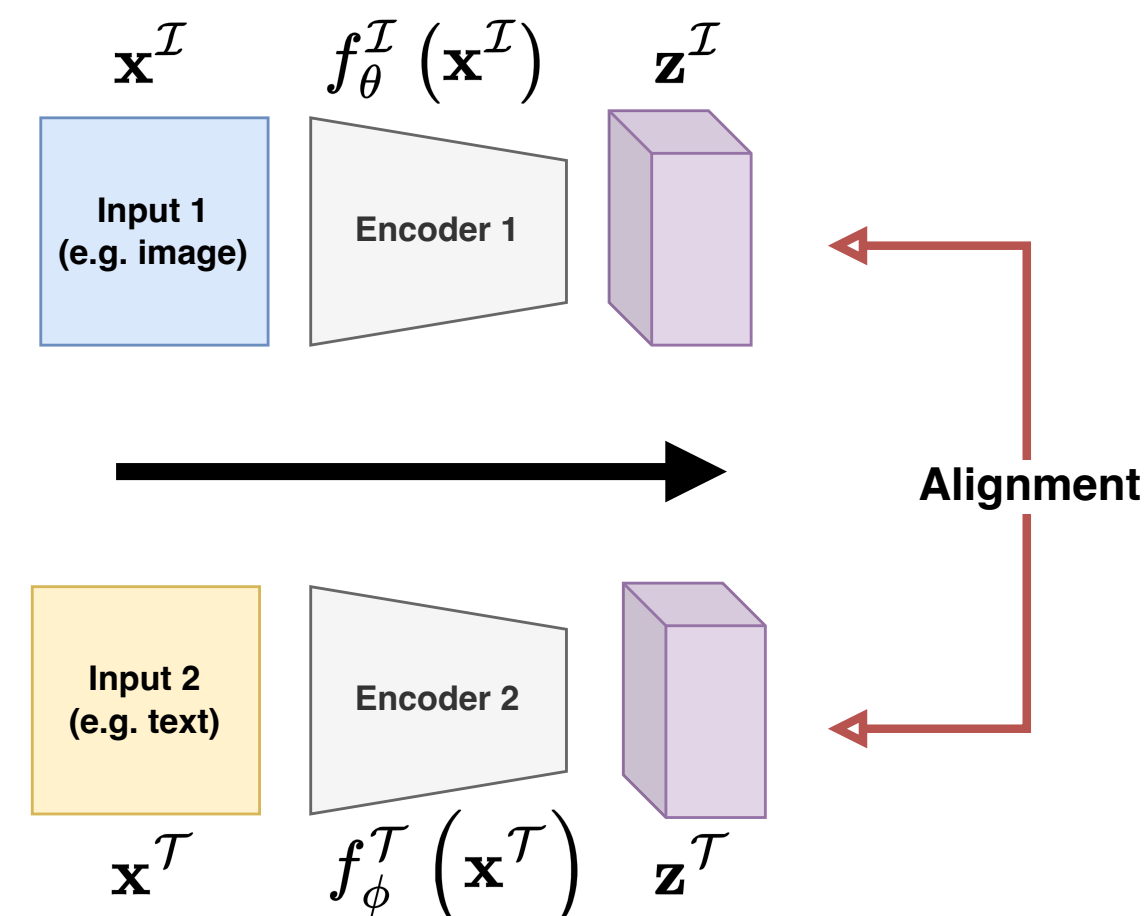


MULTI-MODAL DEEP LEARNING PIPELINES

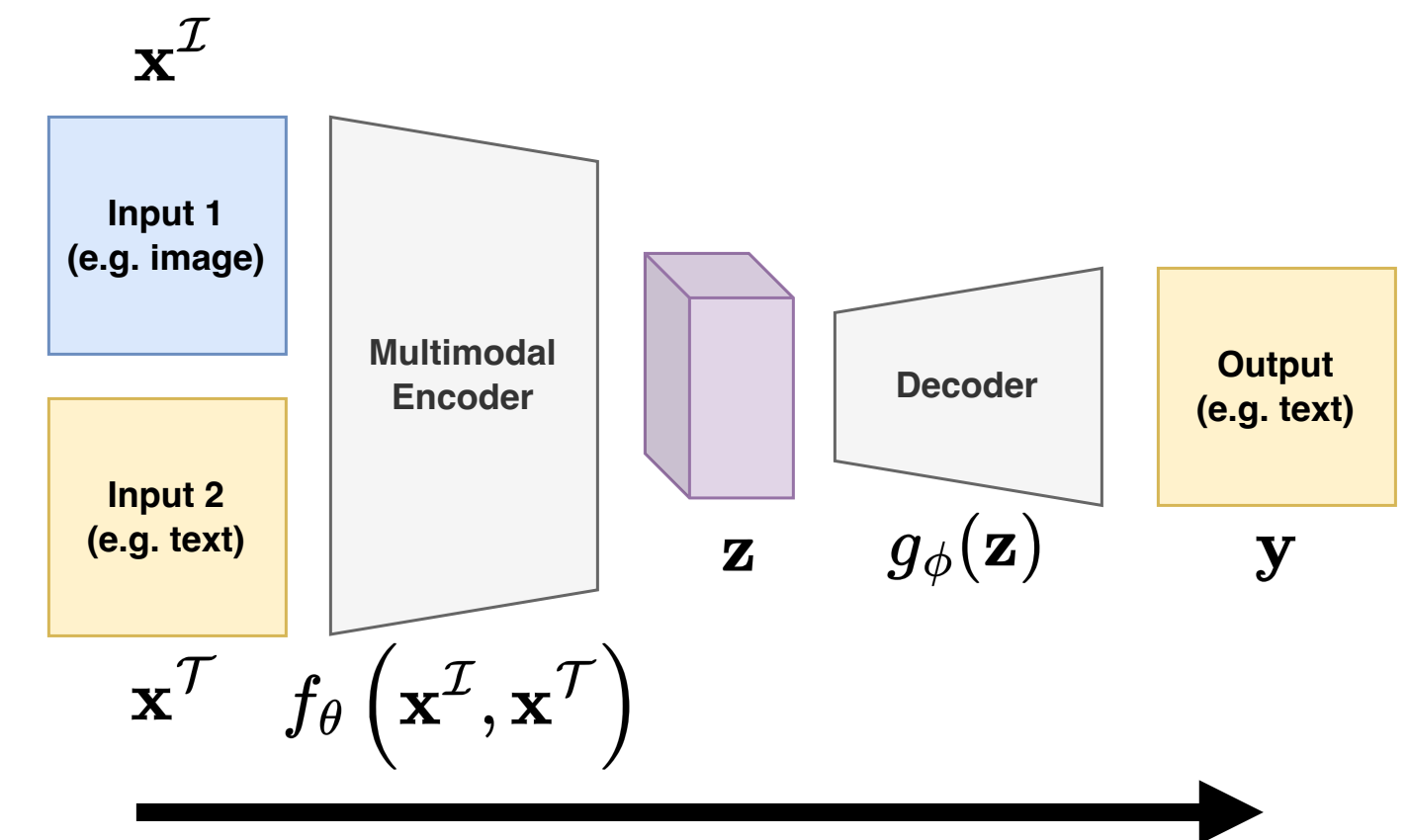
Translation



Alignment



Fusion



In reality multimodal models are a mixture of these!

We'll look at these pipelines to understand individual aspects of multimodal deep learning

MULTI-MODAL DEEP LEARNING PIPELINES

Multimodal Frameworks
combine different multimodal pipelines

Fusion

to combine multiple inputs or inputs with output queries

Translation

to generate the final outputs

Alignment

to pretrain individual components
to make the components work together
to (pre-)train the full framework

AGENDA

- **Fusion:**
How to combine or condition modalities
- **Translation:**
How to generate/decode modalities
- **Alignment:**
How to train

AGENDA

- **Fusion:**
How to combine or condition modalities
 - **Translation:**
How to generate/decode modalities
 - **Alignment:**
How to train
-
- **Multimodal Frameworks:**
Bringing it together / Example Models
 - **How-To:**
Designing Multimodal Systems

Multimodal

FUSION



MULTI-MODAL DEEP LEARNING PIPELINES

Multimodal Frameworks
combine different multimodal pipelines

Fusion

to combine multiple inputs or inputs with output queries

Translation

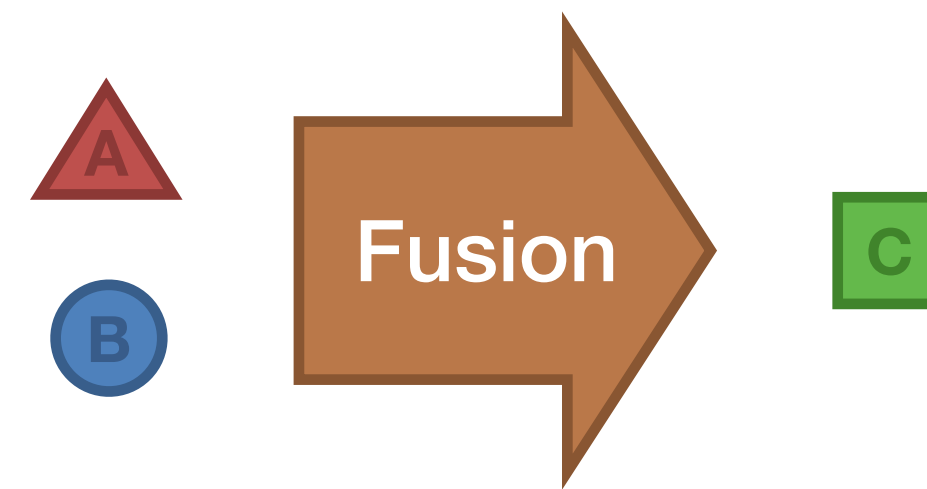
to generate the final outputs

Alignment

to pretrain individual components
to make the components work together
to (pre-)train the full framework

FUSION

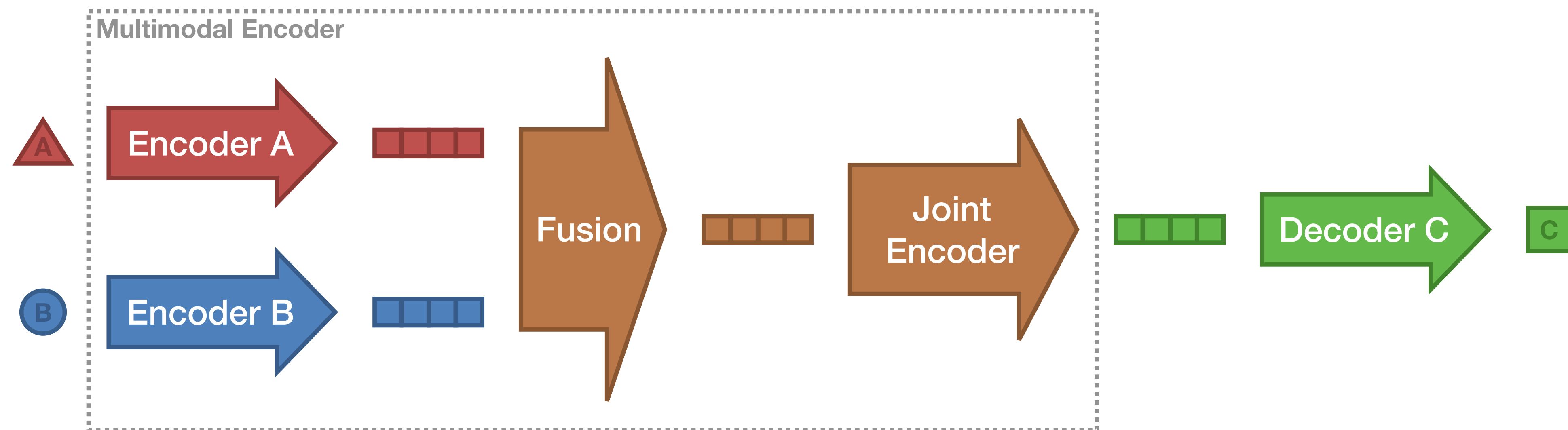
- Given multiple input modalities , , ...
- Combine (fuse) the modalities
- Predict a single output modality , conditioned on the inputs
 - could be a “real modality” (e.g. text) or a joint prediction (e.g. classification)



FUSION

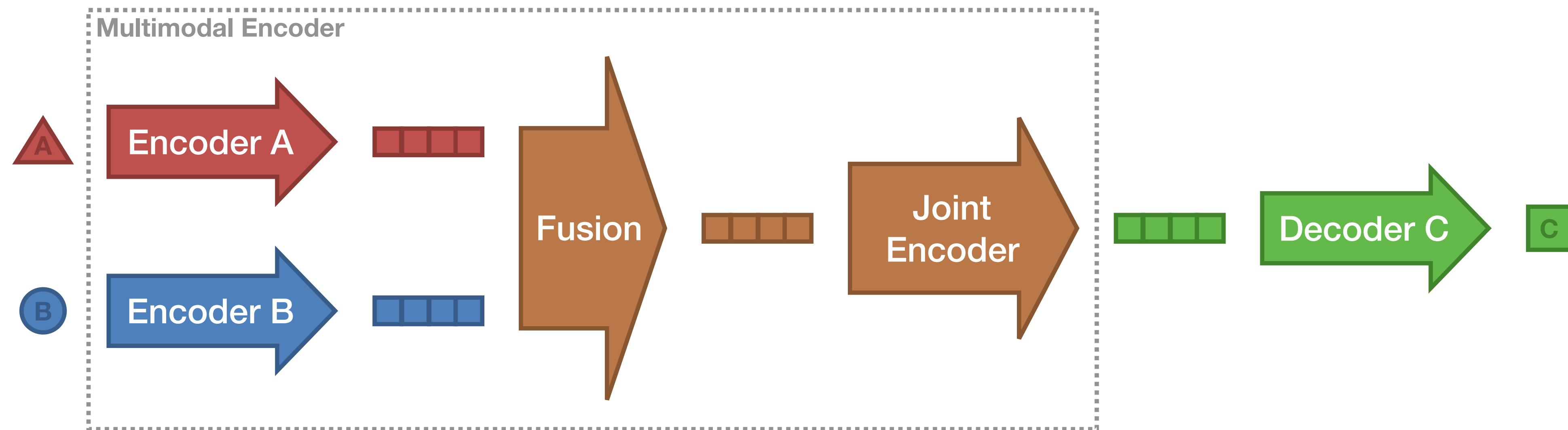
1. Encode each input modality (, , ...) individually into modality feature spaces 

2. Combine (fuse) modality features into a joint feature space 
3. Jointly encode fused modality features
4. Decode/predict output modality 



WHEN TO FUSE

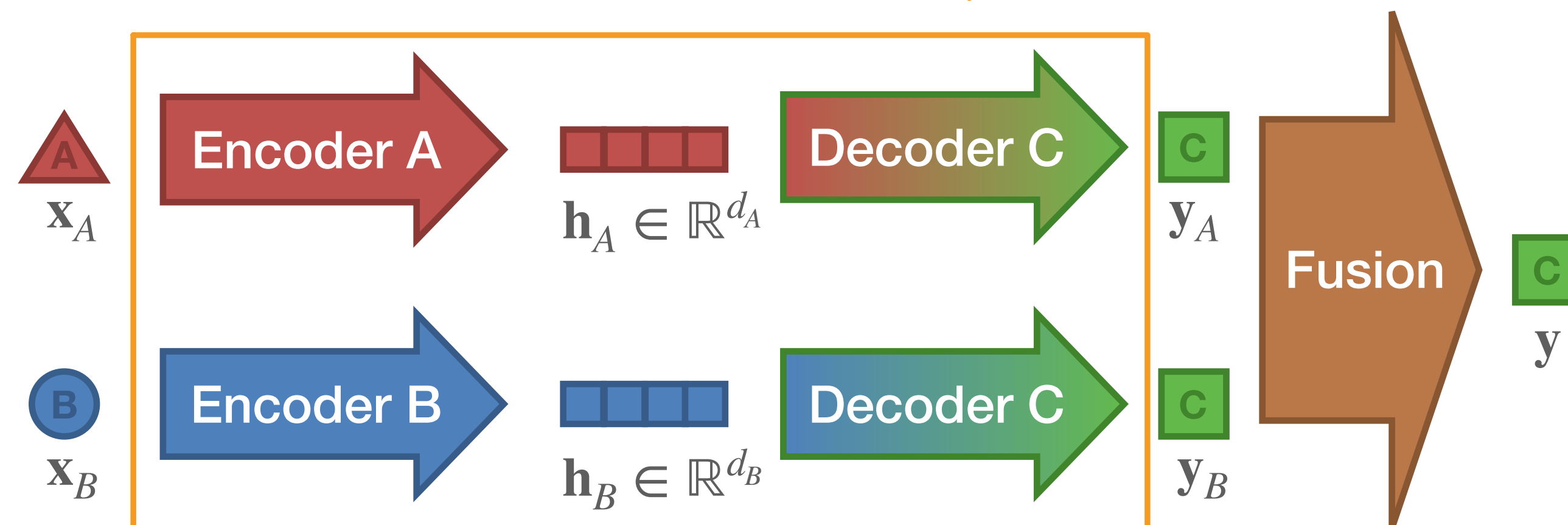
- Different encoders/decoders can be more shallow or deeper
- Fusion can happen at different stages



WHEN TO FUSE

Prediction Fusion (Late Fusion or Decision-level Fusion)

- Individual encoder-decoder (“translation”) models for each modality
 - Each directly predicts the output modality
- Fusion happens in the output/prediction space
 - No fusion at the feature/representation level



PREDICTION FUSION: SIMPLE APPROACHES

Voting

- Predict the class that was predicted by the majority of modality models
- Only applicable to classification tasks (classification, semantic segmentation, ...)
- Requires many modalities

Average

$$\mathbf{y} = \frac{1}{M} \sum_{m=1}^M \mathbf{y}_m$$

- Applicable to regression and classification tasks
- For classification-based tasks, apply averaging to probabilities before thresholding
- Not suitable for most complex output modalities
 - Images get blurry
 - Text and other sequence or set predictions may get incoherent

PREDICTION FUSION: SIMPLE APPROACHES

Weighted Averaging

$$\mathbf{y} = \frac{1}{\sum_m w_m} \sum_{m=1}^M w_m \cdot \mathbf{y}_m$$

- Learn or define coefficient (weight) w_m for each modality
- Similar issues as averaging but influence of modalities can be better controlled

Probabilistic

$$\mathbf{p} = \frac{1}{M} \prod_{m=1}^M \mathbf{p}_m$$

- Combine probabilities per modality multiplicatively (AND logic)
- Requires probabilistic outputs (e.g. softmax-based classification)
- More complicated probabilistic models are possible

Hand-crafted Rules

- Define specific rules how per-modality predictions affect the overall prediction

PREDICTION FUSION: WHAT ABOUT MORE COMPLEX OUTPUTS

Postprocessing with Models

- Apply additional models on per-modality predicted outputs to predict final output
- **Benefit:** We can use intermediate modalities for communication
- **Example:**
 - Multiple models predicting text
 - Combining the outputs by prompting an LLM

Agents / Tool Use

- Controller model (e.g. LLM) orchestrates the use of multiple models
 - Outputs of individual per-modality models can be interpreted by controller or further passed to other models for postprocessing
- ➡ Capabilities of many models can be combined

AGENTS / TOOL USE

Agents

- Typically LLMs
- Prompt include list of potential tools to be called
- LLM calls tools using special syntax (e.g. special tokens, json-format, ...)
- Parse LLM output and forward needed context to tools
- Run tools and insert their outputs back into LLM prompt

Tools

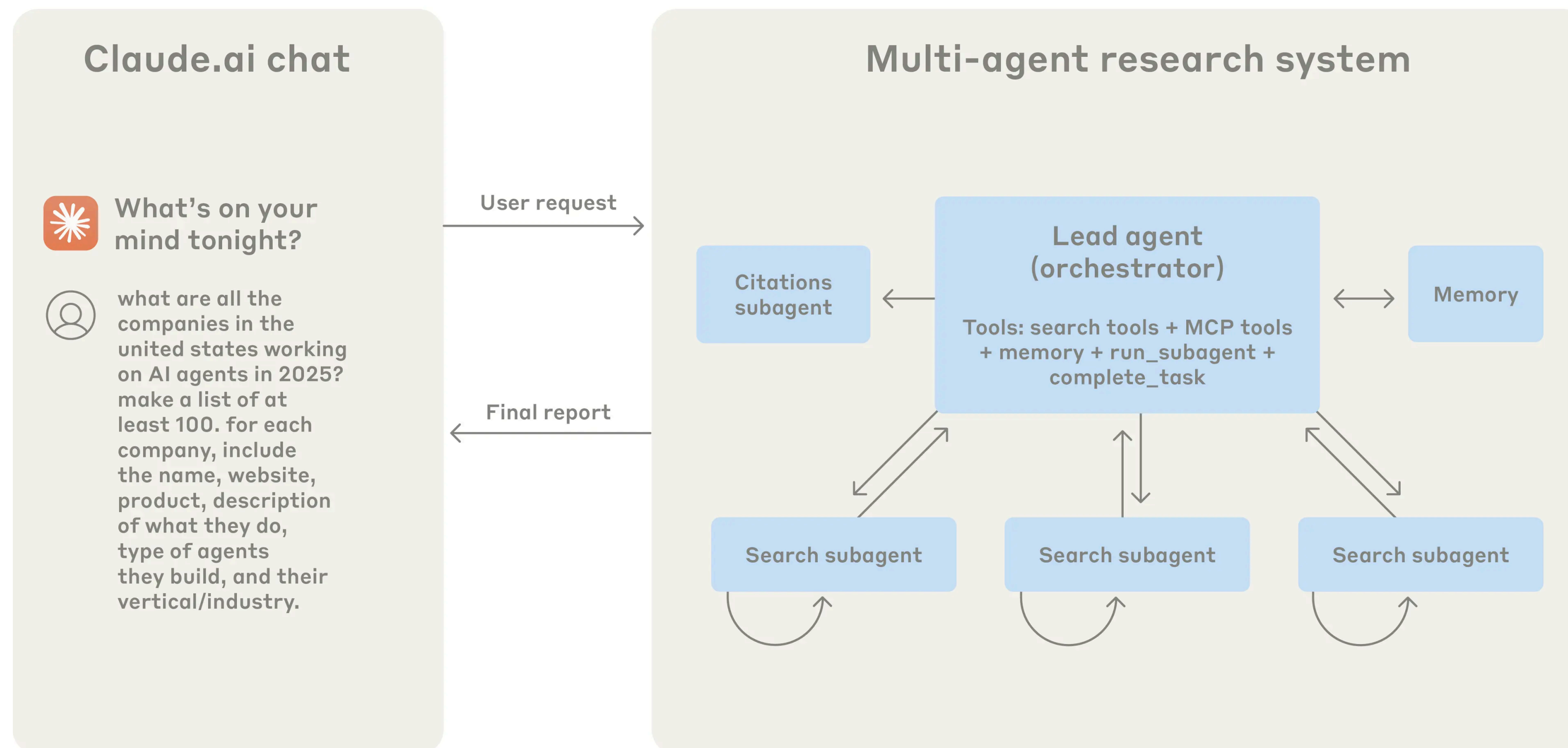
- Specialist models (e.g. image segmentation models)
- Compute environments (e.g. Python code, Wolfram Alpha, ...)
- Databases (e.g. internet access, vector databases, ...)
- Memory (e.g. summaries of previous predictions)
- Other LLMs (e.g. more expansive ones, specialists)

AGENTS / TOOL USE

Example: Anthropic's Research Agent

<https://www.anthropic.com/engineering/built-multi-agent-research-system>

High-level Architecture of Advanced Research



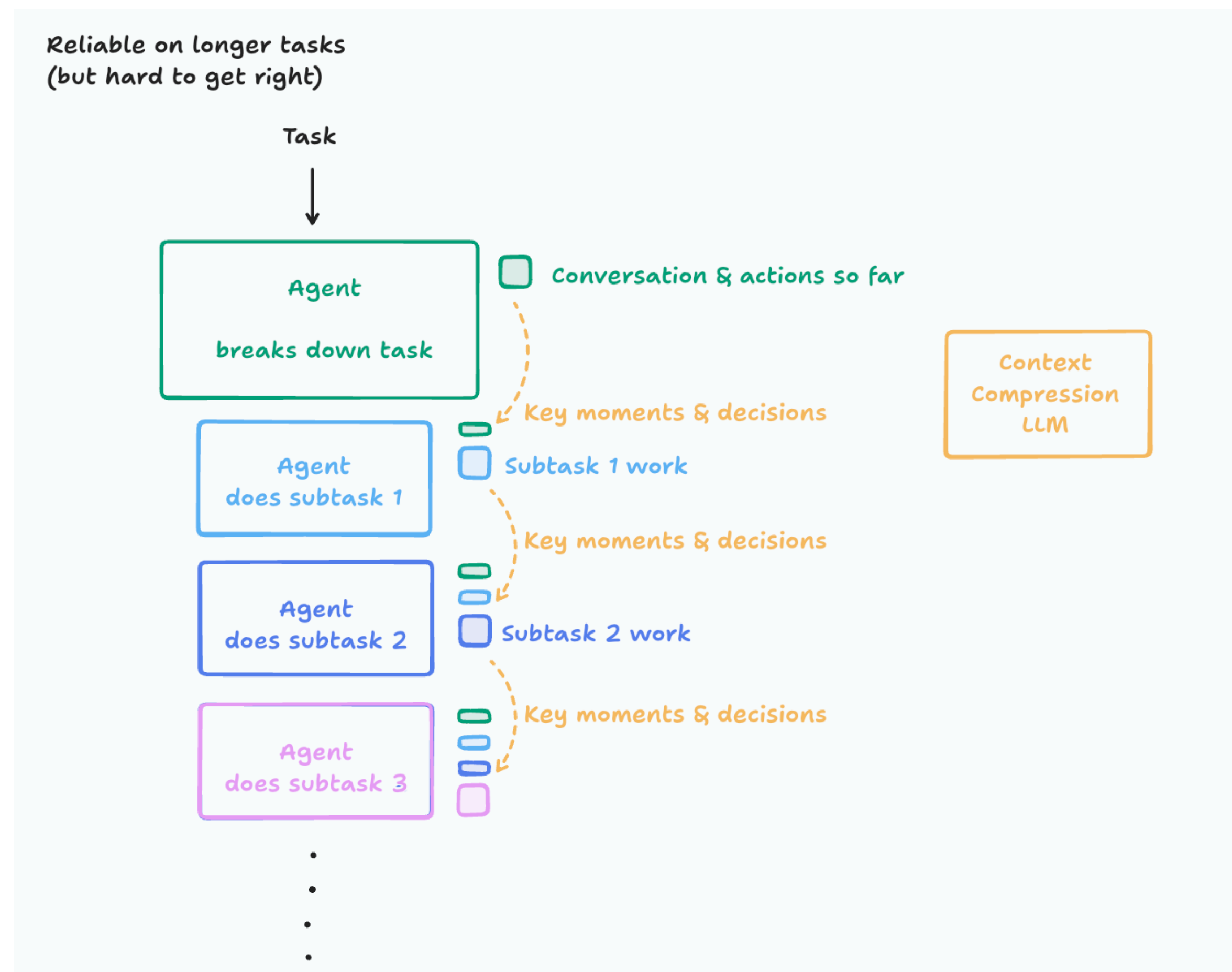
Orchestrator-Worker Pattern

- Orchestrator (lead agent) coordinates the process
- Specialized agents do the actual work in parallel

AGENTS / TOOL USE

Example: Cognition's view on Agents

<https://cognition.ai/blog/dont-build-multi-agents#applying-the-principles>



Don't parallelize (too much):
sub-agents may interpret the task differently
and it is hard to combine results afterwards

Principle 1:

Share context, and share full agent traces, not just individual messages

Principle 2:

Actions carry implicit decisions, and conflicting decisions carry bad results

PREDICTION FUSION

Pro

- Unimodal models can be reused
- No paired training data required (inference-stage combination)
- New modalities can be added easily
- Certainties of predictions may be considered

Contra

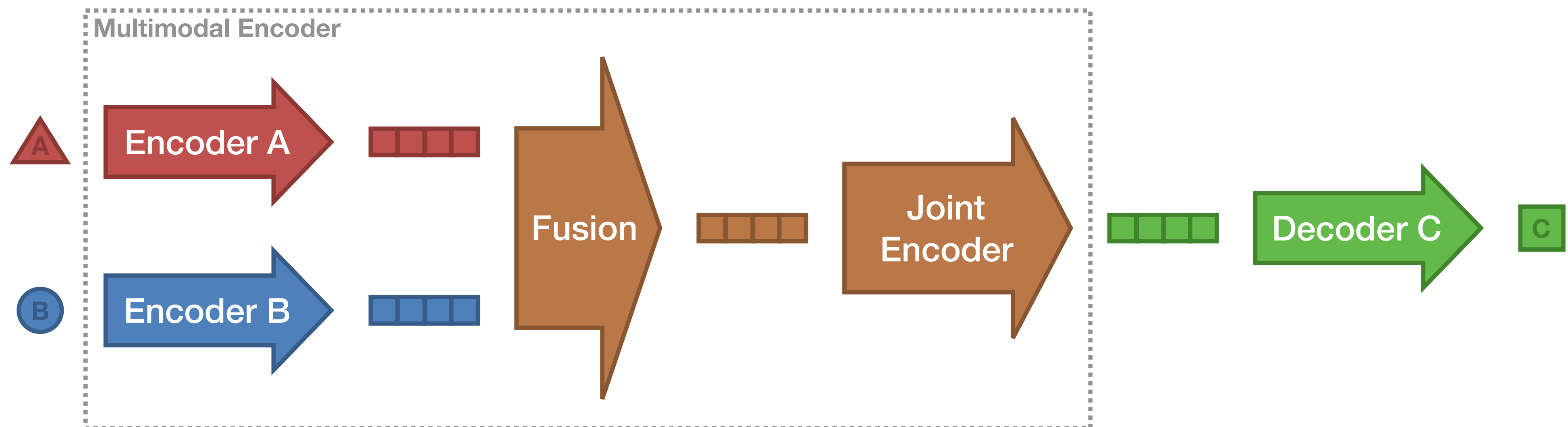
- No interaction of modalities, only of their predictions
- Fusion may be non-trivial for complex outputs

WHEN TO FUSE

little bit obsolete

Representation Fusion (Early Fusion or Feature-level Fusion)

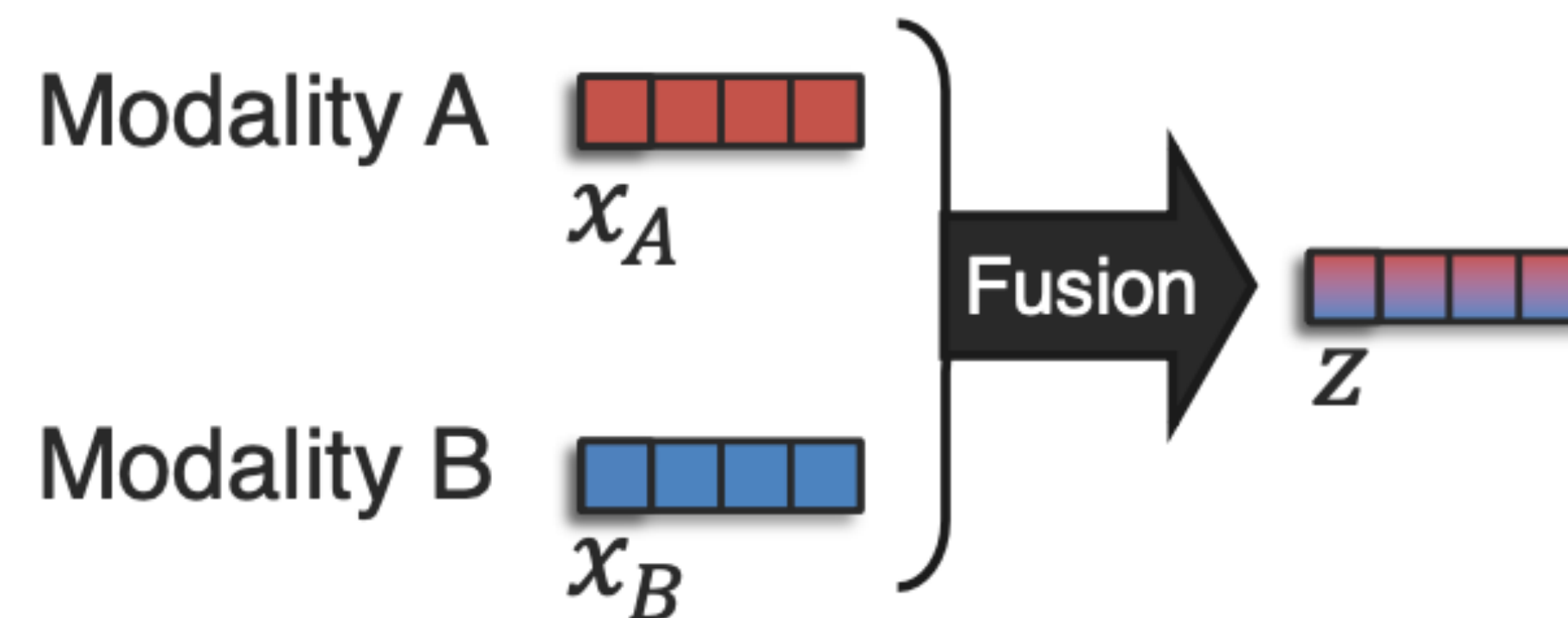
- Individual encoder models for each modality
 - Each directly predicts modality representations
- Fusion of modality features and joint encoder on these features
- Decoder/predictor of output modality



REPRESENTATION FUSION OPERATORS

Given

- Multiple modalities (A, B, C, ...)
- Each modality is represented by a single vector ($\mathbf{x}_A \in \mathbb{R}^{d_A}$, $\mathbf{x}_B \in \mathbb{R}^{d_B}$, $\mathbf{x}_C \in \mathbb{R}^{d_C}$, ...)
 - For now we assume each modality is encoded as a single vector (no structure)
- Compute a joint representation vector $\mathbf{z} \in \mathbb{R}^{d_z}$ by combining modality representations

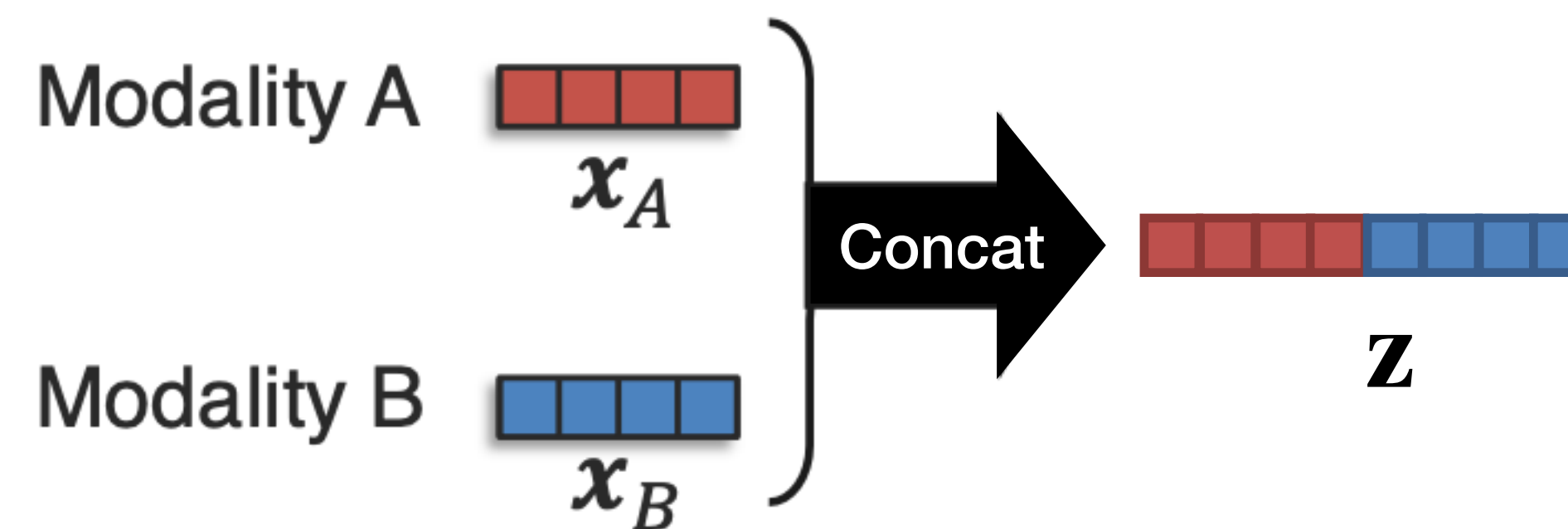


REPRESENTATION FUSION: CONCATENATION

Concat Fusion

- Concatenate vector representations of modalities ($\mathbf{x}_A, \mathbf{x}_B$)
- Feature dimensions d_A, d_B may be different
- Output dimension $d_z = d_A + d_B$

$$\mathbf{z} = \mathbf{x}_A \oplus \mathbf{x}_B = \begin{bmatrix} \mathbf{x}_A \\ \mathbf{x}_B \end{bmatrix}$$



$$\mathbf{z} = \mathbf{w}_A \mathbf{x}_A + \mathbf{w}_b \mathbf{x}_B = \mathbf{W} \begin{bmatrix} \mathbf{x}_A \\ \mathbf{x}_B \end{bmatrix}$$

REPRESENTATION FUSION: CONCATENATION

Concat Fusion with Linear Projection

better in practice

- Concatenate vector representations of modalities $(\mathbf{x}_A, \mathbf{x}_B)$ then project with linear projection $\mathbf{W}$
- Feature dimensions d_A, d_B may be different
- Output dimension d_z can be defined as needed: $\mathbf{W} \in \mathbb{R}^{d_z \times (d_A + d_B)}$

$$\mathbf{z} = \mathbf{W} \begin{bmatrix} \mathbf{x}_A \\ \mathbf{x}_B \end{bmatrix}$$

REPRESENTATION FUSION: CONCATENATION

Concat Fusion with Linear Projection

- Can also be written with individual linear projections applied to individual modality features

$$\mathbf{W}_A \in \mathbb{R}^{d_z \times d_A}, \mathbf{W}_B \in \mathbb{R}^{d_z \times d_B} \quad \mathbf{x}'_A = \mathbf{W}_A \mathbf{x}_A, \quad \mathbf{x}'_B = \mathbf{W}_B \mathbf{x}_B$$

$$\mathbf{z} = \mathbf{W} \begin{bmatrix} \mathbf{x}_A \\ \mathbf{x}_B \end{bmatrix} = \mathbf{W}_A \mathbf{x}_A + \mathbf{W}_B \mathbf{x}_B = \mathbf{x}'_A + \mathbf{x}'_B$$

**This is just the (elementwise) sum of per-modality feature vectors,
i.e.**

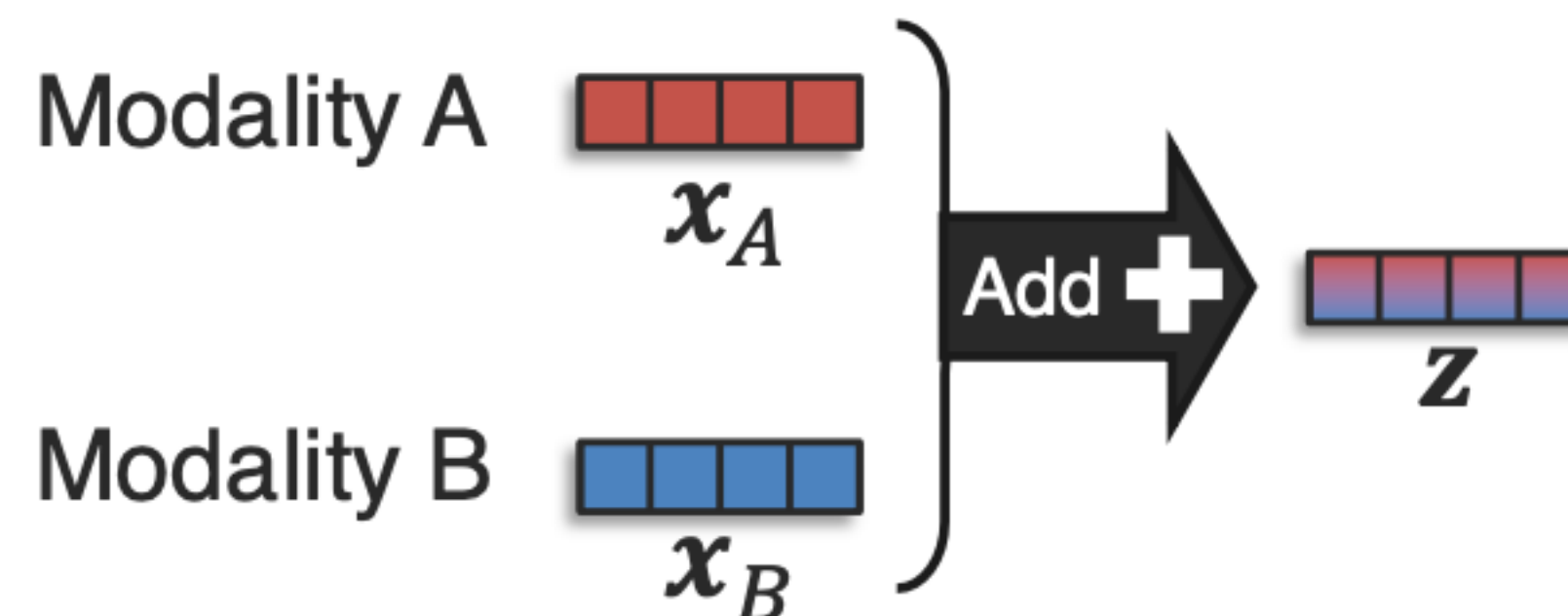
Additive Fusion

REPRESENTATION FUSION: ADDITIVE FUSION

Additive Fusion

- Elementwise sum of vectors of different modalities (2 or more)
- Feature dimensions need to be the same ($d_A = d_B$)
 - But can be projected before (e.g. using linear projection or MLP)
- Output dimension stays the same ($d_z = d_A = d_B$)
- Feature dimensions keep their meaning

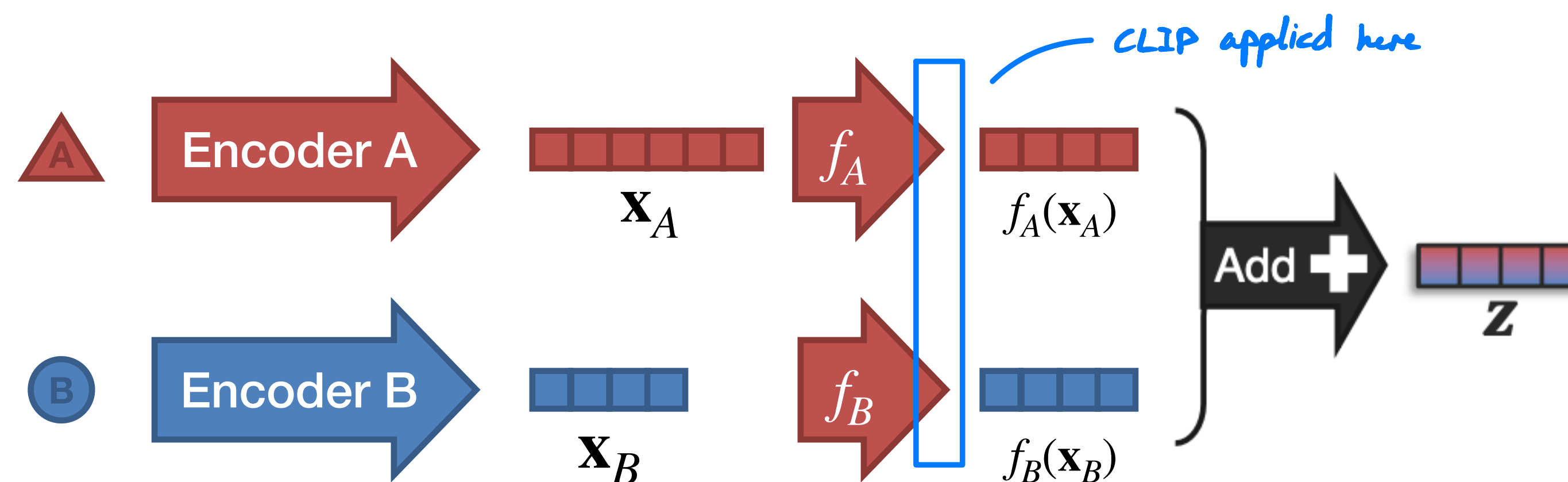
$$\mathbf{z} = \mathbf{x}_A + \mathbf{x}_B$$



REPRESENTATION FUSION: ADDITIVE FUSION

Additive Fusion in Practice

- Modality encoders produce per-modality features $\mathbf{x}_A \in \mathbb{R}^{d_A}, \mathbf{x}_B \in \mathbb{R}^{d_B}$
- Individual projectors per modality: $f_A: \mathbb{R}^{d_A} \rightarrow \mathbb{R}^{d_z}, f_B: \mathbb{R}^{d_B} \rightarrow \mathbb{R}^{d_z}$
 - Linear projections or small MLPs
 - Modality dimensions d_A, d_B may be different but are mapped to same output dimension d_z
- Additive fusion of projected features: $\mathbf{z} = f_A(\mathbf{x}_A) + f_B(\mathbf{x}_B)$



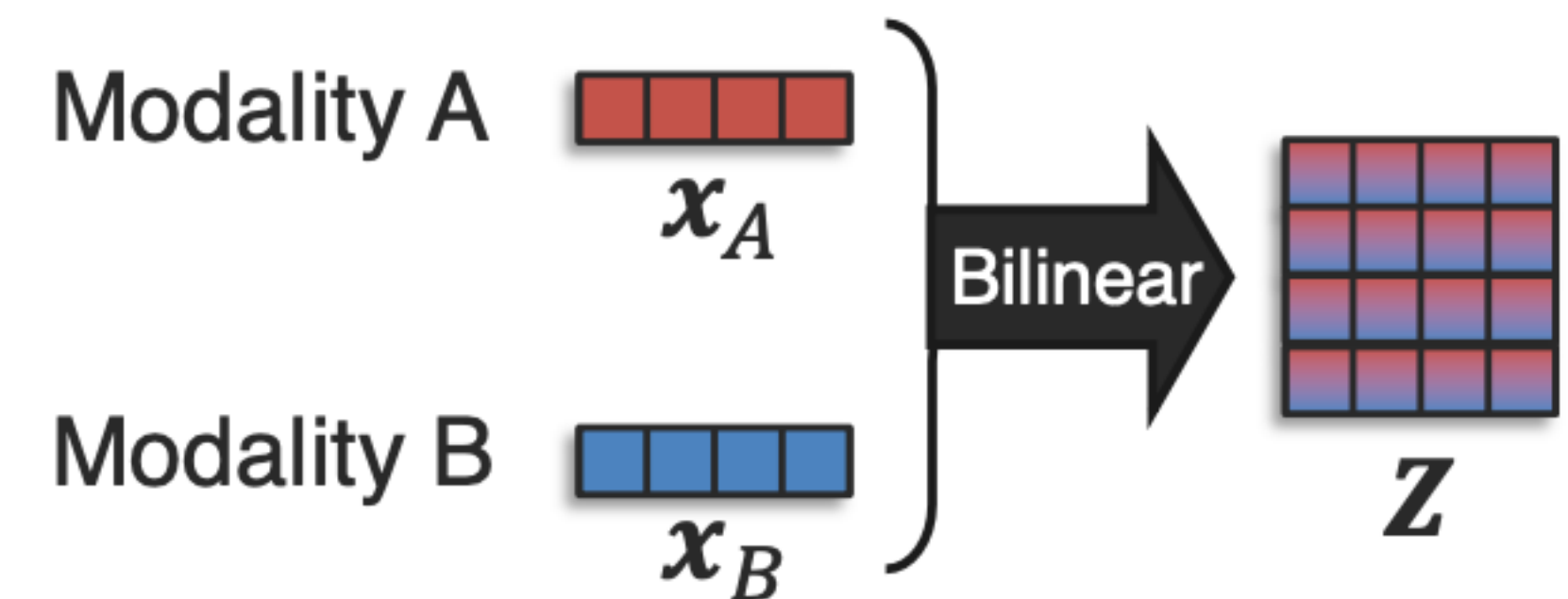
What if we want more complex interactions?

REPRESENTATION FUSION: MULTIPLICATIVE

Bilinear Fusion

- Outer product of vectors of exactly two modalities
 - Multiplicative interactions (pairwise)
 - Modality feature dimension may differ
 - Output dimension $d_z = d_A \times d_B$
 - The output is now a matrix: $\mathbf{z} \in \mathbb{R}^{d_A \times d_B}$
 - Rows represent feature dimension of modality A
 - Columns represent feature dimensions of modality B
- ➡ Interpretable dimensions derived from input dimensions!

$$\mathbf{z} = \mathbf{x}_A \otimes \mathbf{x}_B = \mathbf{x}_A \mathbf{x}_B^T$$



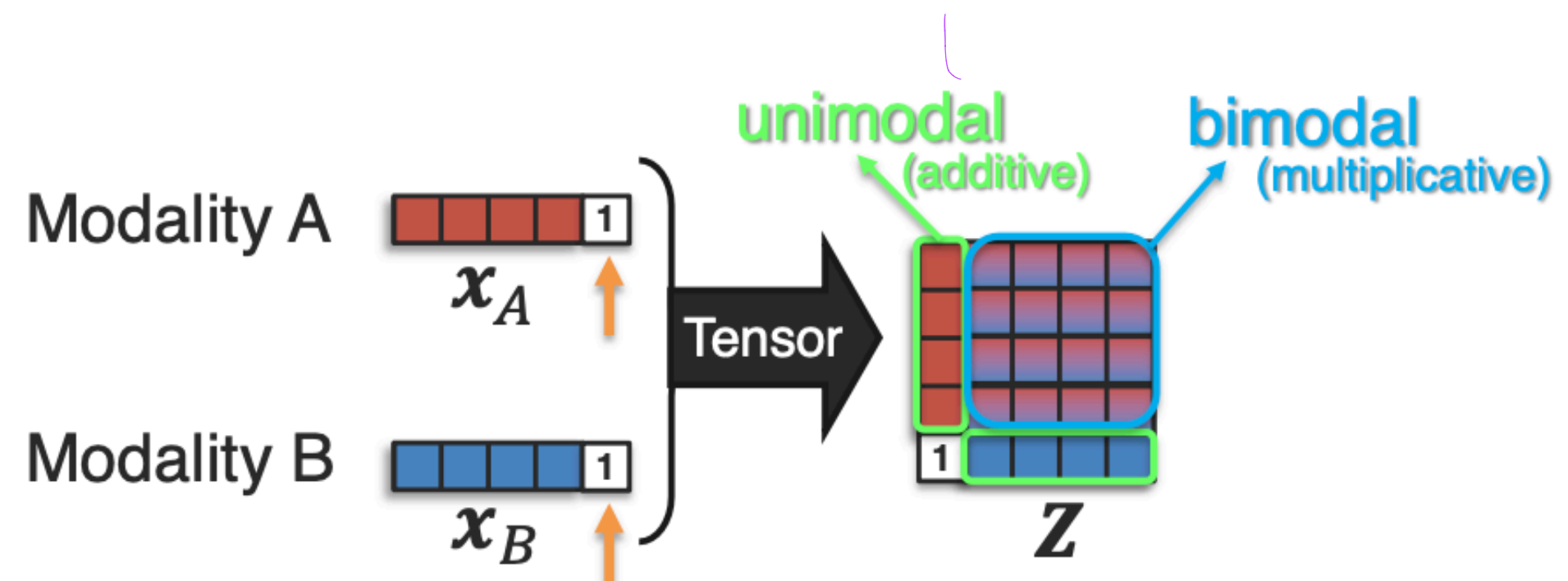
What if we have more than two modalities?

REPRESENTATION FUSION: TENSOR FUSION

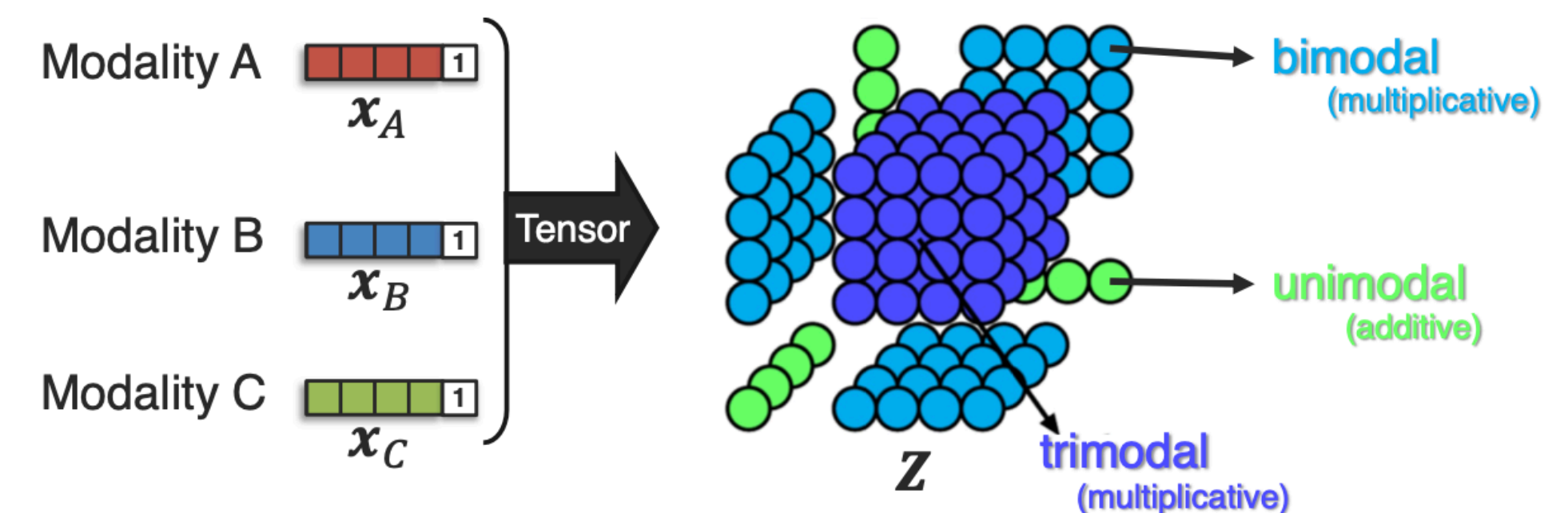
Tensor Fusion

- Generalization of bilinear fusion to any number of modalities
- Output is the tensor product of per-modality feature vectors
 - Unimodal: additive interactions
 - Bimodal: Pairwise multiplicative (bilinear)
 - Trimodal: Rank-3 multiplicative (trilinear)
- Concat 1 to each vector to also include lower rank interactions

$$\mathbf{z} = \bigotimes_{m=1}^M \begin{bmatrix} \mathbf{x}_m \\ 1 \end{bmatrix}$$



$$\mathbf{z} = \begin{bmatrix} \mathbf{x}_A \\ 1 \end{bmatrix} \otimes \begin{bmatrix} \mathbf{x}_B \\ 1 \end{bmatrix}$$



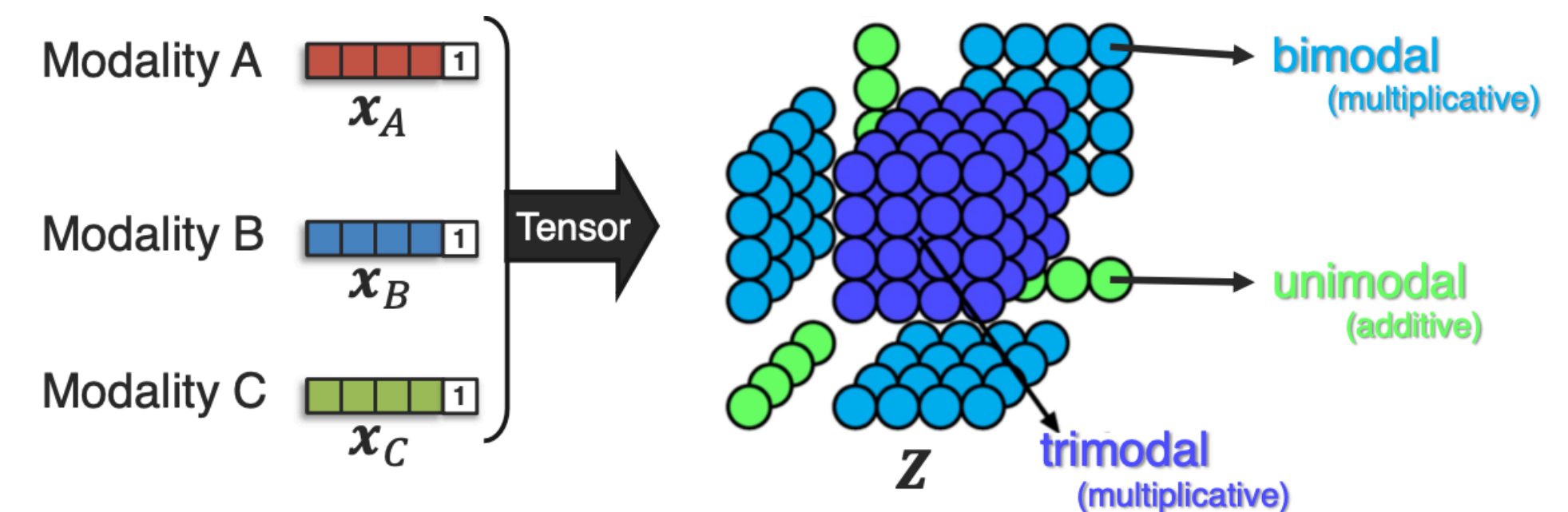
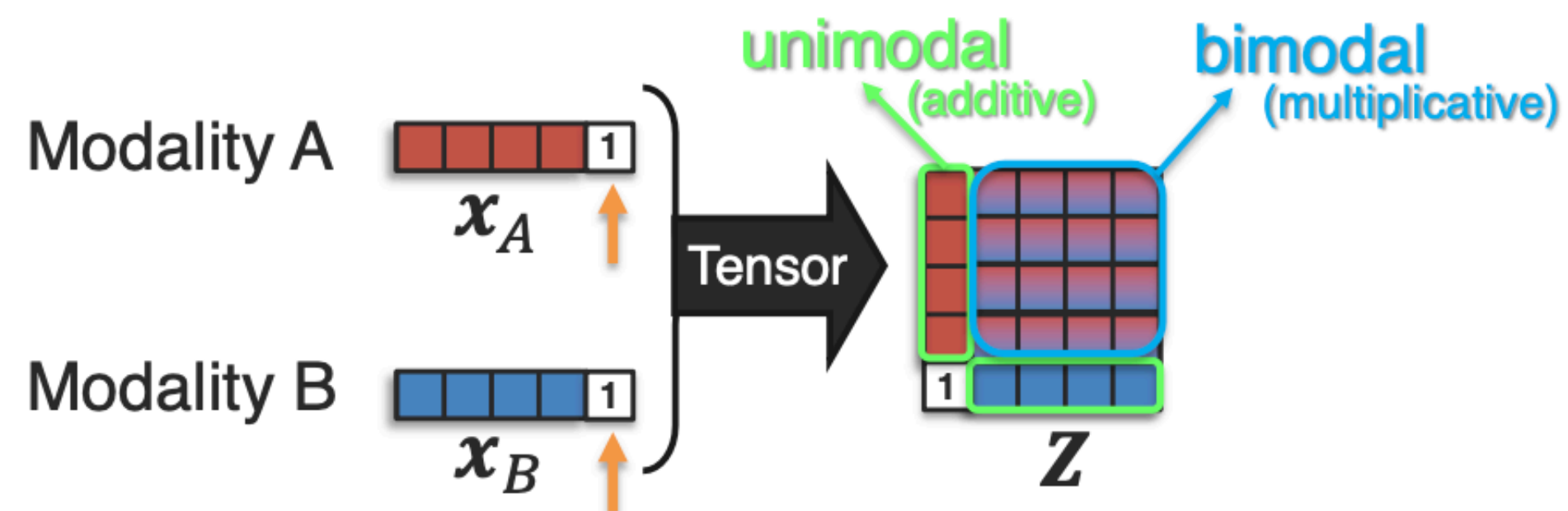
$$\mathbf{z} = \begin{bmatrix} \mathbf{x}_A \\ 1 \end{bmatrix} \otimes \begin{bmatrix} \mathbf{x}_B \\ 1 \end{bmatrix} \otimes \begin{bmatrix} \mathbf{x}_C \\ 1 \end{bmatrix}$$

REPRESENTATION FUSION: TENSOR FUSION

Tensor Fusion

- Input dimension may differ
- Output dimension $d_z = (d_A + 1) \times \dots \times (d_M + 1)$
 - For M modalities, the output is an order- M tensor
 - ➔ Interpretable dimensions derived from input dimensions!
 - **BUT: d_z scales exponentially with number of modalities**

$$\mathbf{z} = \bigotimes_{m=1}^M \begin{bmatrix} \mathbf{x}_m \\ 1 \end{bmatrix}$$

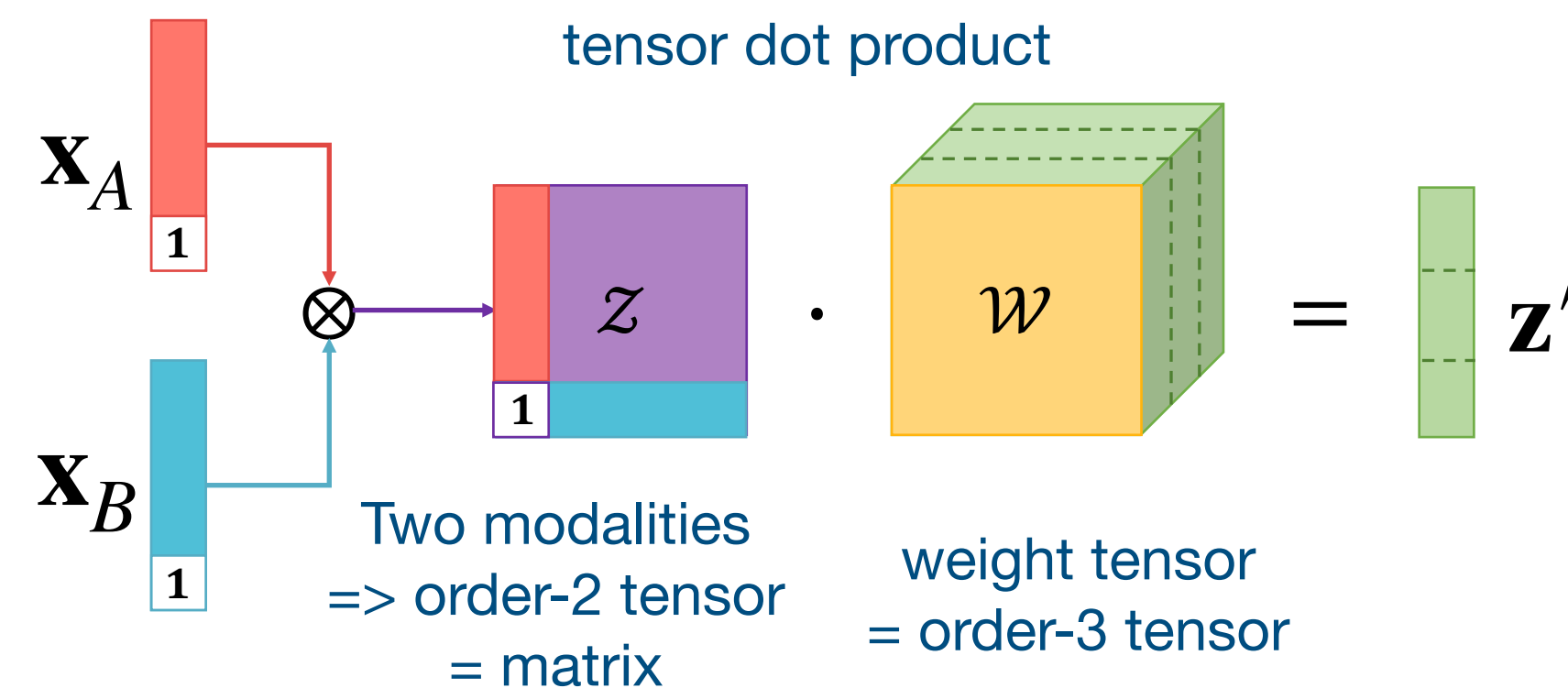


REPRESENTATION FUSION: TENSOR FUSION

Tensor Fusion in Practice: Project

- Output tensor $\mathbf{z} \in \mathbb{R}^{d_1 \times d_2 \times \dots \times d_M}$ is multiplied by learned weight tensor $\mathbf{W}$
 - Weight matrix is rank $M + 1$ tensor: $\mathbf{W} \in \mathbb{R}^{d_1 \times d_2 \times \dots \times d_M \times d_z}$
 - Final output $\mathbf{z}' = \mathbf{z} \cdot \mathbf{W} \in \mathbb{R}^{d_z}$ is vector of definable dimension

$$\mathbf{z} = \bigotimes_{m=1}^M \begin{bmatrix} \mathbf{x}_m \\ 1 \end{bmatrix}$$



REPRESENTATION FUSION

Pro

- Pre-trained unimodal encoders can be (partially) used
- Interactions btw modalities at the representation level

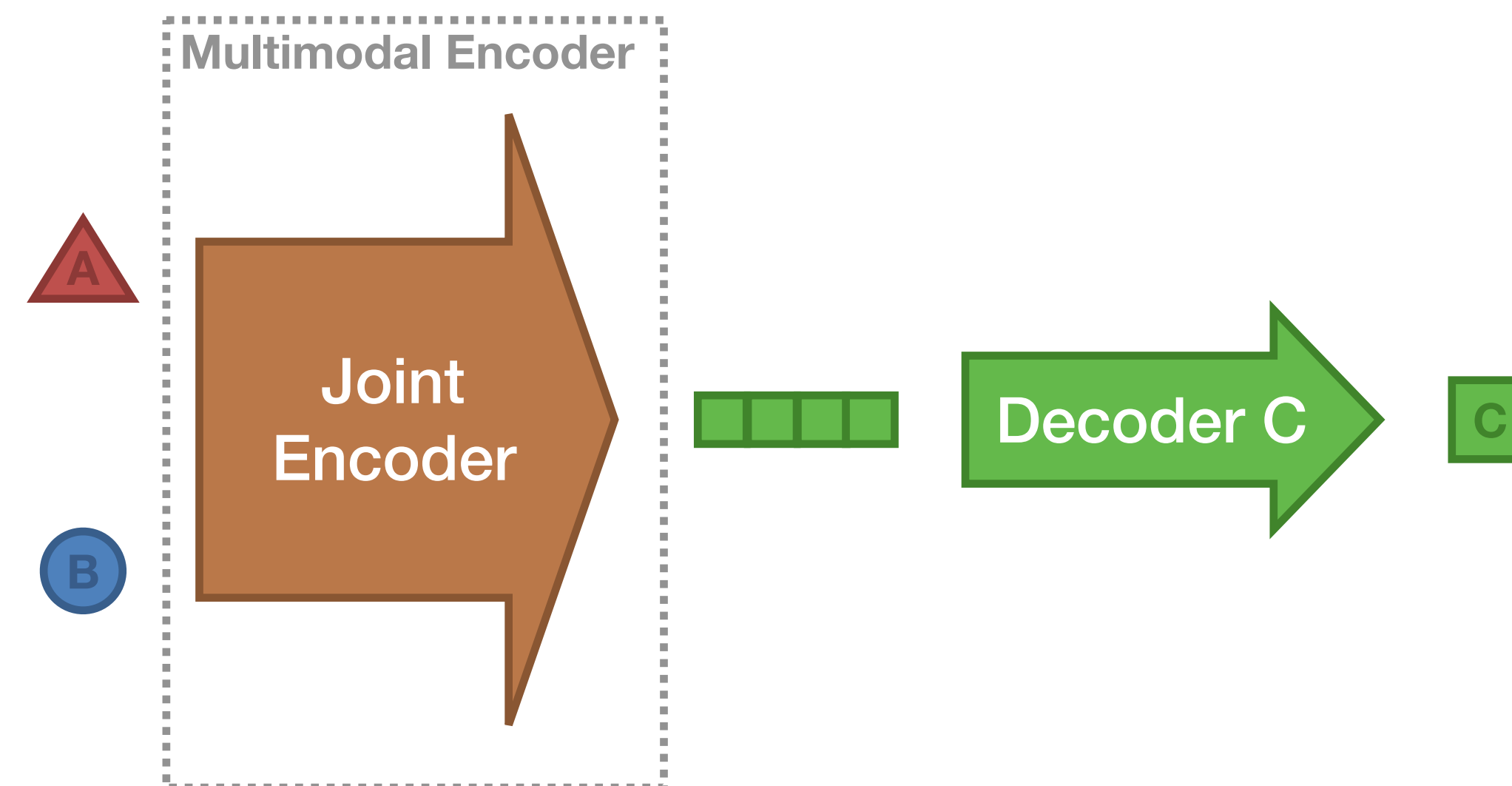
Contra

- Paired training data required
- Adding new modalities requires alignment of new modality encoder

WHEN TO FUSE

Input Fusion (Very Early Fusion or Data-level Fusion)

- Single encoder that directly encodes all modalities
 - Encoder needs to be able to handle different modalities
 - Setup: Highly dependent on modalities and encoder architecture
- No uni-modal features
- No explicit fusion operation required



INPUT FUSION: SIMPLE EXAMPLES

Multi-Image Fusion: All input modalities are images

1. Register images and resize them to the same resolution
 - Align images spatially (associate pixels with each other / overlay them)
2. Concatenate image channels
 - Concatenate individual input images in the channel dimension
 - Now multiple image modalities are represented as a single multi-channel image
 - E.g. 2 RGB images => 6 channels
3. Encode using image encoder architecture
 - E.g. ConvNet, ViT, or similar

Notes:

- No non-image modalities supported
- Images need to be (roughly) aligned/registered and rescaled/padded to the same dimensions

INPUT FUSION: SIMPLE EXAMPLES

Multi-Text Fusion: All input modalities are text

1. Integrate multiple text inputs in a single prompt
2. Encode using text model
 - E.g. an LLM or other language model

Notes:

- No non-text modalities supported
- Quality dependent on prompt
- Also see Agents/Tool Use

INPUT FUSION WITH SET / SEQUENCE ENCODERS

Treat all input modalities as set or sequence inputs

1. Tokenize and embed each input modality into a set/sequence of tokens
 - Modality-specific tokenizers and embedding layers can be used
 - Modality-specific “position” embeddings can also be added
2. Concatenate token sets/sequences into a multimodal token set/sequence
3. Encode using set/sequence encoder
 - E.g. a transformer model

Notes:

- No modality-specific encoders can be used
- Modality-specific tokenizers/embedders are required
- May result in long token sequences => computationally expensive

Excuse:

Input

Tokenization and

Embedding

REMINDER: EVERYTHING CAN BE REPRESENTED AS A SET

(Slides 101-108 from Sets/Sequences/Tuples Lecture)

Takeaways to encode any data type as a set

- Define elements (i.e. tokens), e.g. words, columns, words ...
- Embed element features
- Encode structure, e.g. positions, 2D space, column types, ...

➡ **Select the appropriate elements, embeddings, structure encodings for your use case!**

MODALITIES AS SET/SEQUENCE INPUTS

To encode a modality as a set or sequence inputs, we need:

1. Tokenizer

- Split input sequence into individual elements (the tokens)
- Either as **discrete (hard) tokens**, i.e. each token is picked from **fixed set of available tokens** / token IDs
- Or as **continuous (soft) tokens**, i.e. each token is **a vector, typically projected from the raw input**

2. Embedding Layer

- Encode each token into a vector
- Typically learned end-to-end
- May be shared or distinct for different modalities

3. Structure Embeddings

MODALITIES AS SET/SEQUENCE INPUTS: TEXT

Let's prepare: Text

1. Tokenizer

- Byte-pair-encoding (BPE) or similar (learn the vocabulary)
- Splits text into words or sub-words as needed
- Character-level tokenization also possible but rarely used
- **Discrete tokens!**

2. Embedding Layer

- Learn embedding for each token in the tokenizer vocabulary

3. Structure Embeddings

- Positional embeddings
- Optional: special tokens before/after/within text (e.g. chat turns)

MODALITIES AS SET/SEQUENCE INPUTS: IMAGES

Let's prepare: Images (as soft tokens)

1. Tokenizer

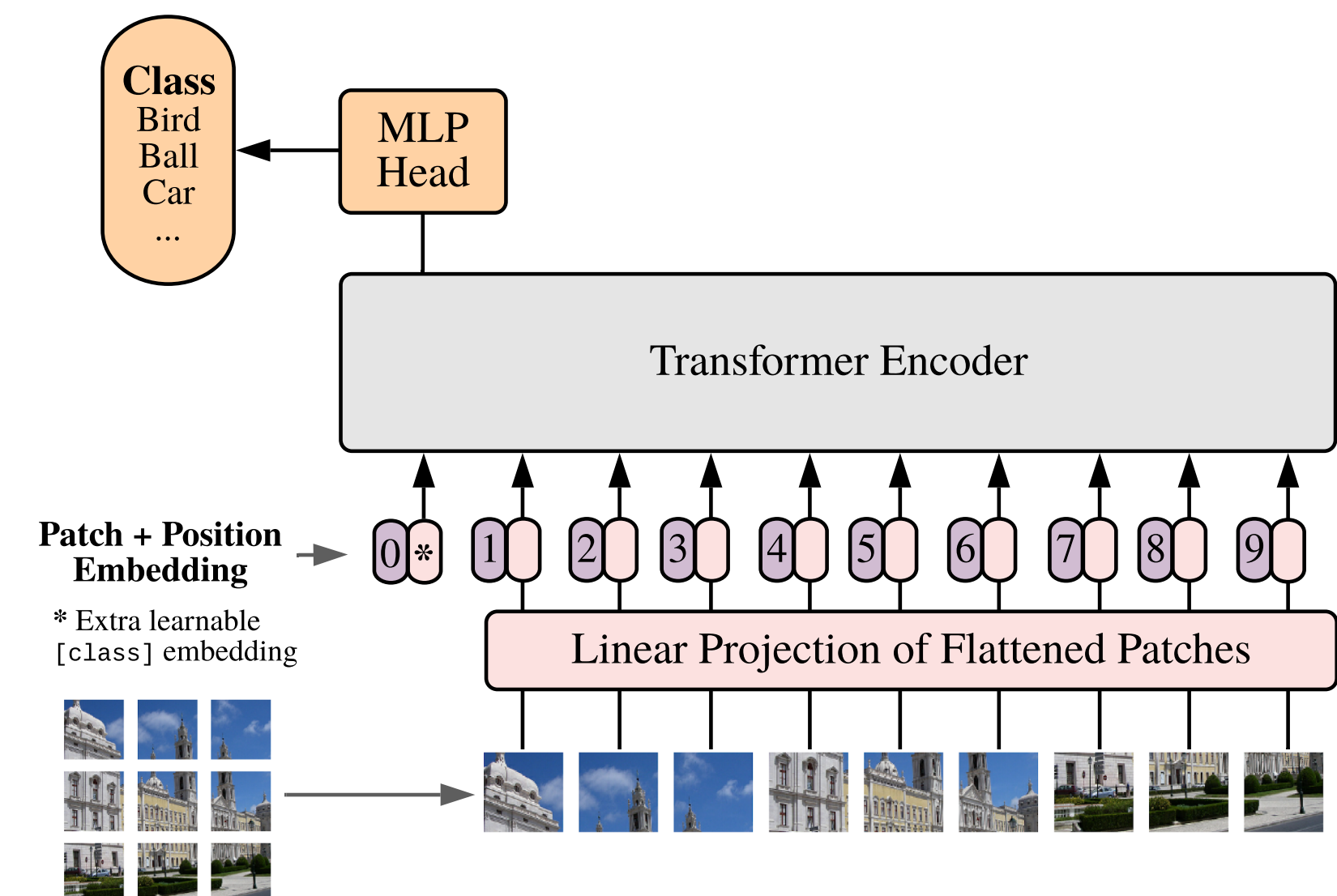
- Split image into patches

2. Embedding Layer

- Project flattened pixels per patch (learned linear projection)

3. Structure Embeddings

- 2D positional embeddings



Can we also use discrete tokens for images?

REMINDER: VQ-VAES

Vector Quantized VAEs

- **Goal:** learn a discrete, high-fidelity latent space (*i.e.* codebook/vocabulary).
 - Better for generation (*avoids averaging affect of continuous representations*).
 - Use as tokens for other (generative) models.

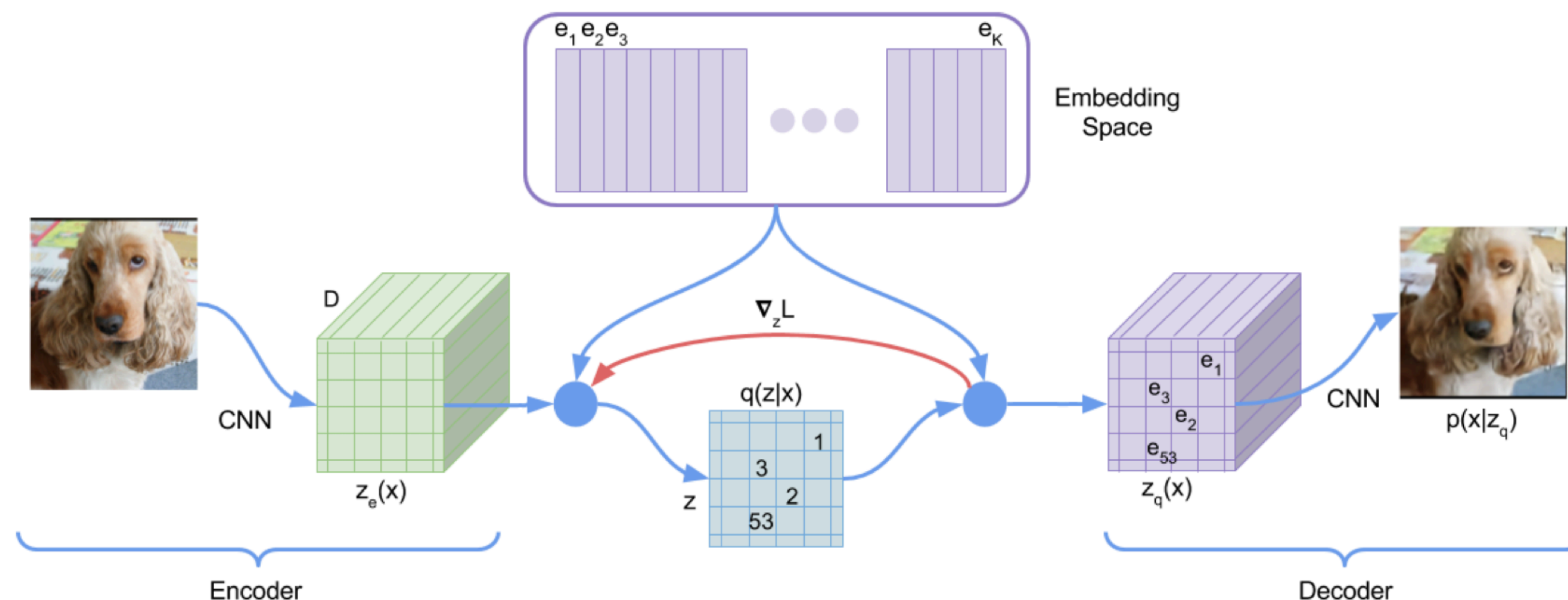
• **Definition:** $L_{VQ-VAE} = \log p(x | z_q(x)) + ||\text{sg}[z_e(x)] - e||_2^2 + \beta ||z_e(x) - \text{sg}[e]||_2^2,$

reconstruction loss
updates decoder

codebook loss
updates codes

stop
gradient

commitment loss
updates encoder

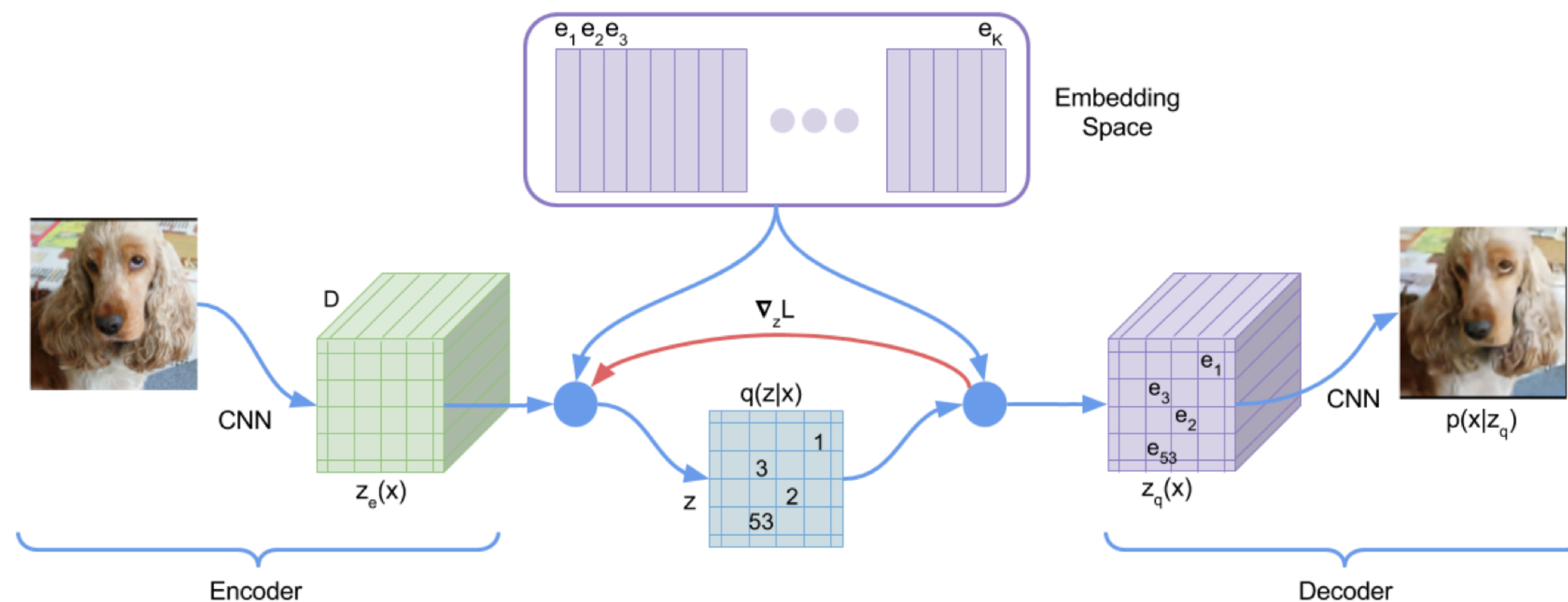


MODALITIES AS SET/SEQUENCE INPUTS: IMAGES

VQ-VAE tokenizer

1. Encode image into patch features (feature map)
2. Discretize patch features
 - **Codebook**: Learn fixed-size set of **codebook** vectors
 - **Quantize**: For each patch feature, pick the closest codebook vector
3. Decode (reconstruct) discretized patch features to image

Other variants:
VQ-GAN



MODALITIES AS SET/SEQUENCE INPUTS: IMAGES

Let's prepare: Images (as discrete tokens)

1. Tokenizer

- VQ-VAE (or VQ-GAN)
- Discrete codebook token per patch

2. Embedding Layer

- Learn embedding for each token in the codebook vocabulary

3. Structure Embeddings

- 2D positional embeddings

Benefits:

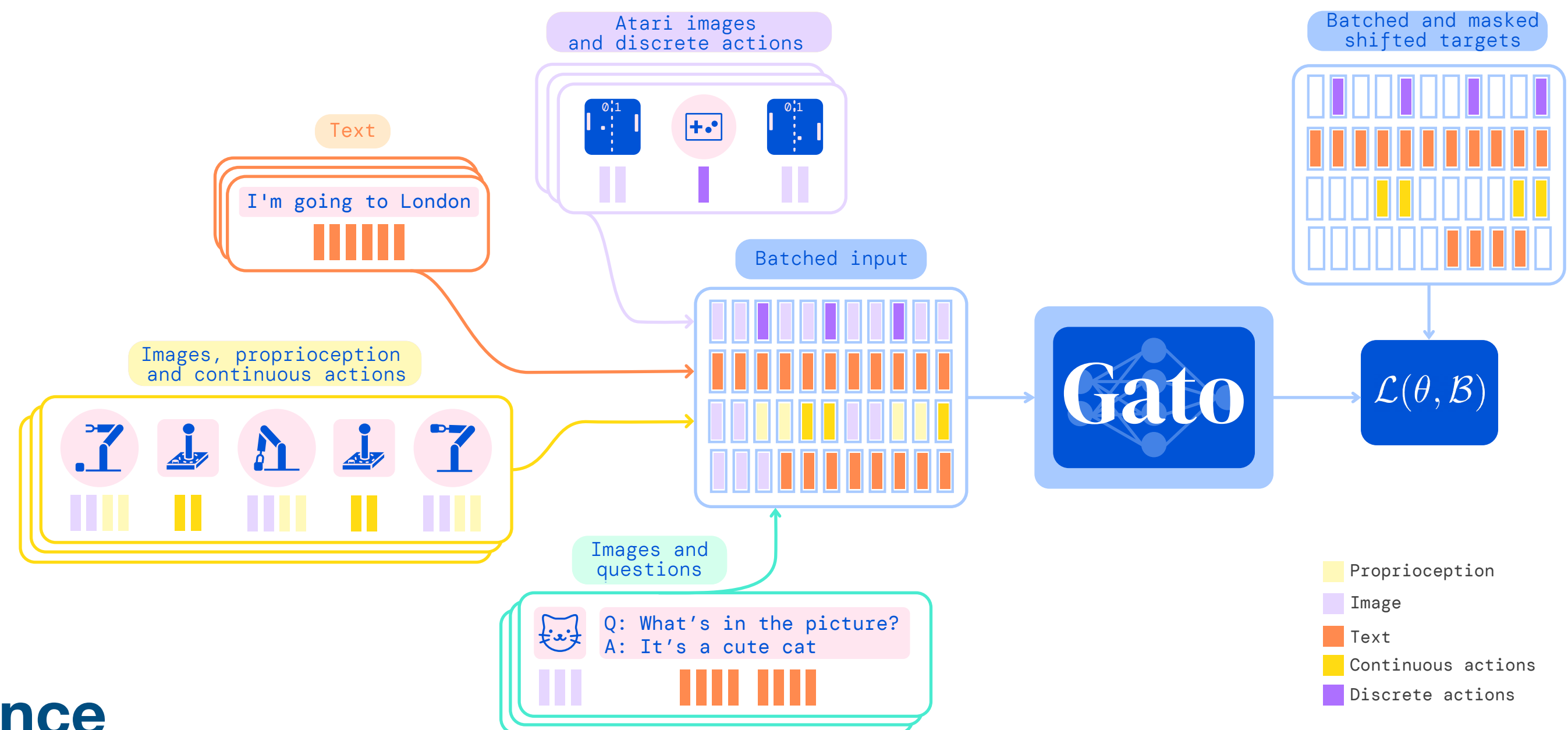
- Cross-entropy loss can be used
- Can be used for image generation

MULTIMODAL INPUT SEQUENCE

Now let's mix tokens from different modalities as needed: **Gato**

Modality-specific tokenizers

- Text tokens: Discrete
- Image tokens: Soft
- Proprioception: Discrete
 - Discretized by binning
- Continuous actions: Discrete
 - Discretized by binning
- Discrete actions: Discrete



Reed et al (2022)

Concatenate token embeddings sequence

- Mixing modalities within each sequence / batch

MULTIMODAL INPUT SEQUENCE

Now let's mix tokens from different modalities as needed: **Gato**

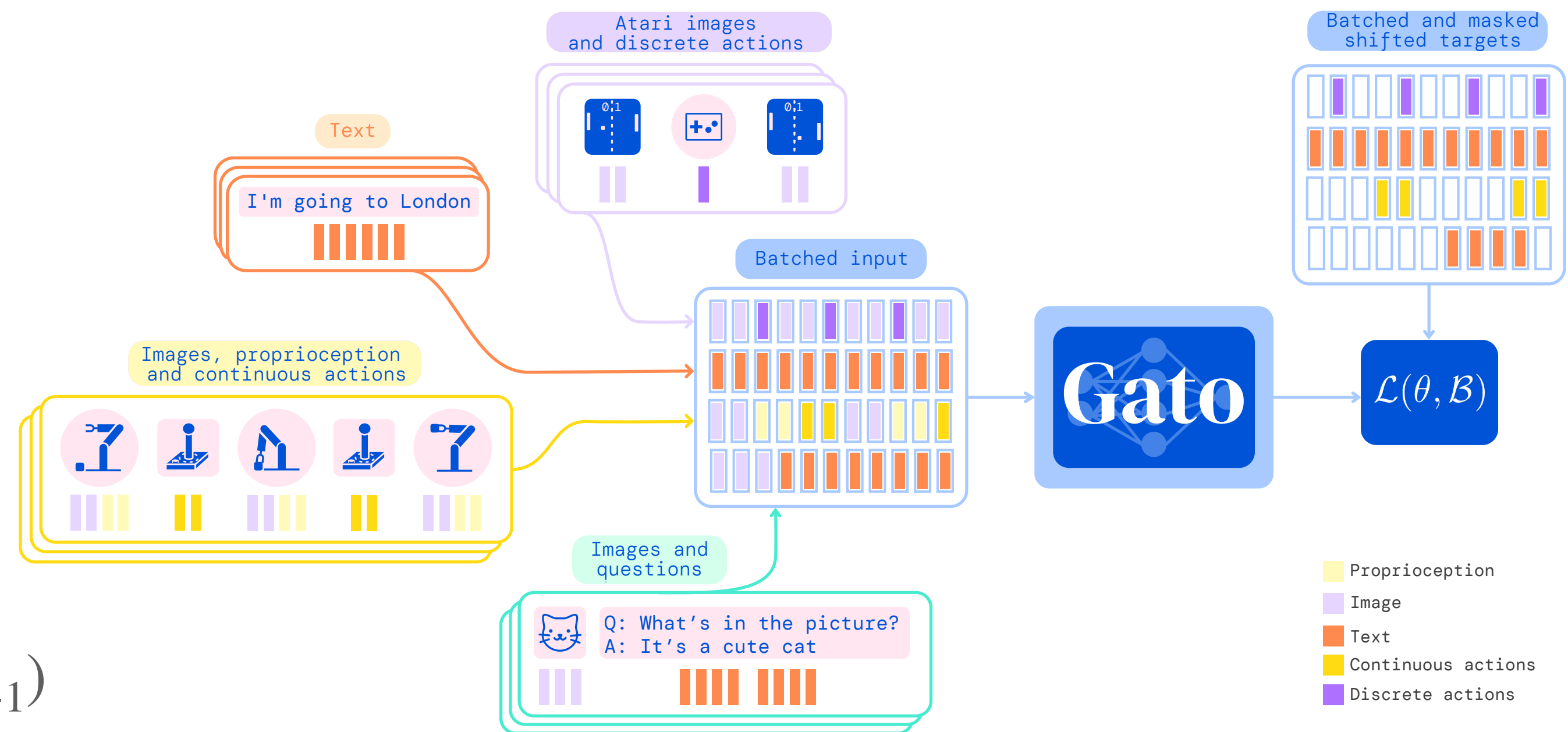
Concatenate token embeddings sequence

- Mixing modalities within each sequence / batch

Benefit:

Training can be unified for all tasks as **autoregressive data modeling** see Losses lecture

$$L_{\text{autoregressive}} = - \sum_{t=1}^N \log p_{\theta}(x_t | x_1, \dots, x_{t-1})$$



Reed et al (2022)

MULTIMODAL INPUT SEQUENCE

Other example: the 4M model

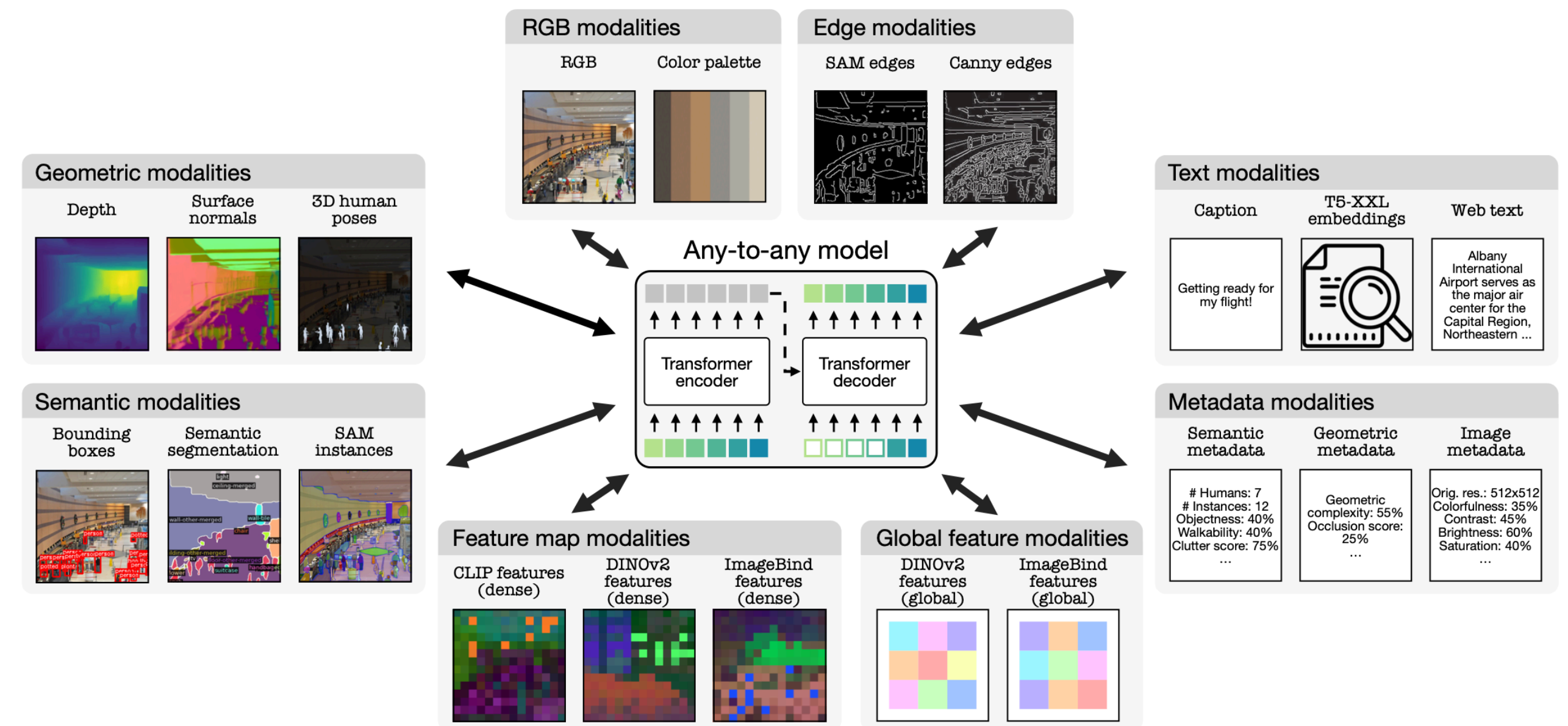
see Losses lecture slide 69

Tokenizers:

- Only discrete tokens for all modalities
 - Modality-specific tokenizers
 - E.g. VQ-VAE
- Shared/concatenated vocabulary

More details:

- Encoder-decoder architecture
- Masked data modeling



Mizrahi et al (2023)

INPUT FUSION

Pro

- No or only shallow modality encoders required
- Interactions btw modalities in all stages possible

Contra

- Paired training data required
- Adding new modalities requires re-training of multimodal model
- Input fusion of heterogeneous modalities is hard / requires special tokenization

WHEN TO FUSE: IS IT REPRESENTATION OR INPUT FUSION?

Assume:

Image tokens with embeddings from full image encoder (e.g. ViT),
mixed with text tokens from shallow embedding layers

Is this representation or input fusion:

- Image tokens are features from encoder => representation fusion?
- Text tokens are (shallow) input encodings => input fusion?

WHEN TO FUSE: IS IT REPRESENTATION OR INPUT FUSION?

Clear distinction is not always possible!

Modern approaches are often both:

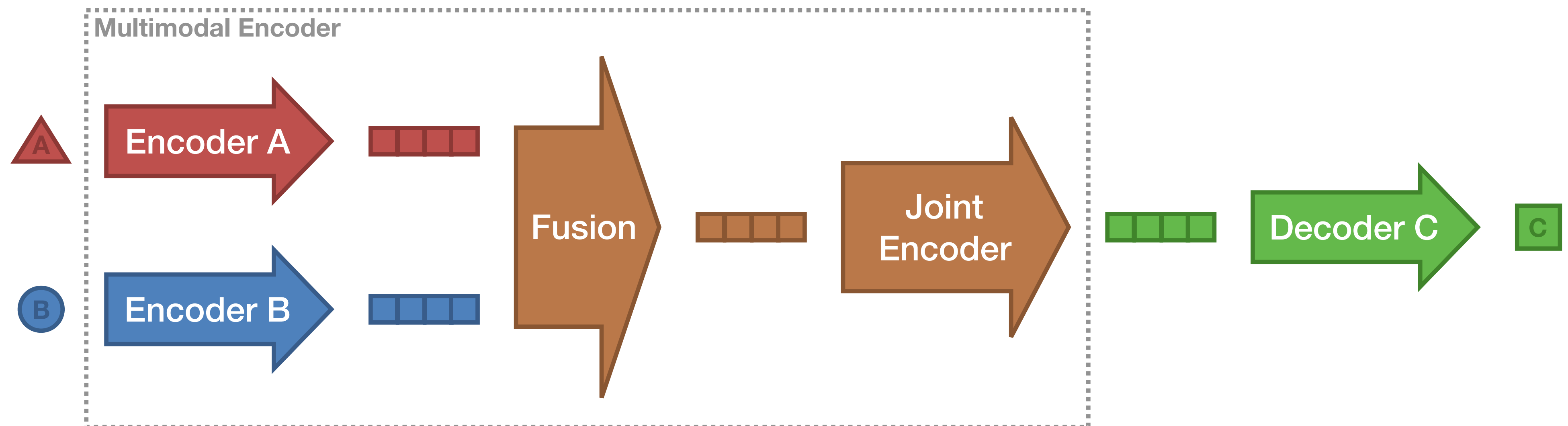
- We fuse representations in a shared space
- Encoders are more shallow or deep depending on the modality
- BUT: we still try to fuse them as early as possible

FUSION: WHAT ABOUT THE STRUCTURE?

Problem:

So far we ignored the structure of input modalities and simply fused *global* vectors per input modality

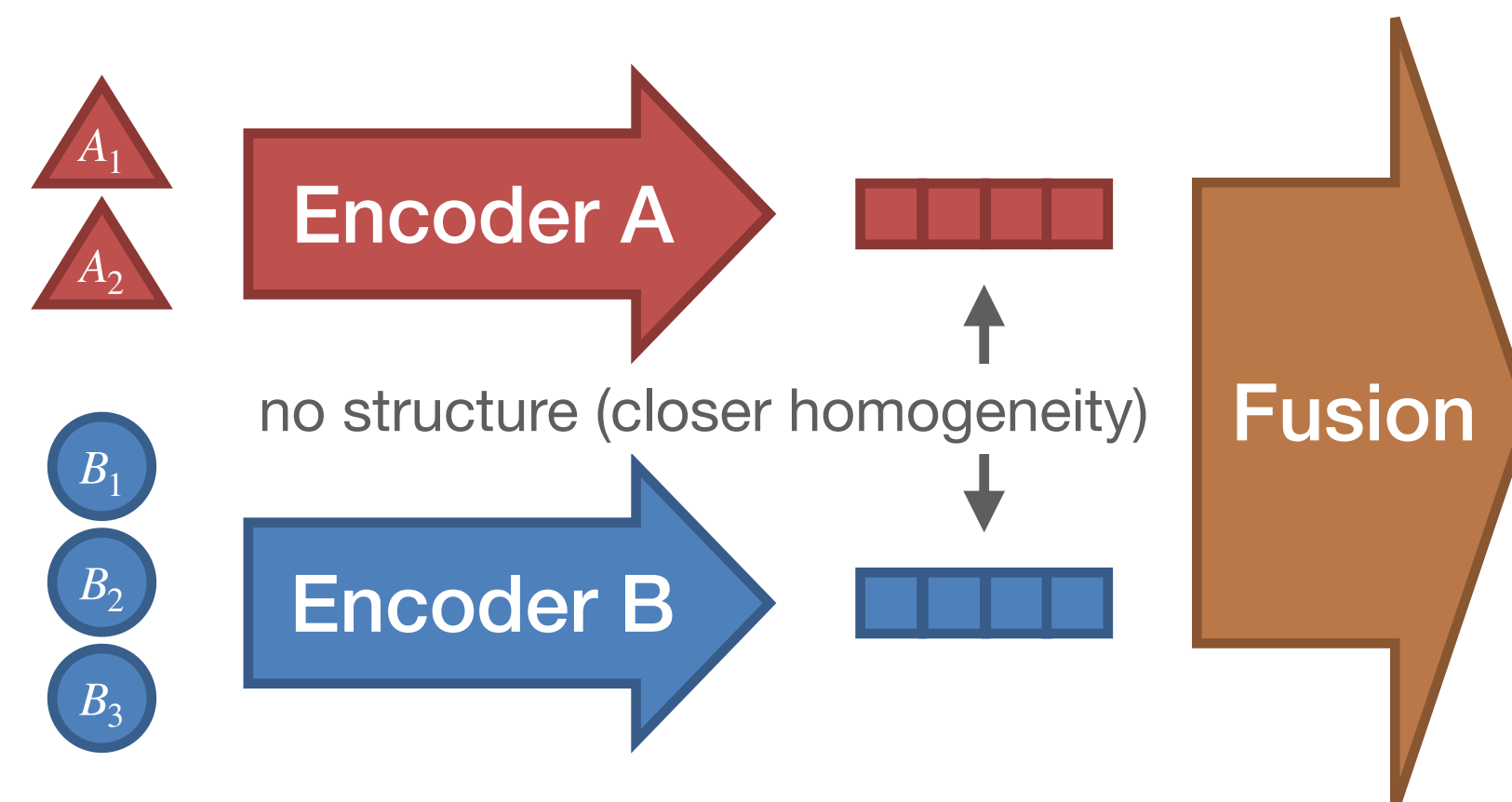
- Modality encoders encode each modality as a single vector
 - Any structure (grid, set, sequence, graph, ...) of the modality is lost
 - No interaction btw modalities possible within their structures
 - All information needs to be compressed into a single vector



FUSION: WHAT ABOUT THE STRUCTURE?

Representation fusion of global vectors does not consider structure of input modalities

- Modality encoders encode each modality as a single vector
- **Benefit: modality representations (features) are easy to combine**

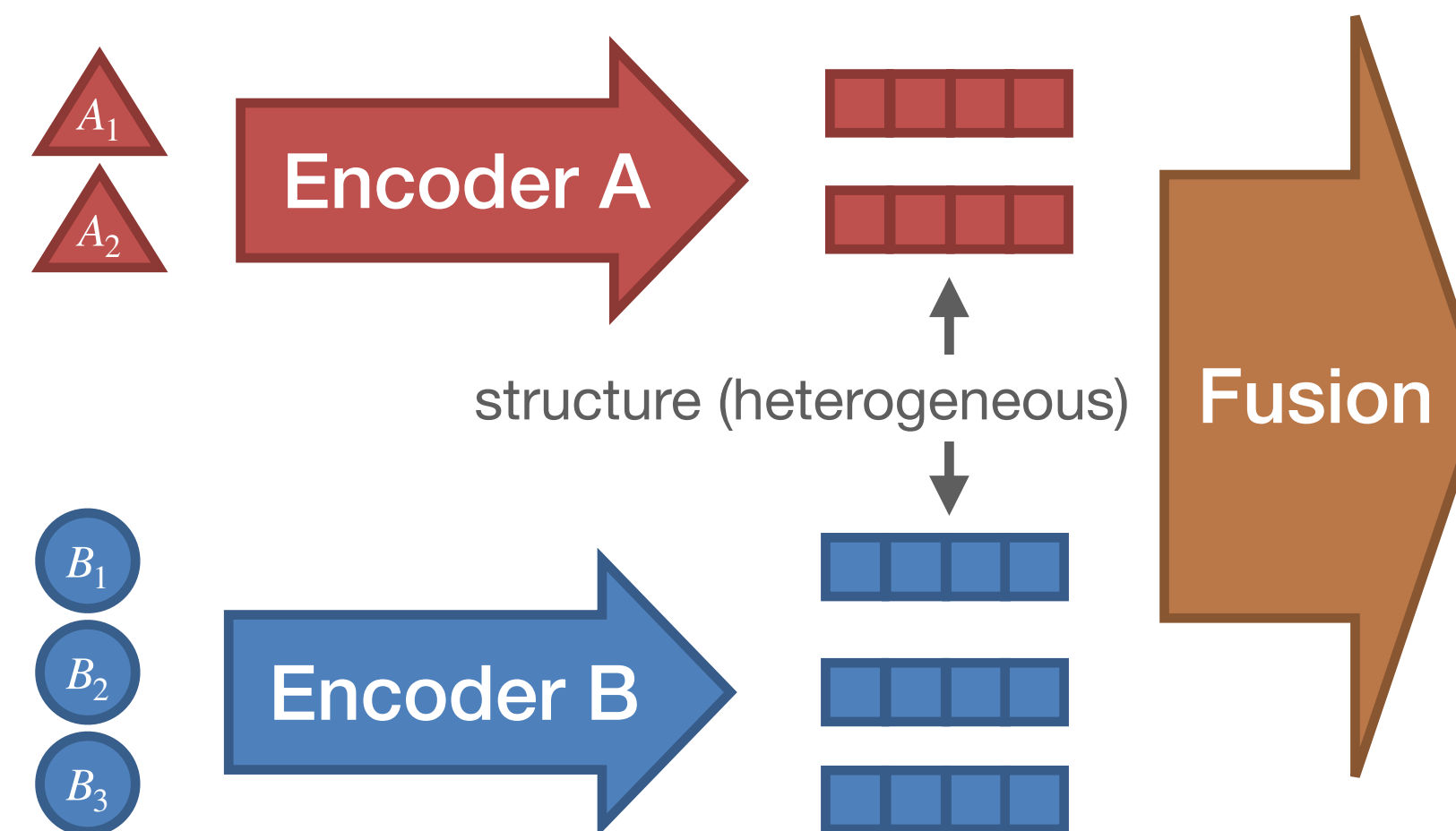


What if we want to keep the structure of input modalities?

STRUCTURAL FUSION

What if we want to keep the structure of input modalities?

- Each modality is represented by multiple vectors
 - **Benefit: Structure is kept intact**
(e.g. feature map / token sequence)
 - Modalities can interact within their structures
 - Modalities can be represented as needed

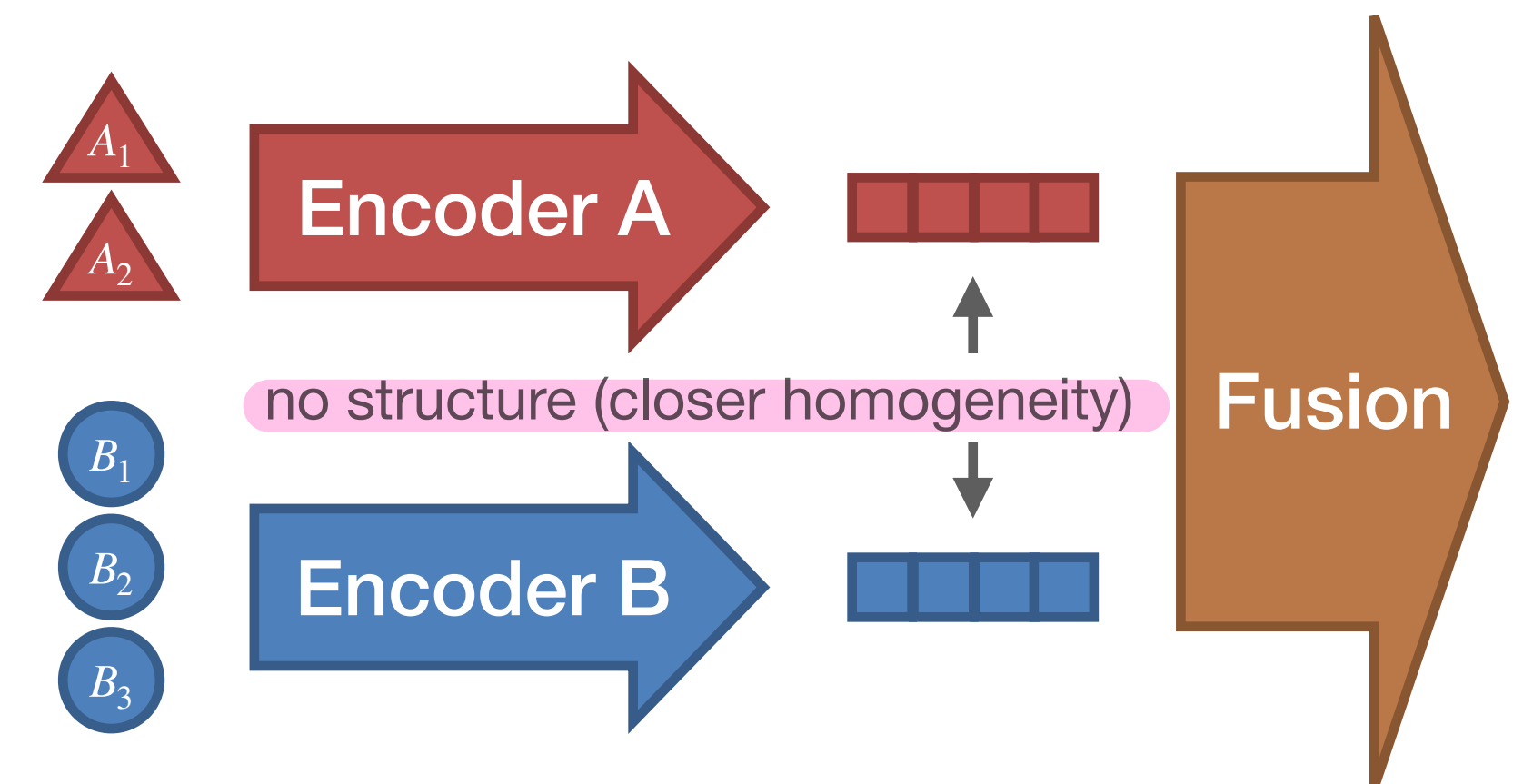


Problem:
Fusion needs to handle heterogeneous structures

STRUCTURAL FUSION

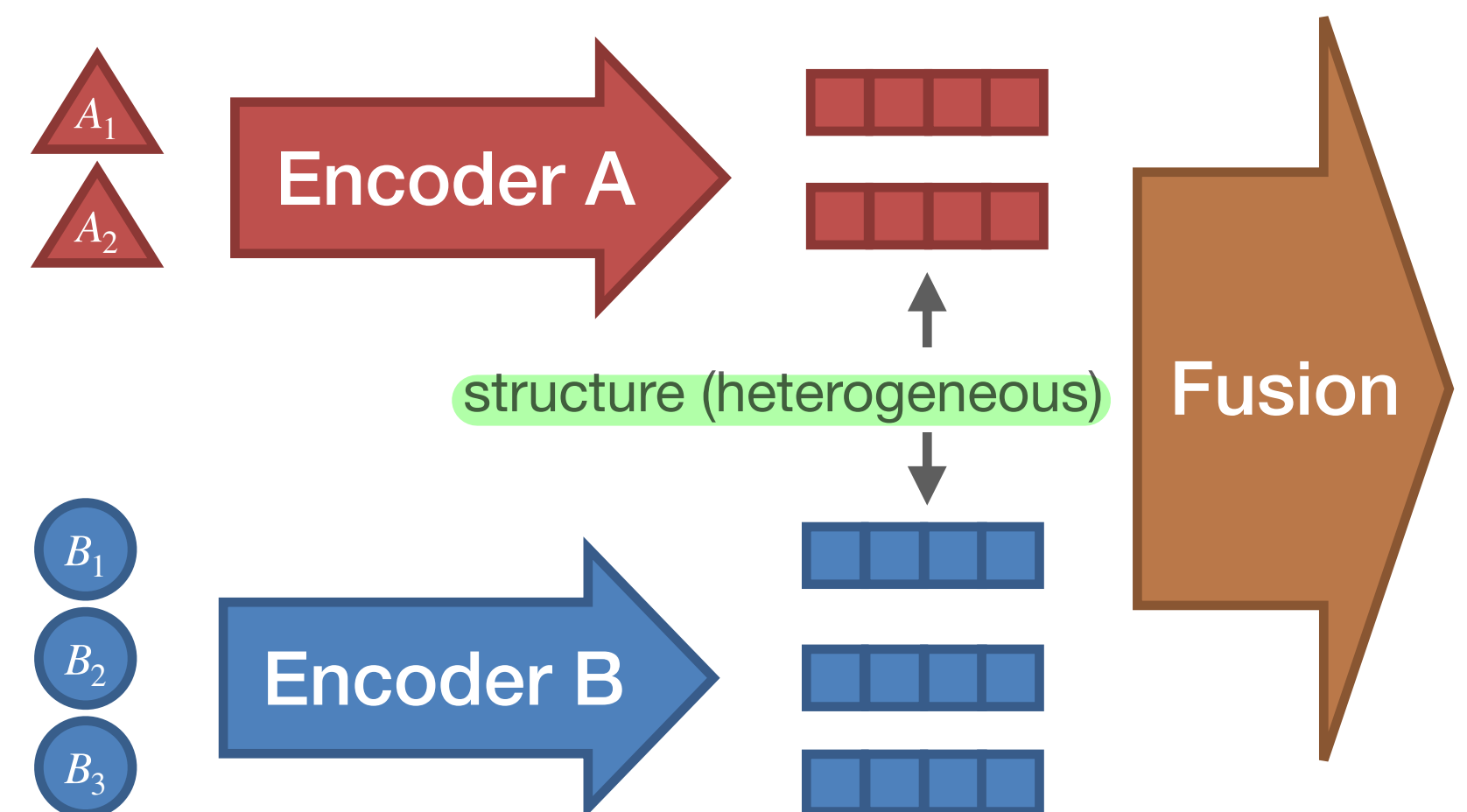
Representation fusion of global vectors does not consider structure of input modalities

- Modality encoders encode each modality as a single vector
 - Problem: Structure is lost**
 - Benefit: Easy to combine (homogeneous)**



What if we want to keep the structure of input modalities?

- Each modality is represented by multiple vectors
 - Benefit: Structure is kept intact**
(e.g. feature map / token sequence)
 - Problem: Fusion needs to handle heterogeneous structures**



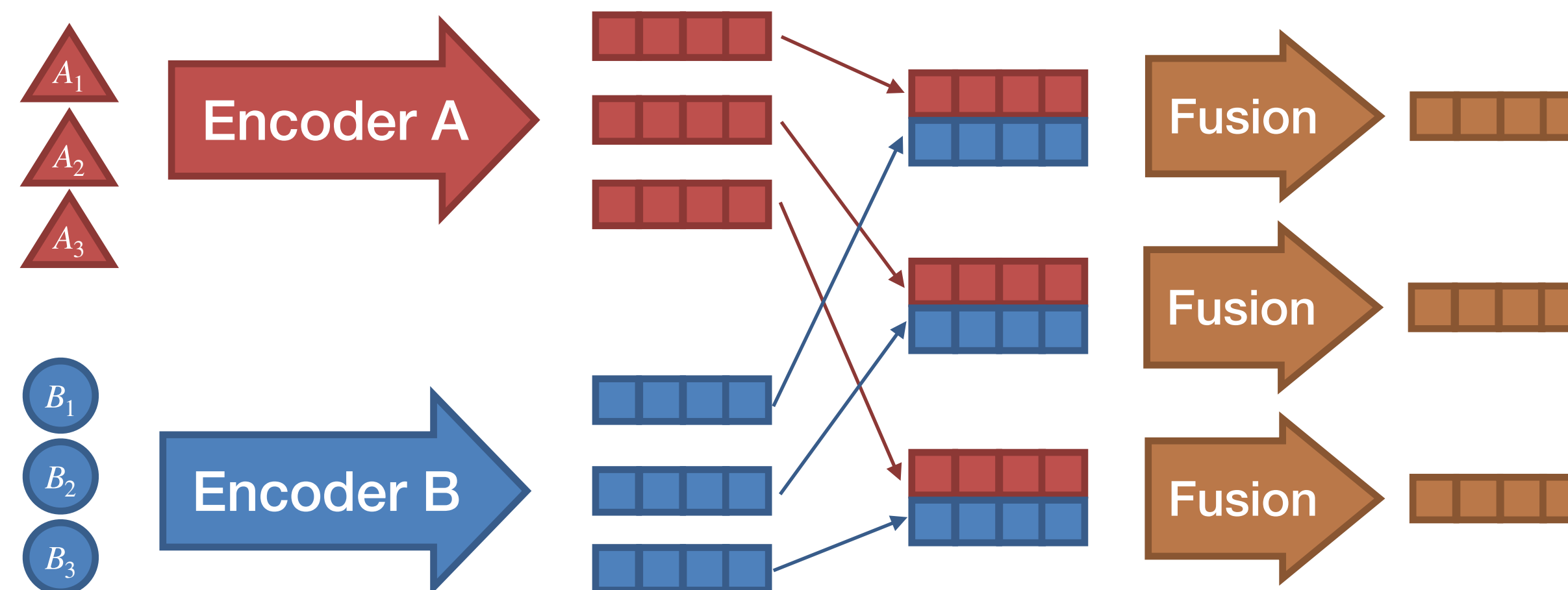
Let's look at the simple cases first

STRUCTURAL FUSION: SHARED STRUCTURES

Element-wise Fusion with Shared Structure

Requirements:

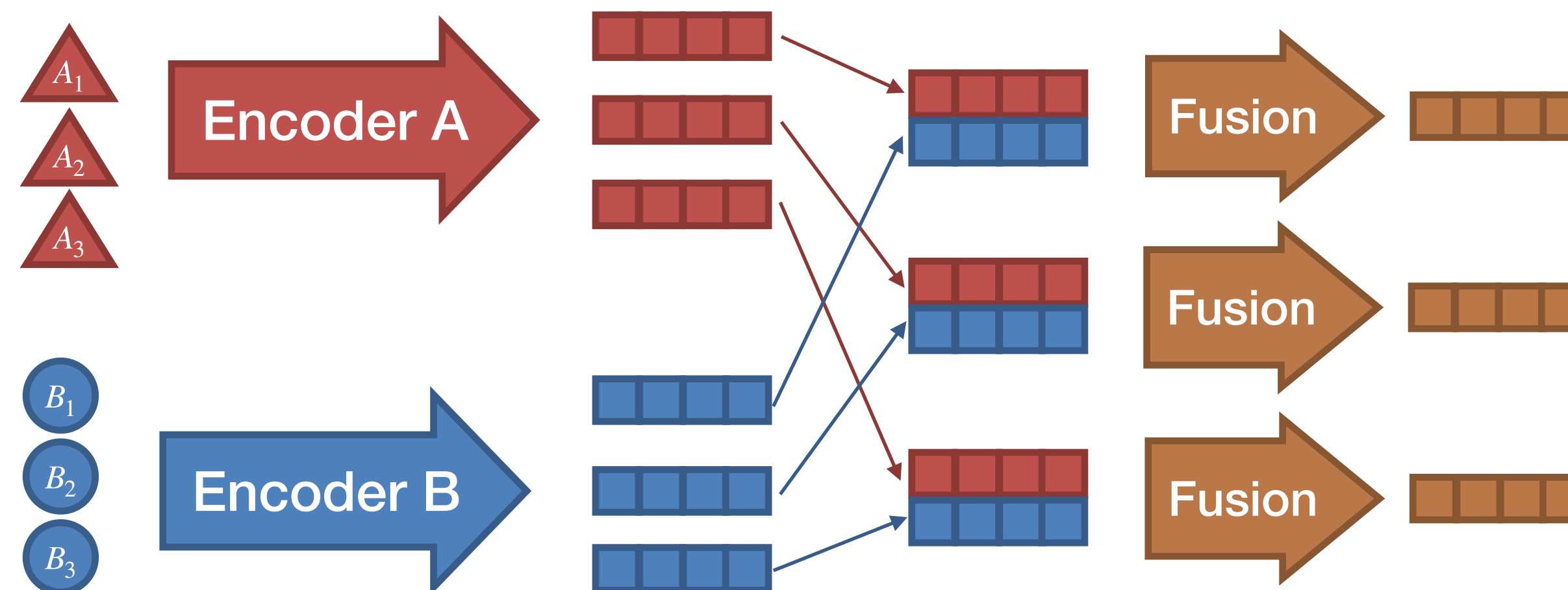
- Modalities have same type of input structure (grid, sequence, set, graph, ...)
- Same number of elements per input (e.g. same resolution, sequence length, ...)
- Input structures are registered
 - each element (e.g. pixel, position) in one modality can be associated to element in other modalities



STRUCTURAL FUSION: SHARED STRUCTURES

Element-wise Fusion with Shared Structure

- Identify associated elements from different modalities
 - E.g. associated/overlapping patches
- Apply representation fusion to associated elements
 - Typically sum/concat fusion
- Fused structure is the same as the input structures

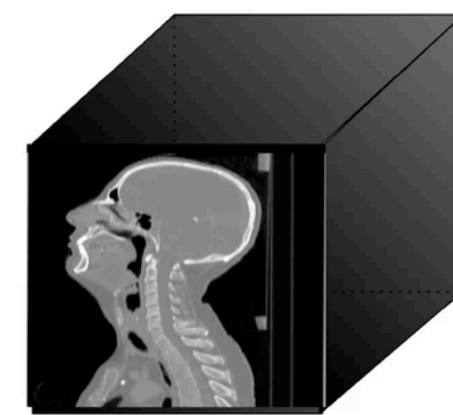


STRUCTURAL FUSION: SHARED STRUCTURES

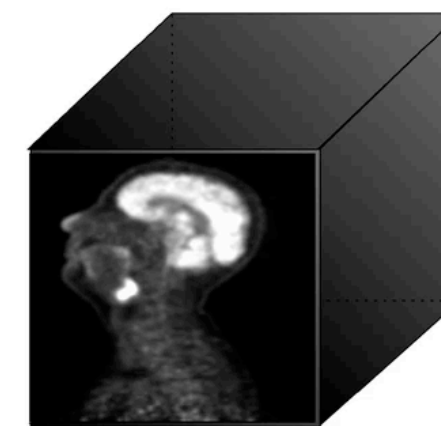
Element-wise Fusion with Shared Structure

Example: Multimodal Medical Imaging

- Multiple scans of them same patient available (e.g. CT + PET volumes)



CT Volume



PET Volume

- Image encoders per volume (e.g. CNNs)
 - parameters may or may not be shared
- Fuse feature maps, i.e. fuse individual patch features (per patch)
- Apply decoder to fused feature map
 - e.g. CNN-based classifier

What if the resolutions are different?

STRUCTURAL FUSION: SHARED STRUCTURES

Shared Grids with Different Resolution

Option 1: Rescale input images

- Up- or downscale (interpolate) all input modalities to same resolution before encoding

Option 2: Rescale during encoding

- Use encoders with different downscaling factors
 - strides in convolutional layers or pooling layers
 - interpolations between layers

Option 3: Rescale after encoding

- (Bilinear) Interpolation of feature maps

What if the inputs are not registered?

STRUCTURAL FUSION: SHARED STRUCTURES

Non-registered Structures

We cannot assume that these share the same grid if the grids are not aligned/registered

➡ **Same resolution does not mean shared grid!**

We now need global interactions

➡ **Local neighborhoods do not transfer to other modalities**

- i.e. position (2, 1) on grid A may not be “close” to position (2, 1) on grid B

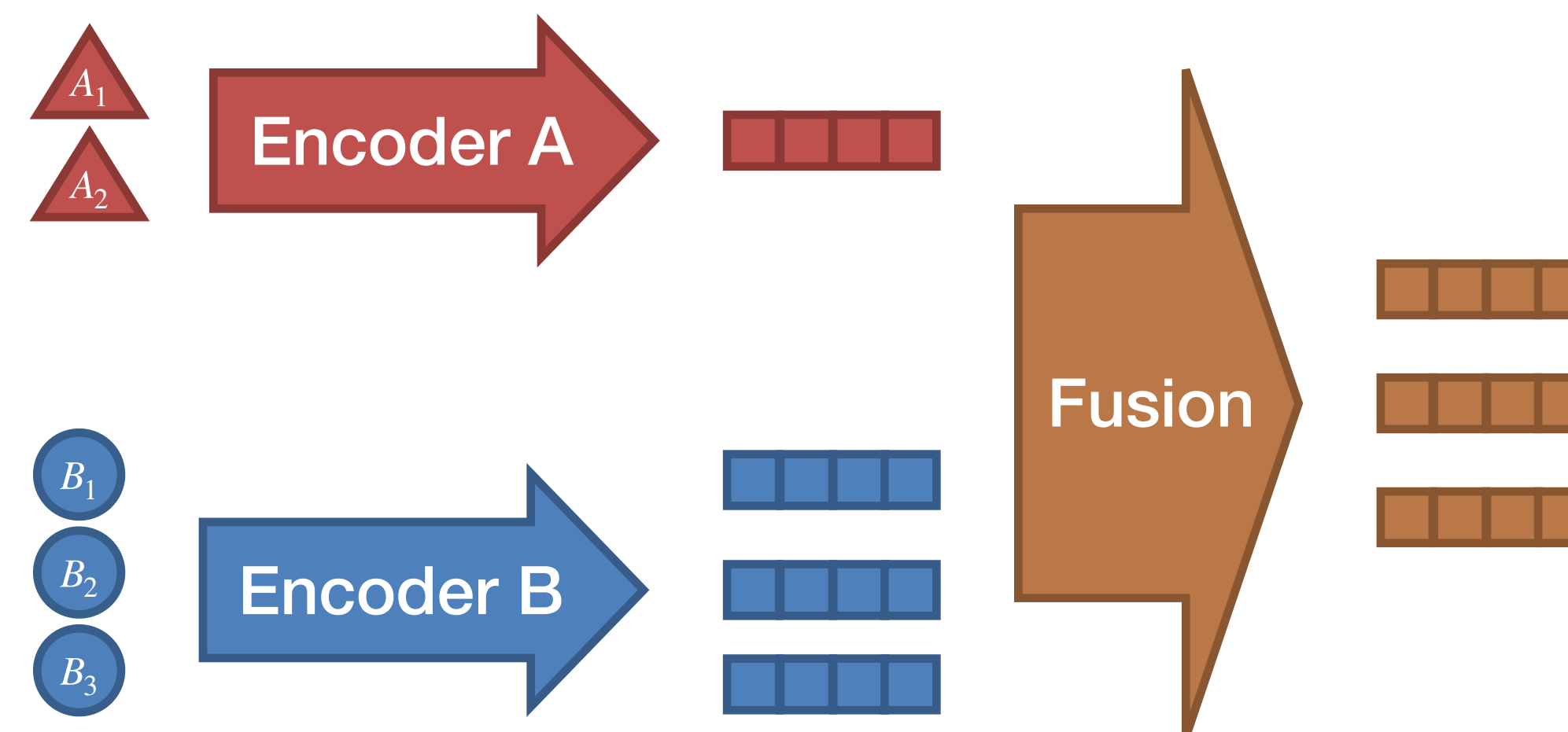
➡ **Treat them as different structures**

How can we combine modalities without shared structures?

STRUCTURAL FUSION WITHOUT SHARED STRUCTURES

Simple Case: **There is a *Primary Modality***

- Structure of primary modality should be kept
 - Processing should happen in this structure
- Information from other modality should be injected into this structure



STRUCTURAL FUSION: INJECTING FEATURES INTO PRIMARY STRUCTURE

1. Encode “secondary” modality
 - Use modality-specific encoder
2. Encode “primary” modality while injecting “secondary” features
 - Adapt encoder to include the “secondary” features

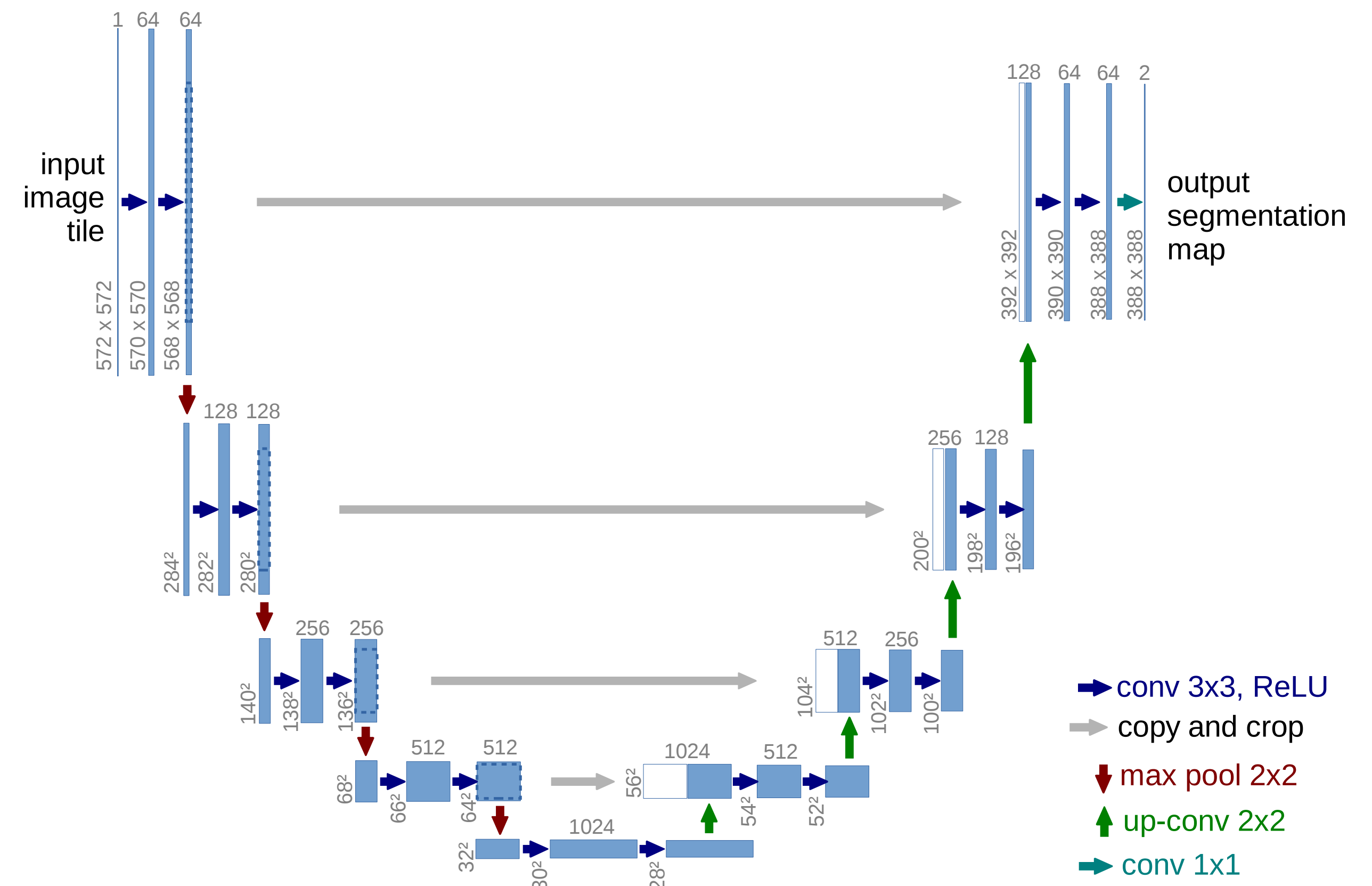
How can we modify the encoder?

STRUCTURAL FUSION: INJECTING FEATURES INTO PRIMARY STRUCTURE

U-Net Encoder for primary modality

Recap of U-Net

- Downsampling encoder
- Upsampling decoder
- Low-resolution bottleneck
- Skip connections at each resolution

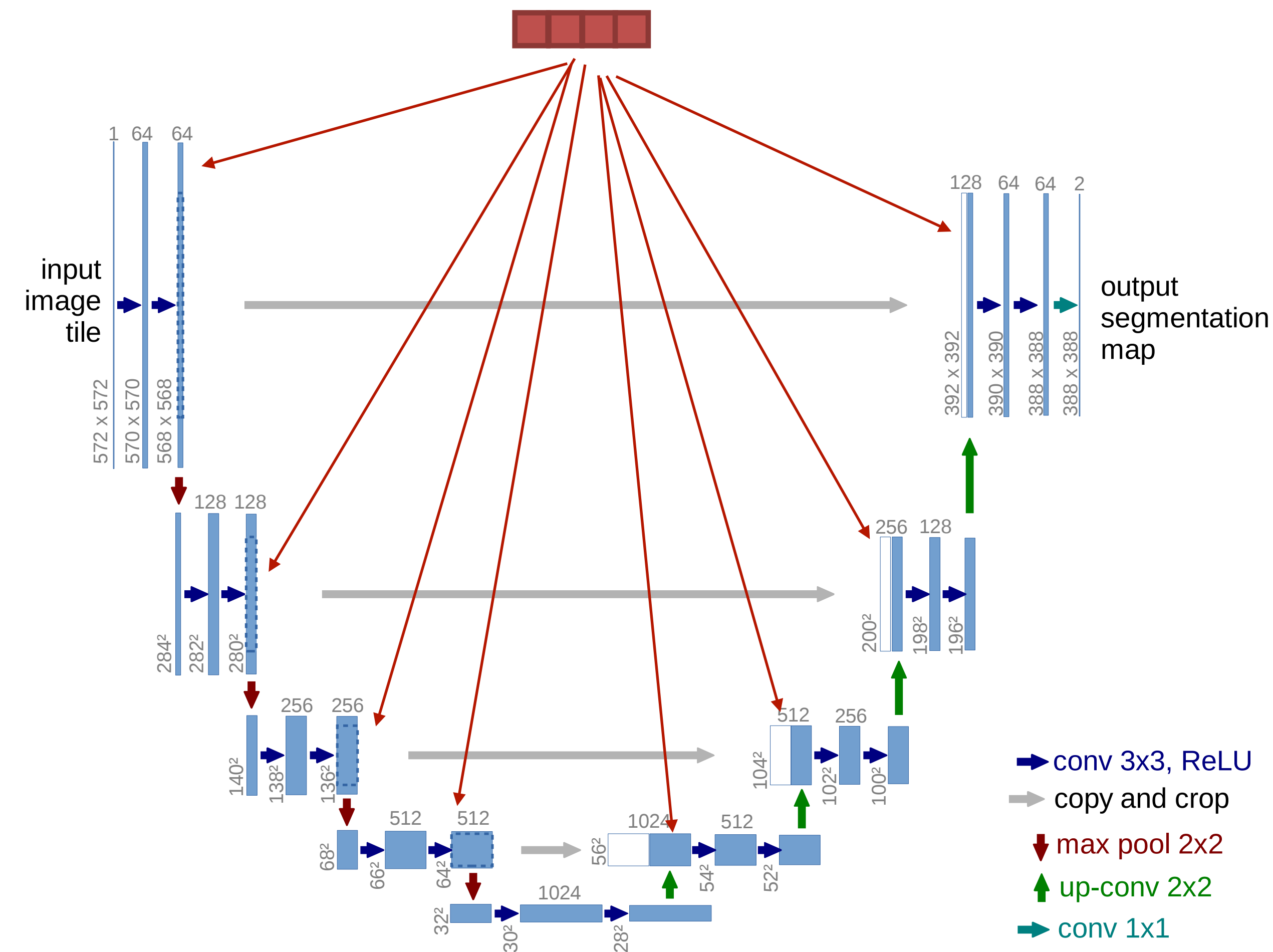


STRUCTURAL FUSION: INJECTING FEATURES INTO PRIMARY STRUCTURE

U-Net Encoder for primary modality

Simple Fusion

- Single global feature vector for secondary modality
- Fuse at each block
 - e.g. additive / concat fusion
 - e.g. before/after down/up sampling



STRUCTURAL FUSION: INJECTING FEATURES INTO PRIMARY STRUCTURE

U-Net Encoder for primary modality

Cross Attention

- Add cross-attention layer after/before/inside each block
 - Queries = U-Net feature map
 - Keys/values = secondary modality
- **Benefit:** multiple/structured secondary feature vectors supported
 - ➡ better information transfer from secondary modality

STRUCTURAL FUSION: INJECTING FEATURES INTO PRIMARY STRUCTURE

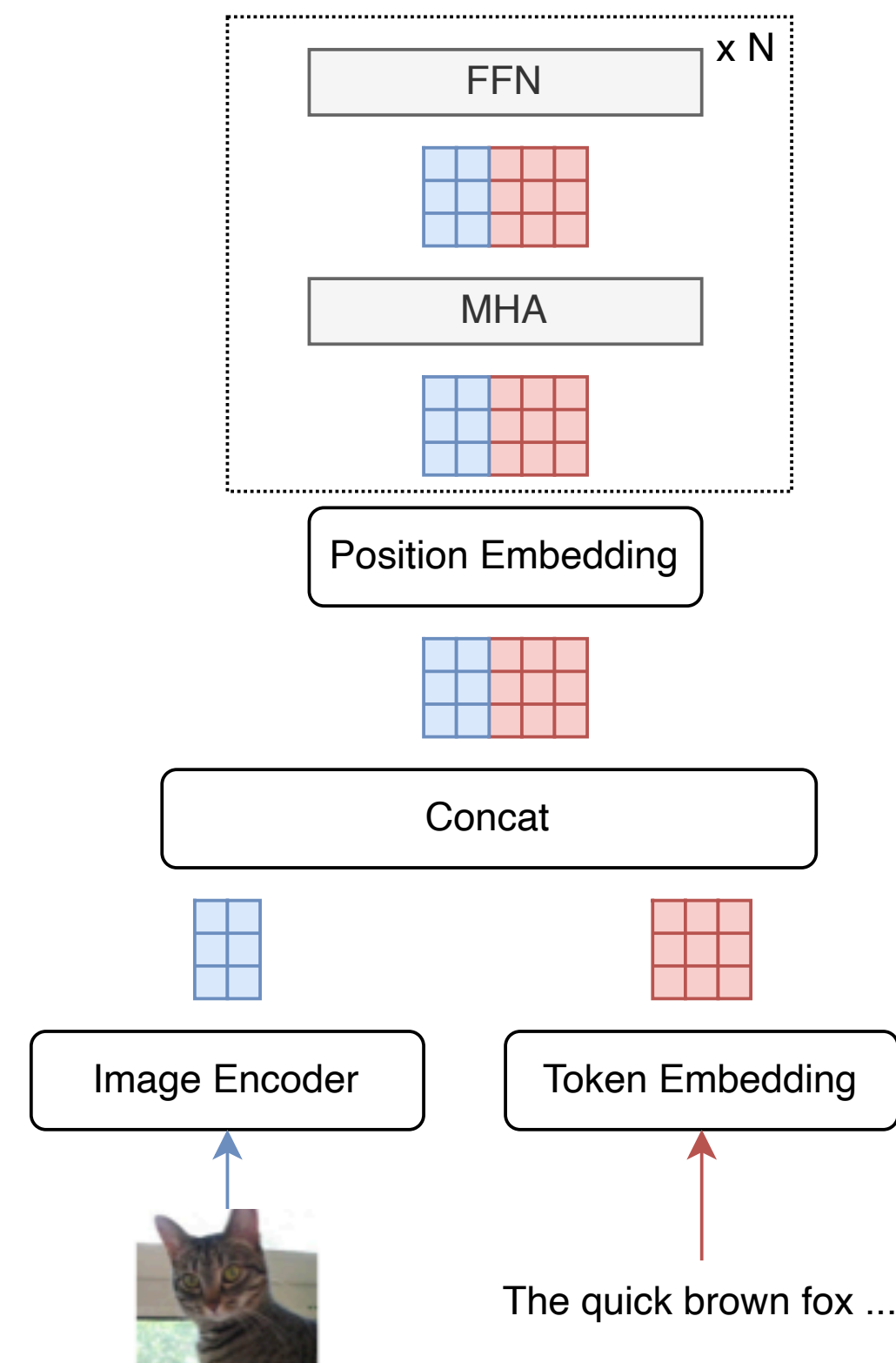
Transformer Encoder for primary modality

Simple Solution: Modify Self-Attention

- Queries only come from primary modality
- Keys/Values: Concatenate primary key/value sequence with secondary features

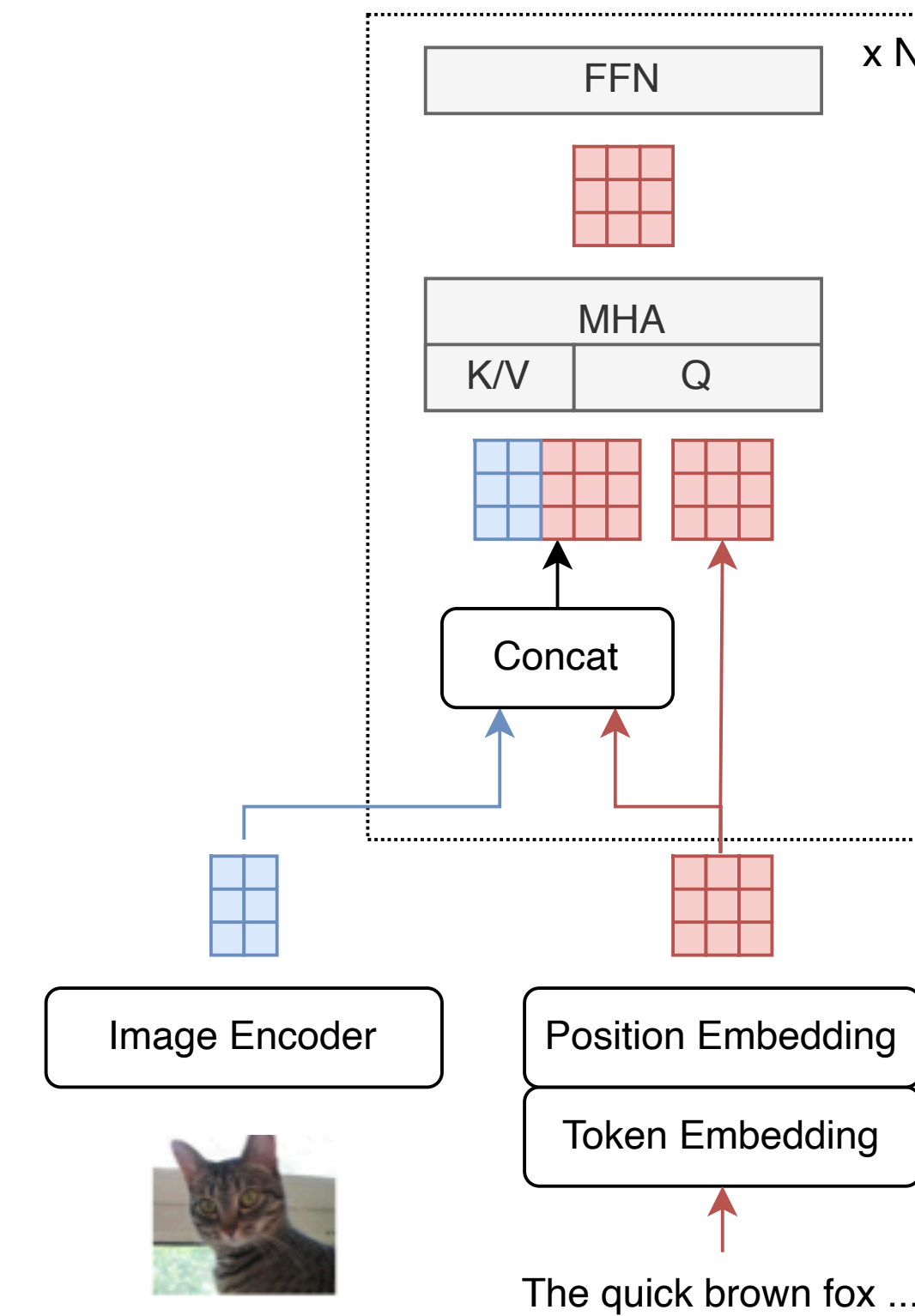
STRUCTURAL FUSION: INJECTING FEATURES INTO PRIMARY STRUCTURE

Transformer Encoder for primary modality: Extending an LLM to understand Images



Llava:
concatenate once after embedding layer

Liu et al (2023)



Pseudo self attention:
concatenate K/V sequence at every layer

Ziegler et al (2019)

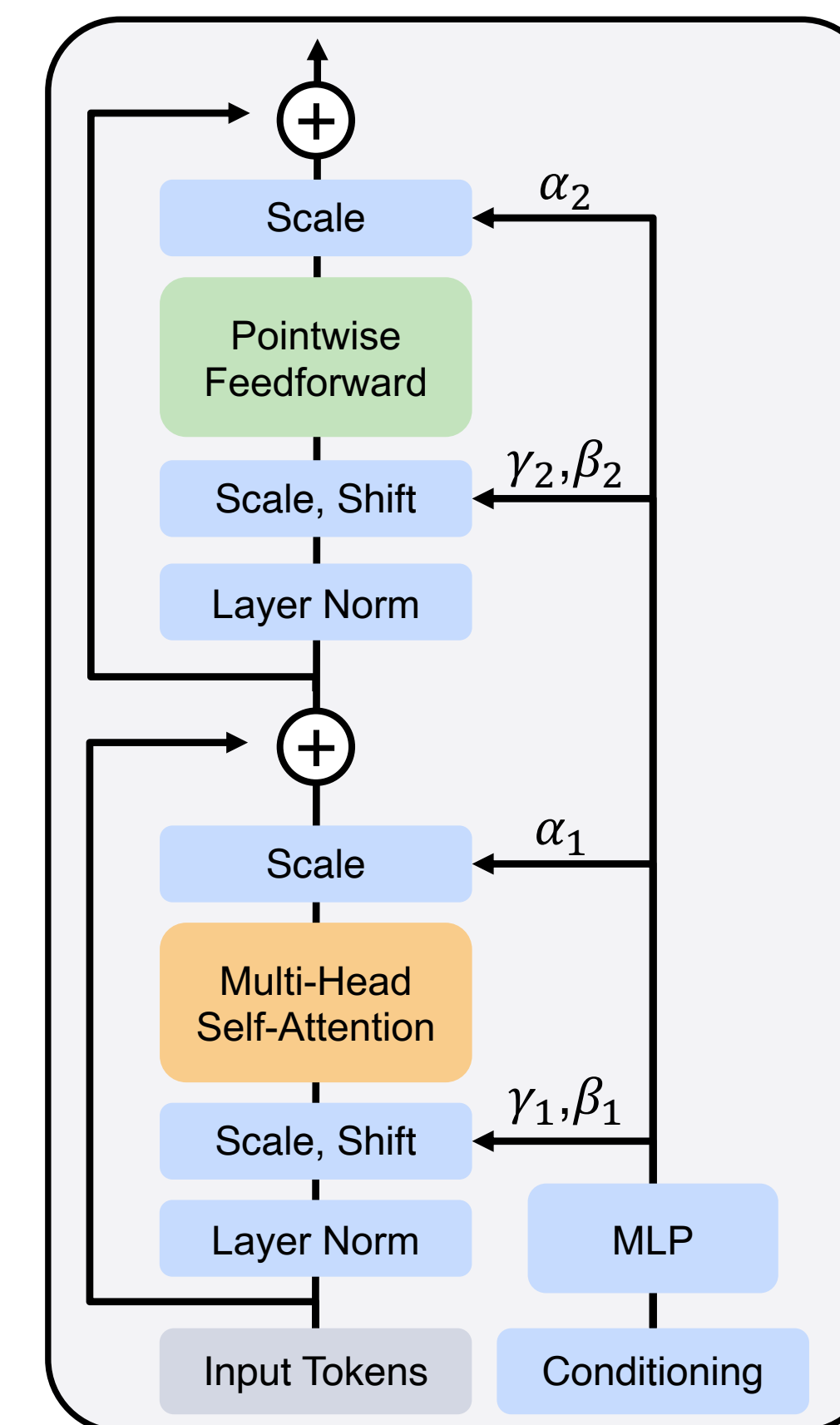
STRUCTURAL FUSION: INJECTING FEATURES INTO PRIMARY STRUCTURE

Transformer Encoder for primary modality: Conditioning through shift and scale

- Only scale and shift features of primary modality
- Scale and shift parameters conditioned on secondary modality

Benefit:

- Primary encoder features are only slightly modified
 - less “forgetting”



Peebles et al (2023)

STRUCTURAL FUSION: MULTIMODAL STRUCTURE

Assume we don't have a primary modality:

Which structure should we use for fusion?

Remember:

Sets or Sequences are well fit as “multimodal” input sequences

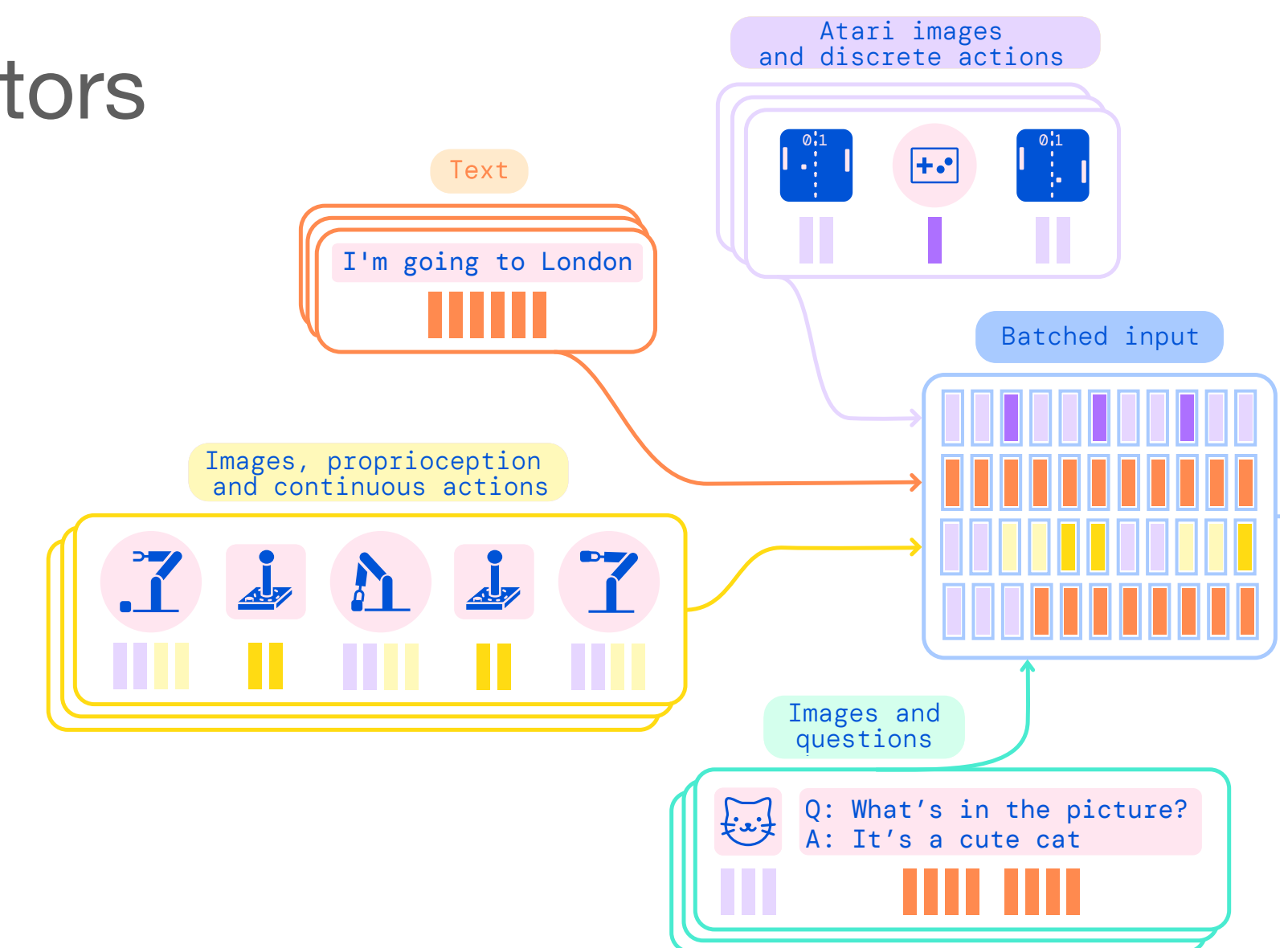
But how do we enable structural interactions?

MULTIMODAL STRUCTURE: SETS AND SEQUENCES

Reminder: Sets / sequences as multimodal input:

- Each input modality is embedded as set/sequence of token vectors
- Sequences are concatenated / interleaved

➡ Single multimodal token sequence



How do we enable structural interactions?

➡ Self-Attention in a Transformer

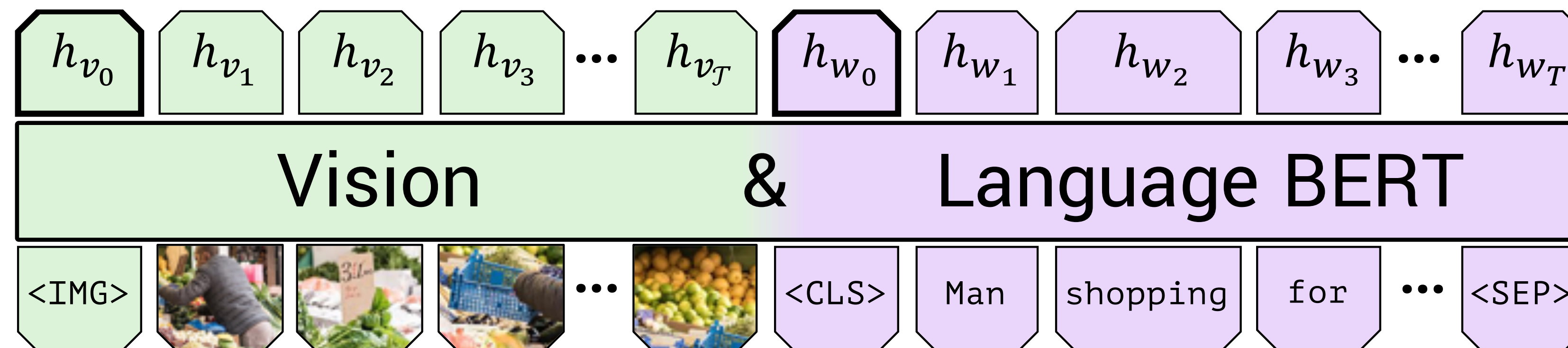
Problem:

What if tokens should be processed differently depending on the modality?

MULTIMODAL STRUCTURE: SETS AND SEQUENCES

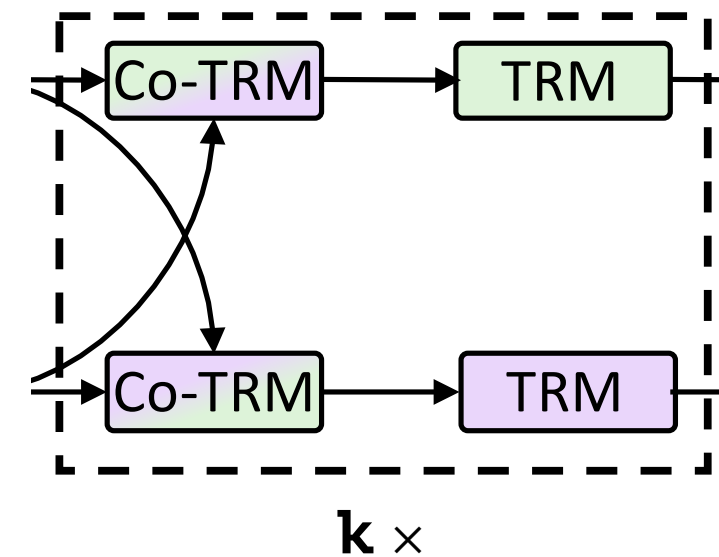
Multi-Stream Encoders

- Sequences of different modalities (e.g. vision and language) are concatenated
- **BUT:** modality sub-streams (vision/language) are handled as individual streams
 - Distinguish between **intra- and inter-stream interactions**
 - No or partial parameter-sharing (different parameters may be used per modality)



MULTIMODAL STRUCTURE: SETS AND SEQUENCES

Multi-Stream Encoders: ViLBERT



Co-Transformer layer (Co-TRM)

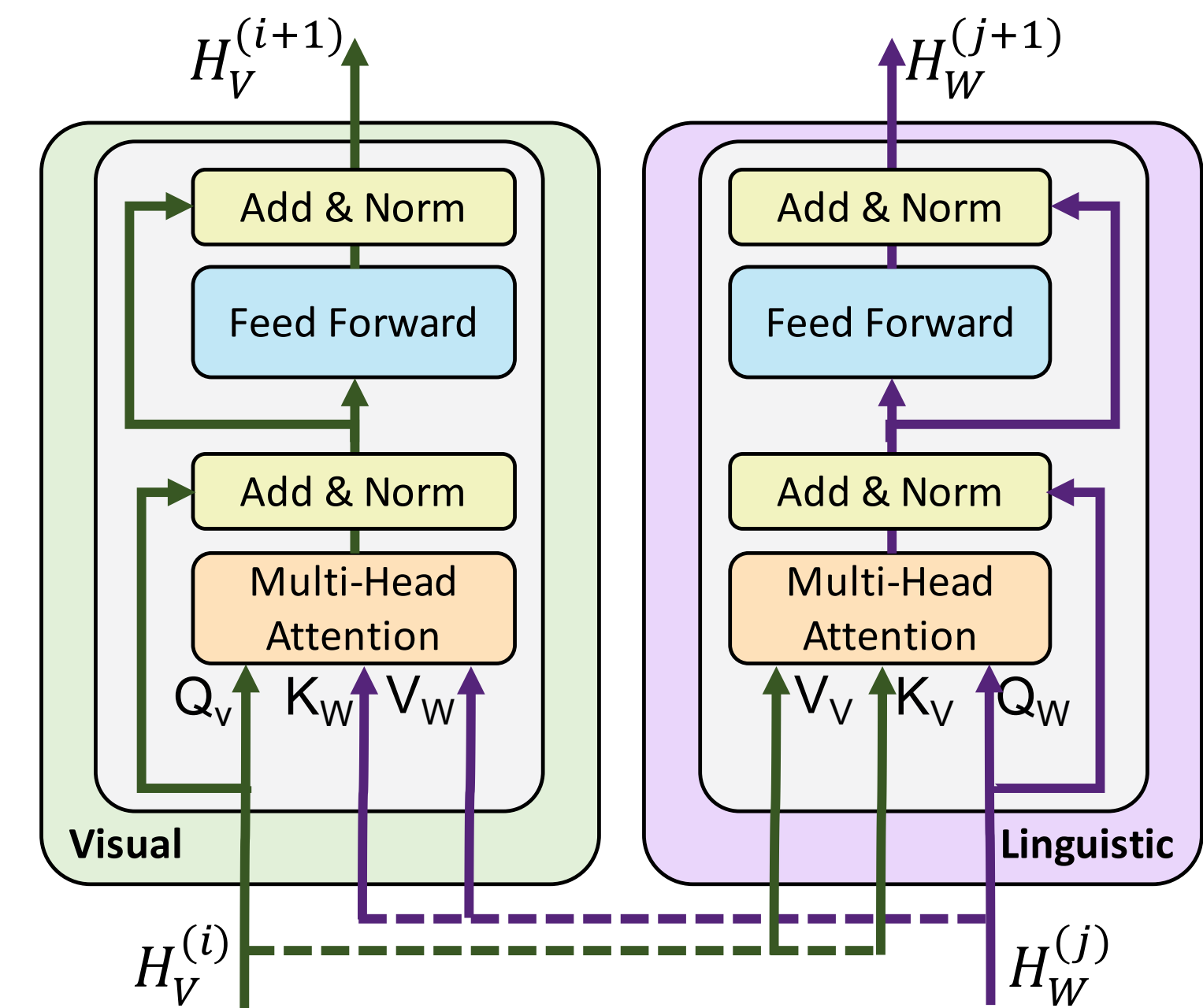
Inter-modality interactions: Condition on other modality

- Cross attention
- Q = current modality
- K/V = other modality

Transformer layer (TRM)

Intra-modality interactions: Interactions within one modality

- Self-attention (only within one modality)



Lu et al (2019)

MULTIMODAL STRUCTURE: SETS AND SEQUENCES

Multi-Stream Encoders: ViBERT

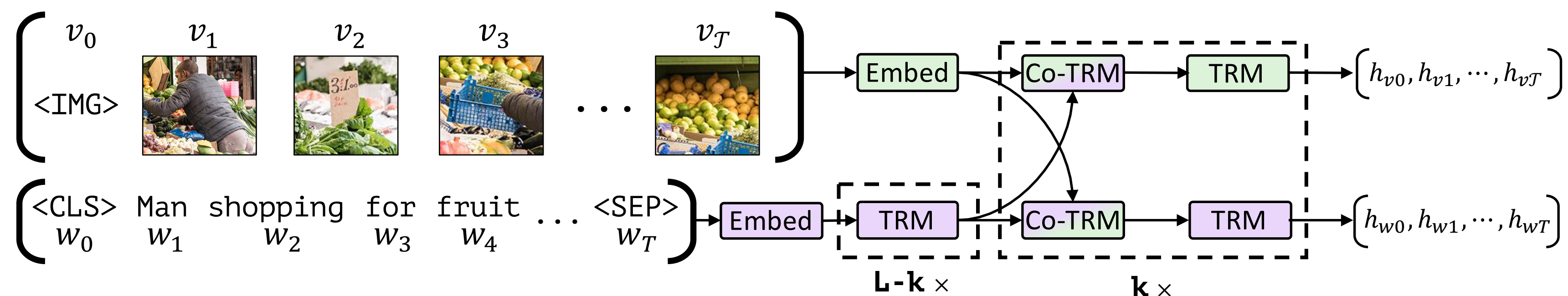
No parameter-sharing!

Vision Stream

- Tokenizer/Embedding: Object detector + projection + object position embeddings
- Series of Co-Transformer (K/V from language) and Transformer layers

Language Stream

- Tokenizer/Embedding: Word/subword embedding + Transformer layers
- Series of Co-Transformer (K/V from vision) and Transformer layers



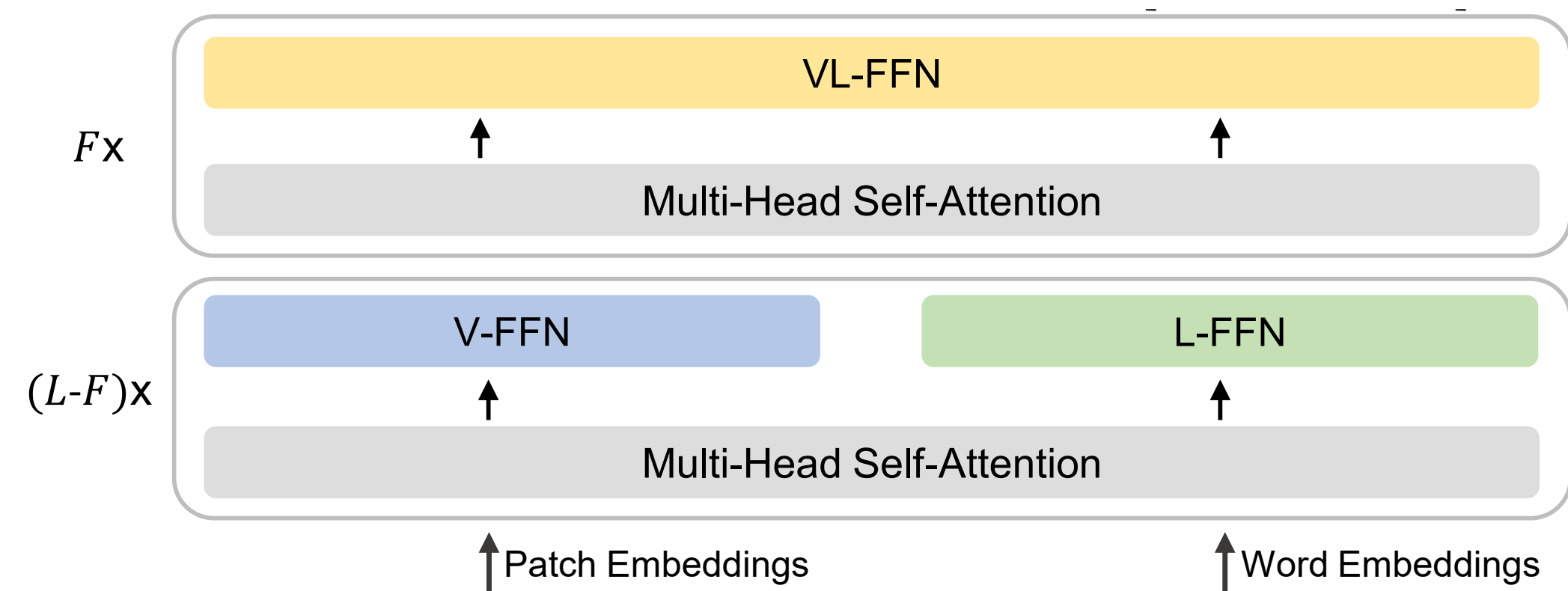
BUT: no parameter sharing yet...

MULTIMODAL STRUCTURE: SETS AND SEQUENCES

Multi-Stream Encoders: Multimodal Experts (BeiT3)

Unifying multi-stream interactions

- Use the “standard” self-attention to interact between streams
 - Streams are not separated, i.e. intra- and inter-stream interactions are unified
 - ➔ No need for inter-stream cross-attention (covered by self-attention)
- Modality-specific FFN layers as multimodal experts
 - Enables modality-specific processing
 - e.g. vision experts (V-FFN), language experts (L-FFN), and vision-language experts (VL-FFN)



MULTIMODAL STRUCTURE: SETS AND SEQUENCES

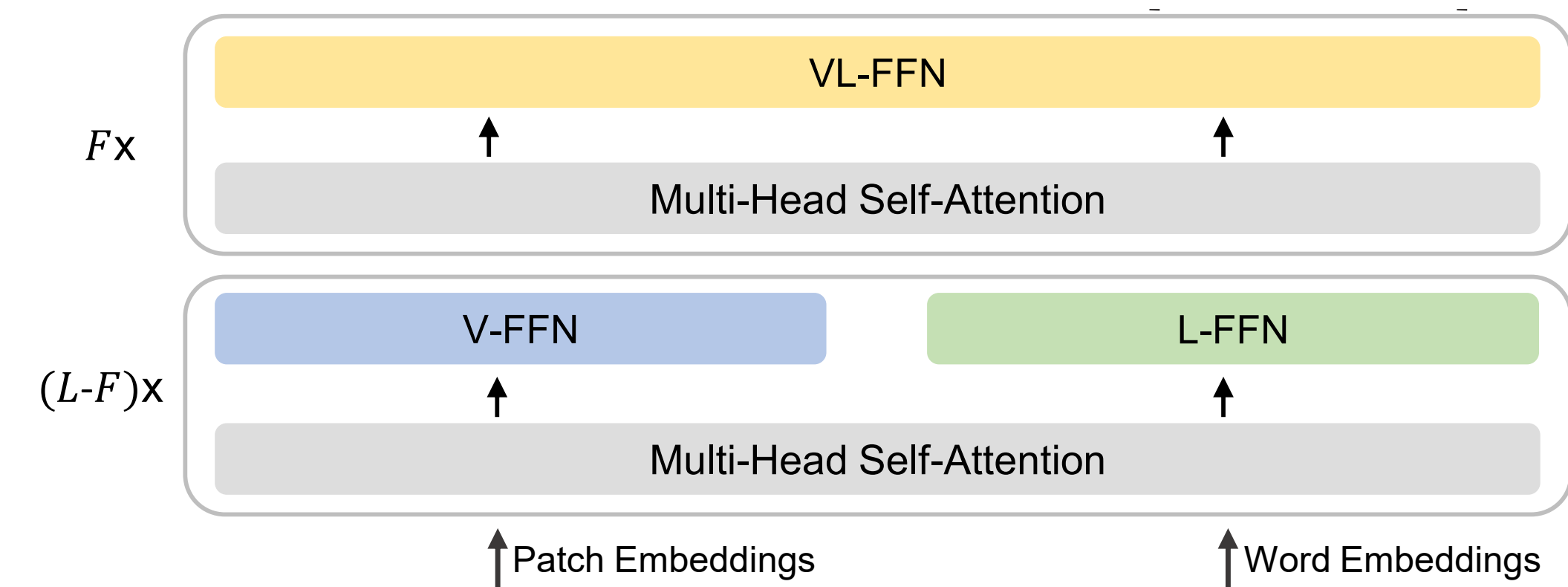
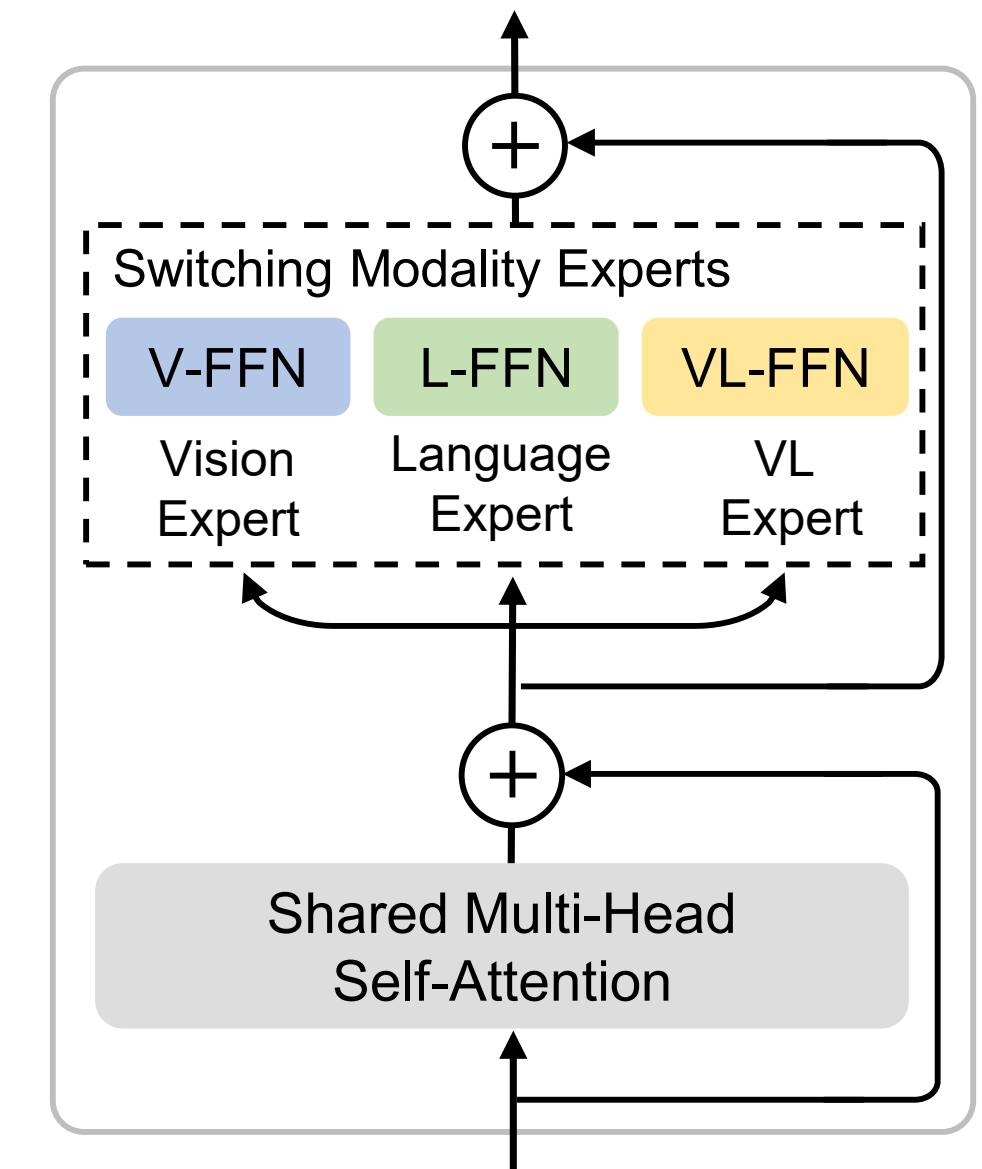
Multi-Stream Encoders: Multimodal Experts

Modality-specific FFN layers as multimodal experts

- One set of FFN parameters per expert
- Switch between modality experts based on type of token (and task)

Vision-language experts (VL-FFN)

- Enable multimodal feature processing
- Used for vision and language tokens in the last layers



MULTIMODAL STRUCTURE: SETS AND SEQUENCES

Remaining Problem:

Multimodal sequence is the concatenation of unimodal sequences:

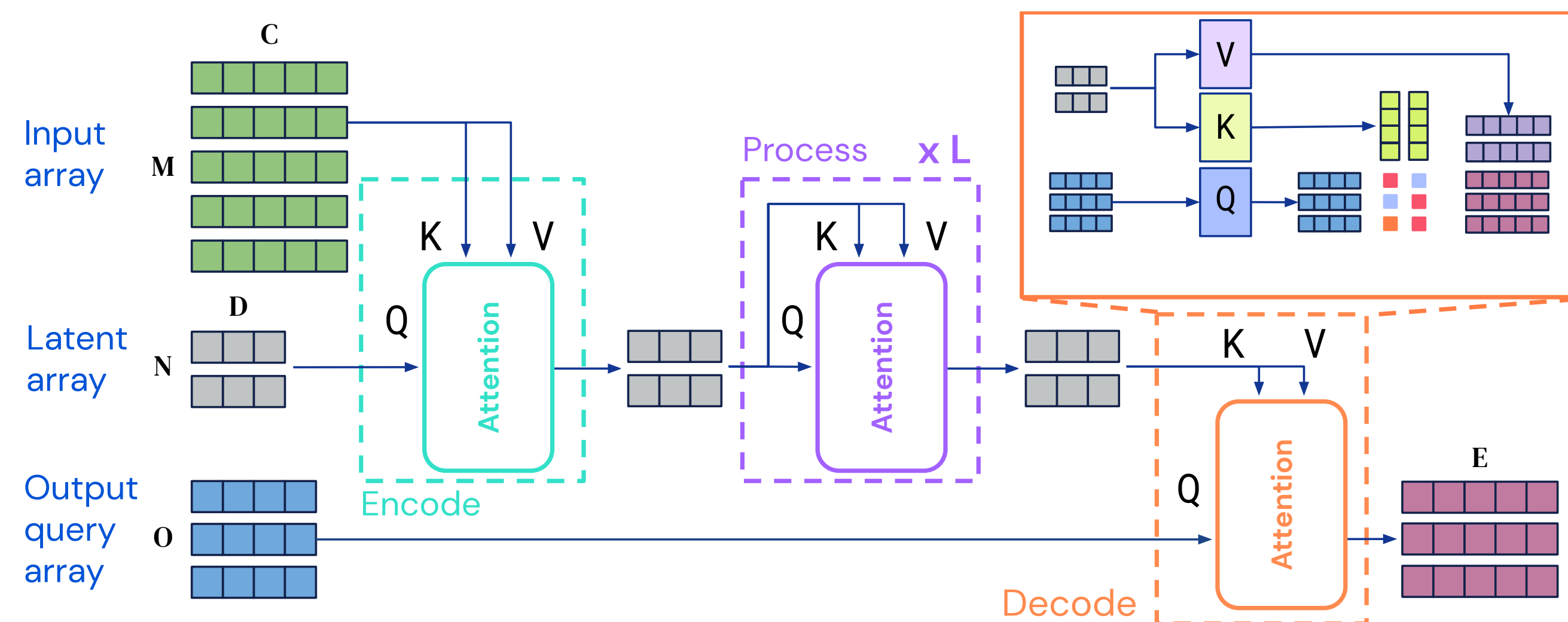
Processing units are tied to unimodal elements (the tokens)

Can we introduce a multimodal structure independent of input sequence lengths?

MULTIMODAL STRUCTURE: LATENT QUERIES

Perceiver IO

1. Encode the input sequence (length M) into a latent sequence (length N)
2. Process the latent sequence (length N)
Most of the compute is happening here!
3. Decode the latent sequence (length N) into output sequence (length O)

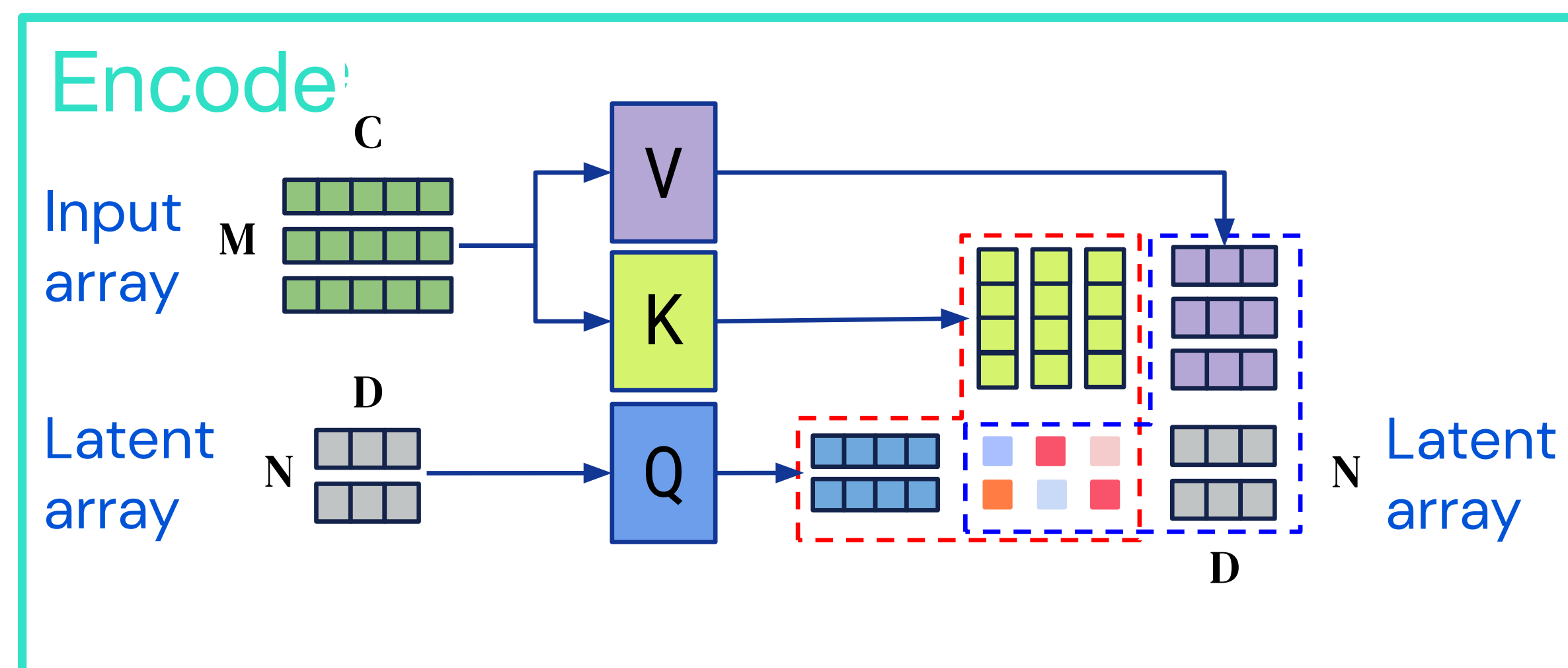


MULTIMODAL STRUCTURE: LATENT QUERIES

Perceiver IO

1. Encode the input sequence (length M) into a latent sequence (length N)

- Fixed number of **learned latent query vectors**
- N and D can be chosen (may differ from input!)
- Transformer layer with cross-attention
(Q = latents, K/V = input)

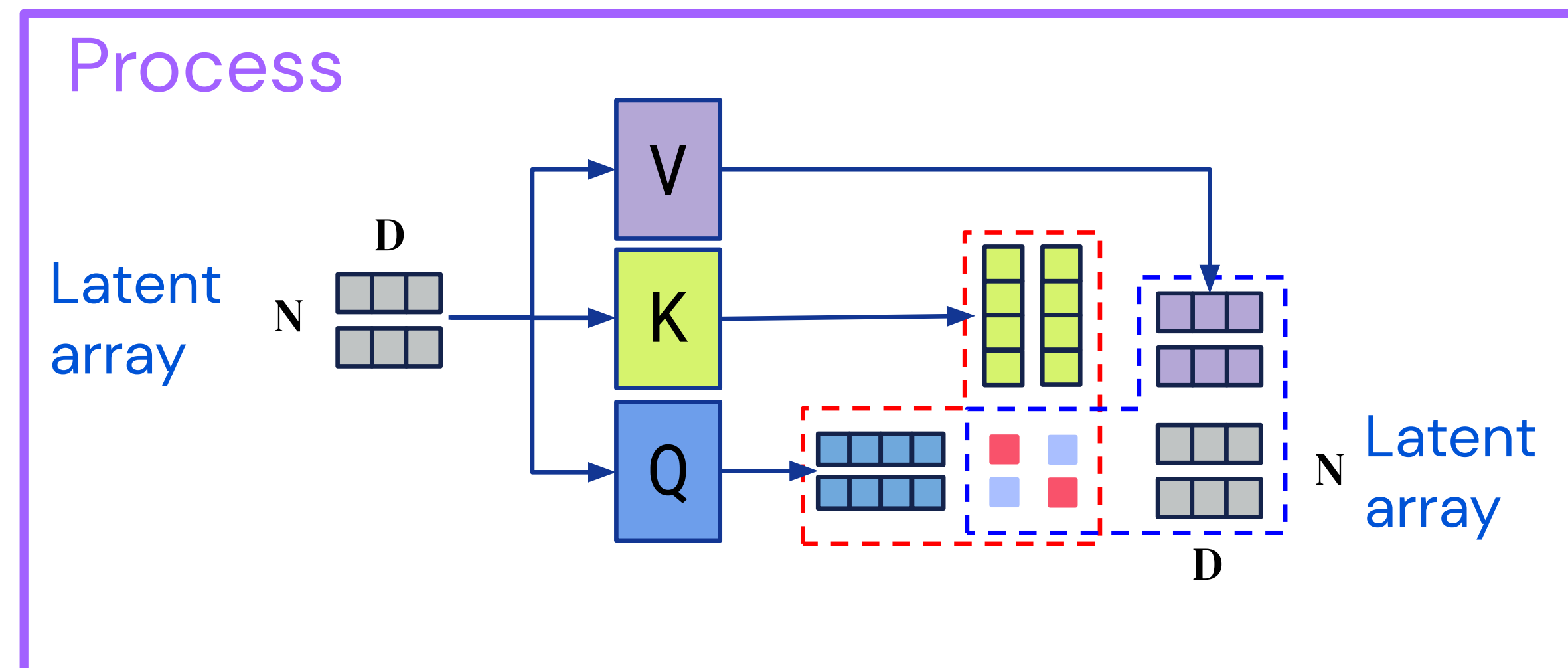


MULTIMODAL STRUCTURE: LATENT QUERIES

Perceiver IO

2. Process the latent sequence (length N)

- “Standard” Transformer encoder (multiple layers)
- Self-attention within latent sequence (Q, K, V from same source)

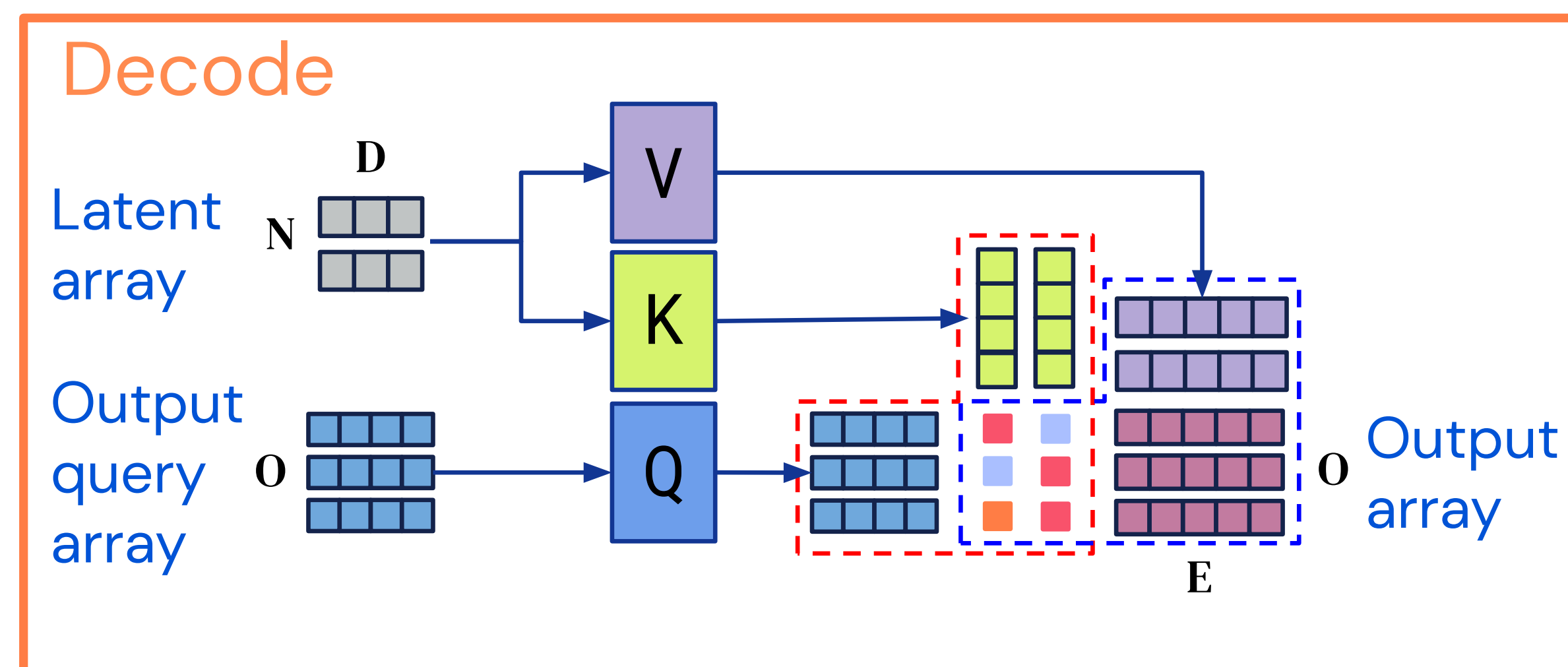


MULTIMODAL STRUCTURE: LATENT QUERIES

Perceiver IO

3. Decode the latent sequence (length N) into output sequence (length O)

- Output-specific output query vector sequence (e.g. position embeddings)
we will later look at this in more detail
- Transformer layer with cross-attention
(Q = output, K/V = latents)
- Final output vectors are then used in task-specific prediction heads (e.g. MLPs)

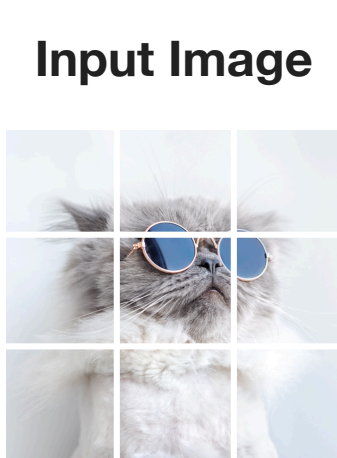


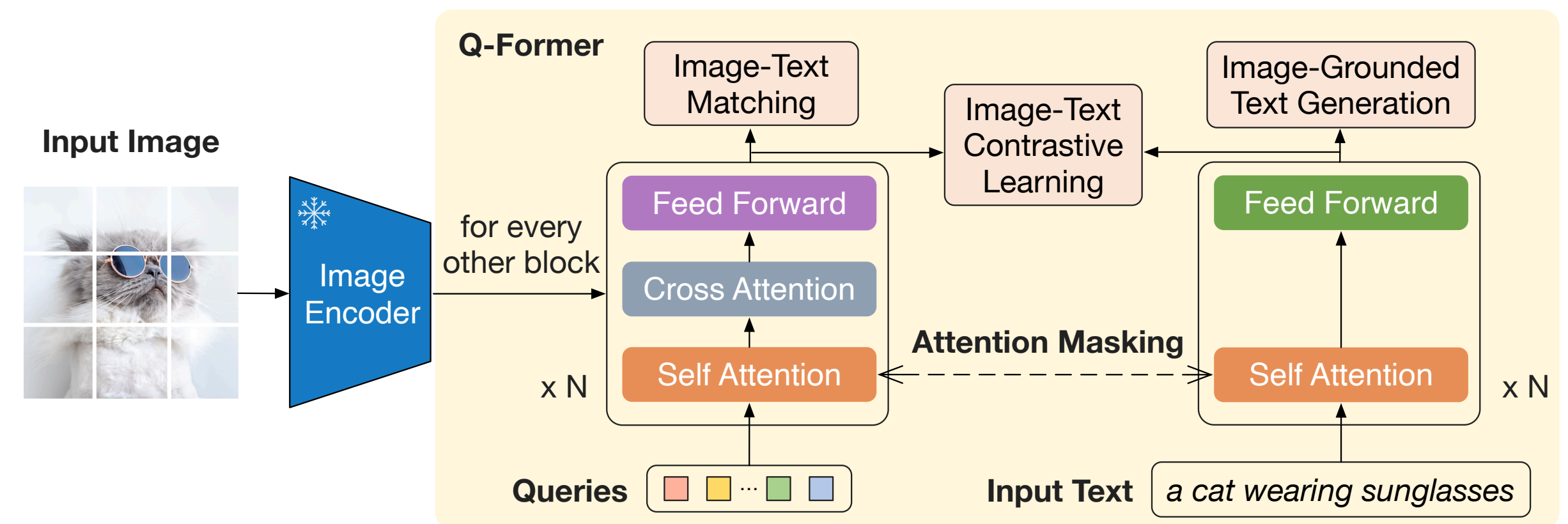
MULTIMODAL STRUCTURE: LATENT QUERIES

Q-Former

Extracting vision-language features using latent queries

- Input sequence: Latent queries + Text
 - Fixed number of learned query vectors
 - (Optional) embedded text tokens
 - Transformer encoder
 - Shared self-attention for queries and text
 - **Cross-attention to image features only for query tokens**
 - Non-shared Feed Forward layers
 - Task-dependent attention masking during pretraining
 - Causal masking / limited query-text interactions
- Input Image

A 3x3 grid of a cat wearing sunglasses. The center cell of the grid is highlighted with a blue border, while the other cells have white borders. The cat is looking directly at the camera.



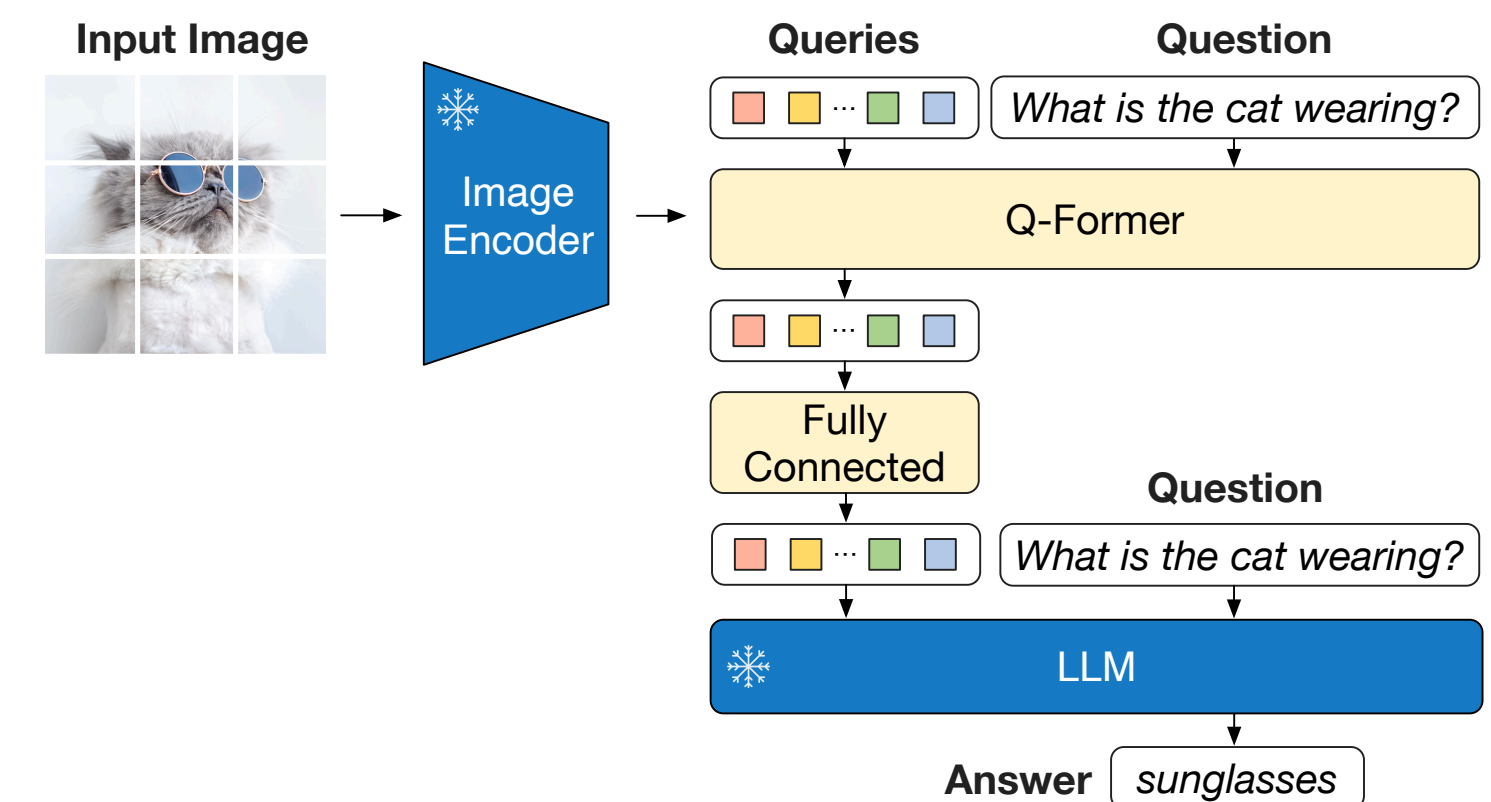
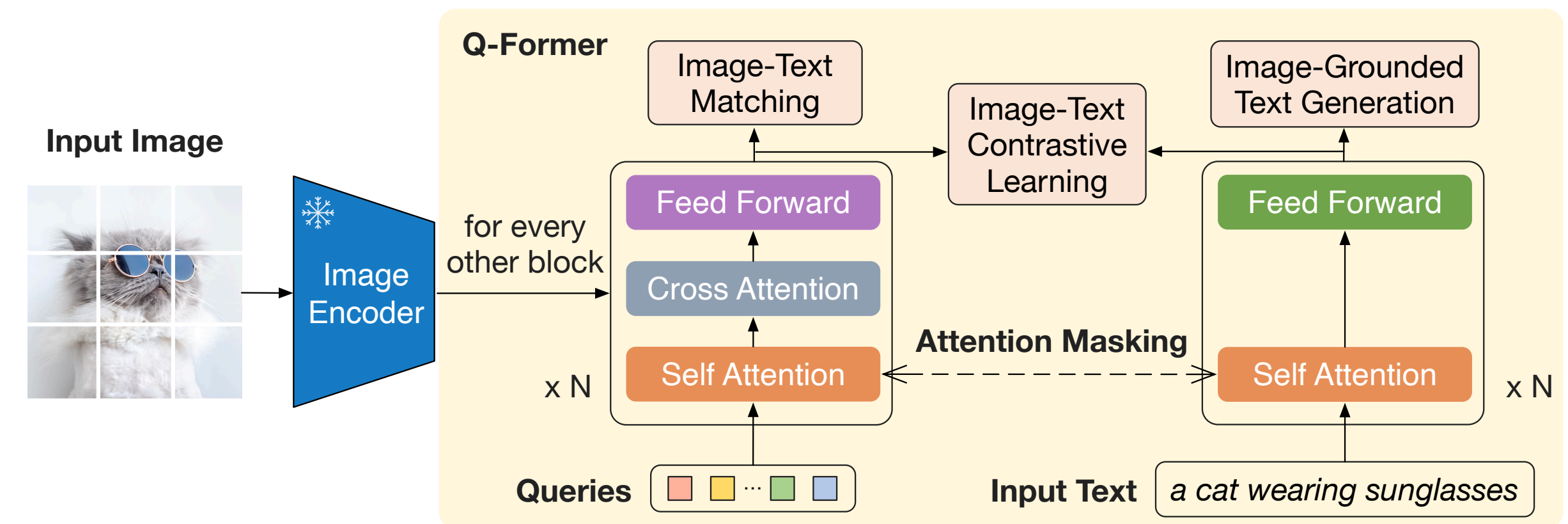
Li et al (2023)

MULTIMODAL STRUCTURE: LATENT QUERIES

Q-Former

Extracting image features using latent queries

- Input sequence: Latent queries + Text
 - Fixed number of learned query vectors
 - (Optional) embedded text tokens
- Transformer encoder
 - Shared self-attention for queries and text
 - **Cross-attention to image features only for query tokens**
 - Non-shared Feed Forward layers
- Task-dependent attention masking during pretraining
 - Causal masking / limited query-text interactions
- **Queries encode compressed image or image-text information**



➡ more on pretraining and usage later

MULTIMODAL STRUCTURE: GRAPHS

Reminder:

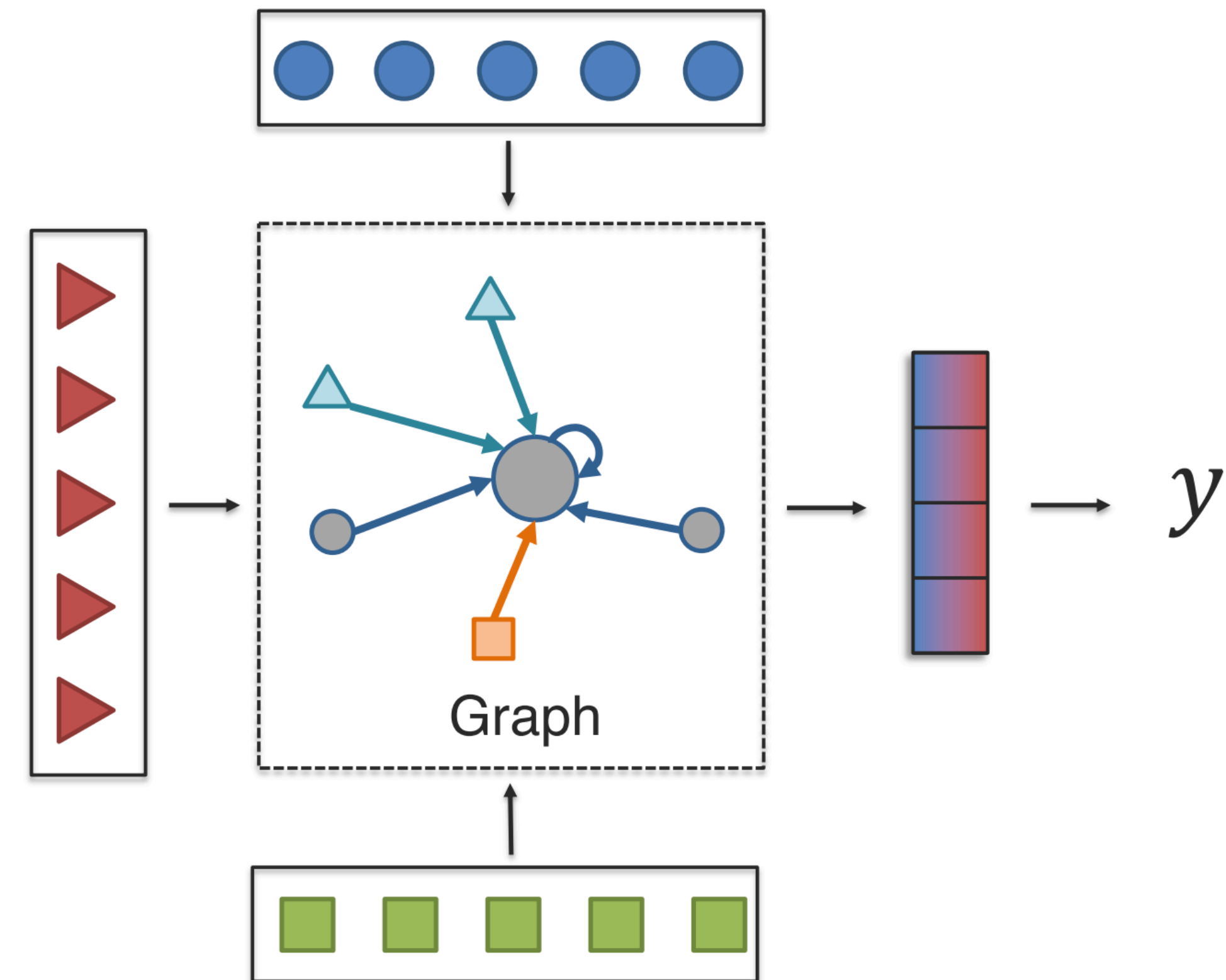
Graph as structure to relate multimodal data embeddings

MULTIMODAL STRUCTURE: GRAPHS

Using graphs as the multimodal structure

- Graph represents the available view of “reality”
- Each modality represents one or more nodes
- Modalities are connected via edges or implicitly via intermediate nodes
- Information exchange along the edges
 - External knowledge could be inserted via edges or intermediate communication nodes

Graph is too complicated, and cannot be done. However, in theory, it should perform better. The more modalities (more than 4-5), we might fully utilize graph structure.

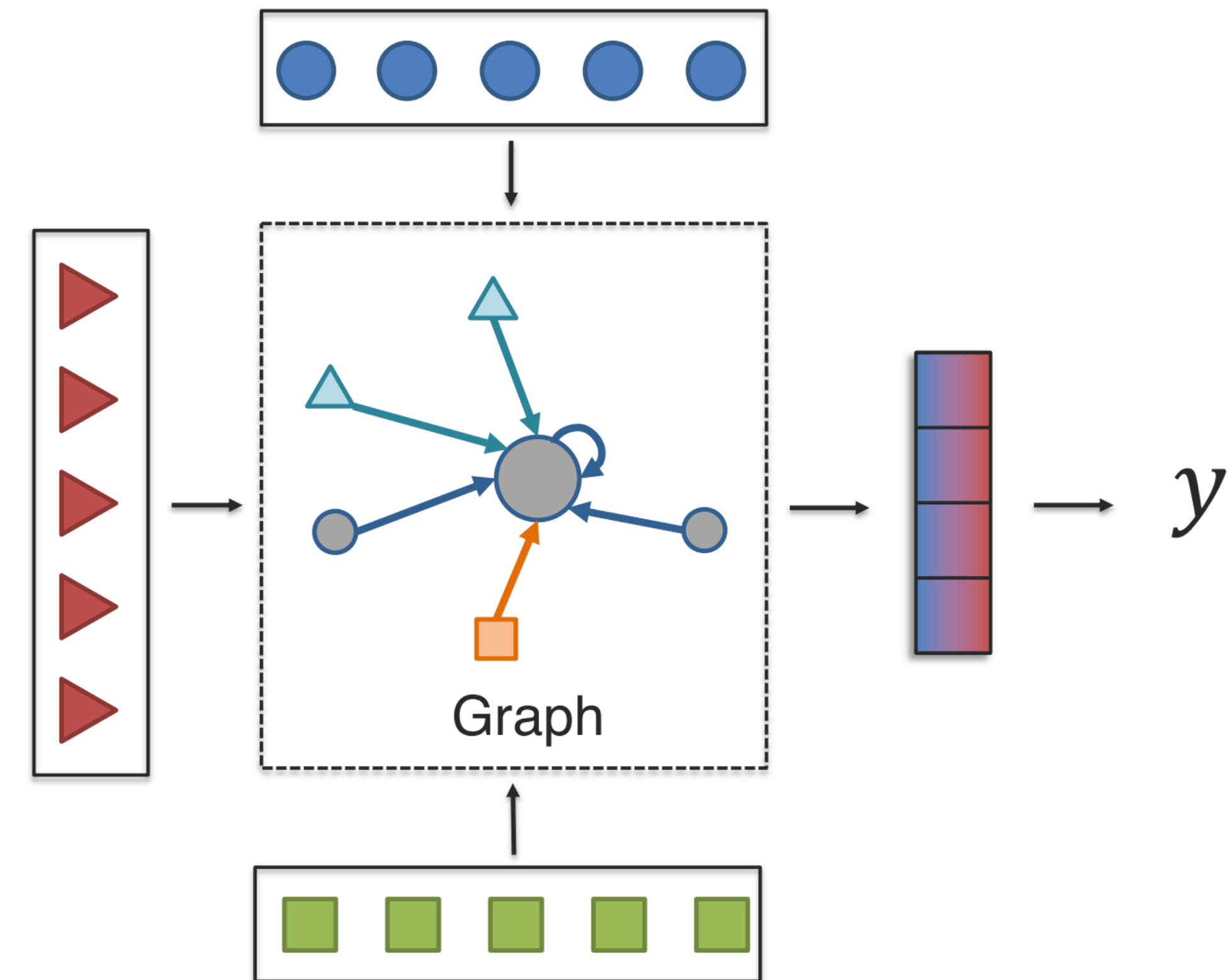


MULTIMODAL STRUCTURE: GRAPHS

Using graphs as the multimodal structure

Example: Medical Information of a Patient

- **Nodes:**
 - Anatomical structures, biological processes, pathologies, ...
 - Imaging modalities
 - Measurements, lab tests, ...
- **Edges:** Based on medical knowledge graphs



MULTIMODAL STRUCTURE: GRAPHS

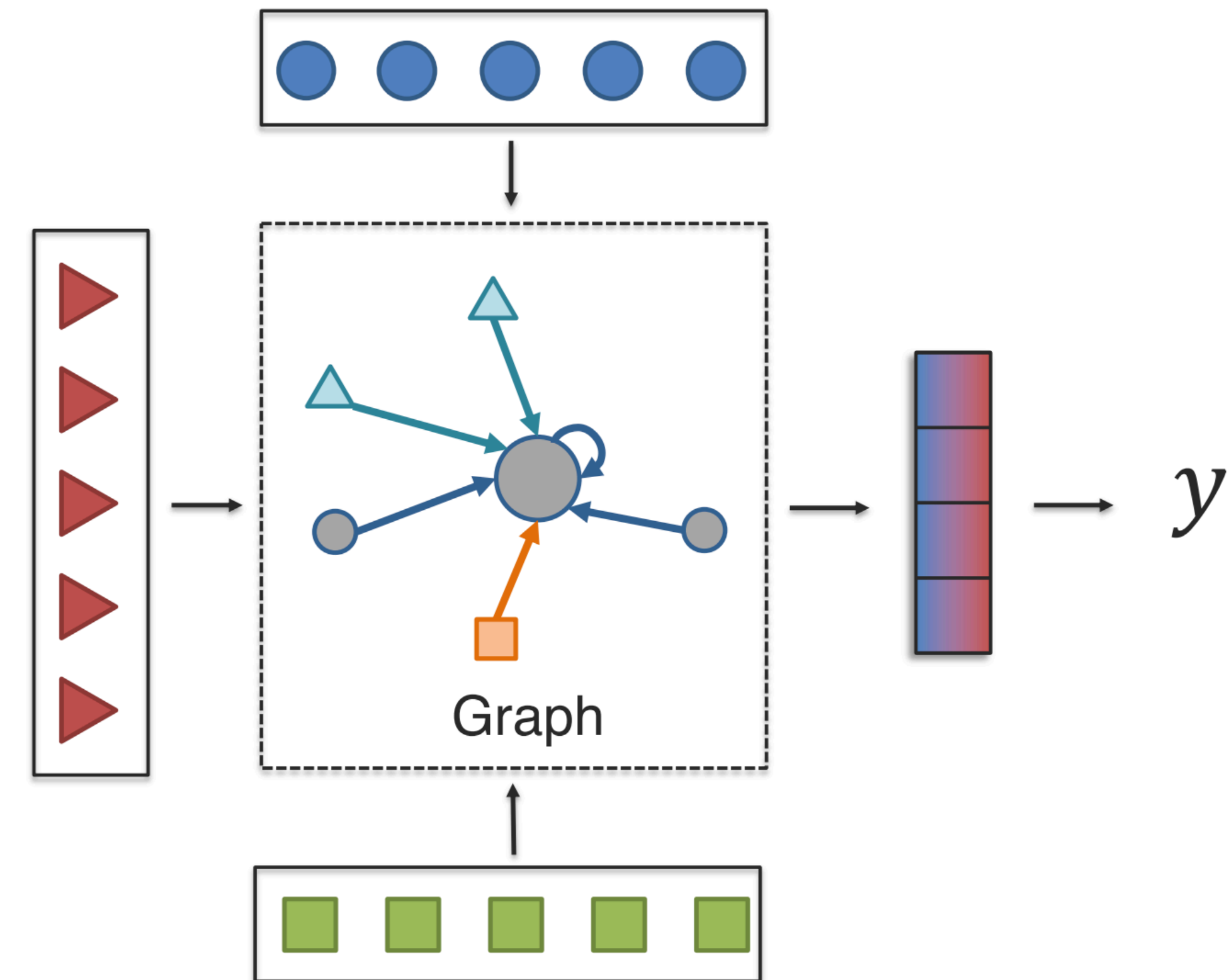
Using graphs as the multimodal structure

Example: Medical Information of a Patient

- **Nodes:**
 - Anatomical structures, biological processes, pathologies, ...
 - Imaging modalities
 - Measurements, lab tests, ...
- **Edges:** Based on medical knowledge graphs

Problems:

- **Data availability (for training AND evaluation)**
- How to design the knowledge graphs
- Computational complexity



WHICH MULTIMODAL STRUCTURE TO CHOOSE

Set	Sequence	Graph	Grid
Flexible structure (compatible with any input)	Versatile structure but canonical ordering required	Heterogeneous structure but edges need to be defined	Restricted to grid modalities
Hard to generate	Autoregressive generation	Hard to generate	Hard to generate
Few pre-trained models available	LLMs ...	Few pre-trained models available	Pretrained models for imaging available
Rarely used in multimodal settings	Most common generalist choice	Only used if meaningful graphs and data are available	Used when images are the primary modality

WHICH MULTIMODAL STRUCTURE TO CHOOSE

Set	Sequence	Graph	Grid
Flexible structure (compatible with any input)	Versatile structure but canonical ordering required	Heterogeneous structure but edges need to be defined	Restricted to graph modalities
Hard to generate	Autoregressive generation	Hard to generate	Hard to generate
Few pretrained models available	Is ...	Few pretrained models available	Pretrained models for images available
Rarely used in multimodal settings	Most common generalist choice	Only used if meaningful graphs and data are available	Used when images are the primary modality

This might be our future

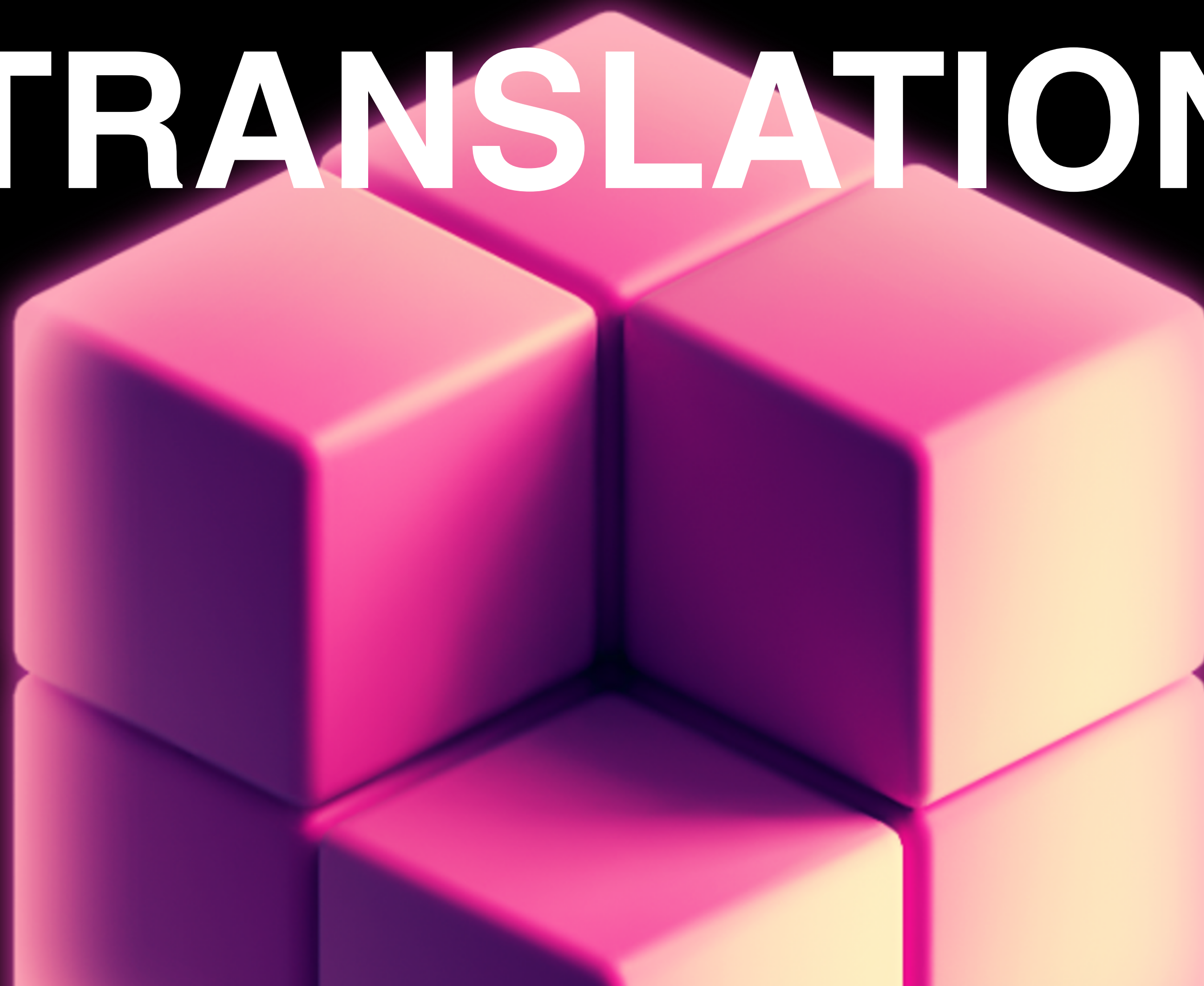
Few benefits over sequences

Easy to use, today's default

Nice to have but not yet tractable

Bad choice for generalist models

Multimodal TRANSLATION



MULTI-MODAL DEEP LEARNING PIPELINES

Multimodal Frameworks
combine different multimodal pipelines

Fusion

to combine multiple inputs or inputs with output queries

Translation

to generate the final outputs

Alignment

to pretrain individual components
to make the components work together
to (pre-)train the full framework

TRANSLATION AS INFORMATION FLOW

**How can we convert from the input modality
to the output modality?**

or

**How can we generate the output modality
conditioned on the input modality**

TRANSLATION WITHIN THE SAME STRUCTURE

Simple Case:

Assume input structure = output structure

- Same type of structure (e.g. grid)
- Same number of elements (e.g. same resolution)
- Aligned/registered input and output

Examples:

- Image to image, segmentation

Solution:

- U-Net

Note: This does not work for sequence-to-sequence

TRANSLATION WITH GLOBAL BOTTLENECK

Simple Case:

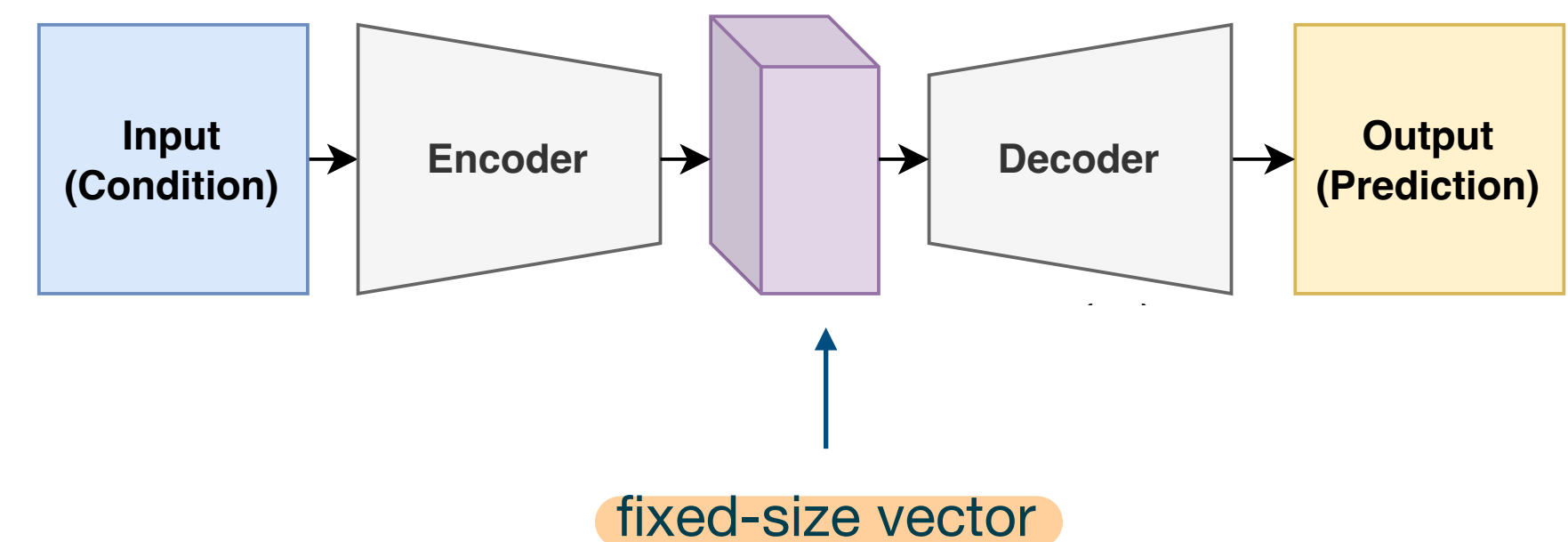
Fixed-size output structure

- (Small) tuple / tabular outputs or global prediction tasks like classification

Solution:

1. Aggregate input into fixed size vector
2. Extend to output from fixed size vector (e.g. MLP)

Problem: information flow limited by bottleneck



How can we make this work in a general case?

TRANSLATION: GENERAL CASE

General Case:

How can we translate between two different structures?

1. Design “queries” for output generation

- Basis of the output structure
- Dependent on generation strategy!

2. Structure fusion between input structure and output conditioning

- Queries are conditioned on input
- Any structural fusion could be used, typically (cross-)attention-based

Information Flow through Queries

TRANSLATION: GENERATION STRATEGIES

(Learned) Query Tokens

Autoregressive

Denoising

TRANSLATION: QUERY-TOKENS

(Learned) Query Tokens

1. Prepare set/sequence of output query tokens

- Learned, input- or task-dependent

2. Condition query tokens on input or latent tokens

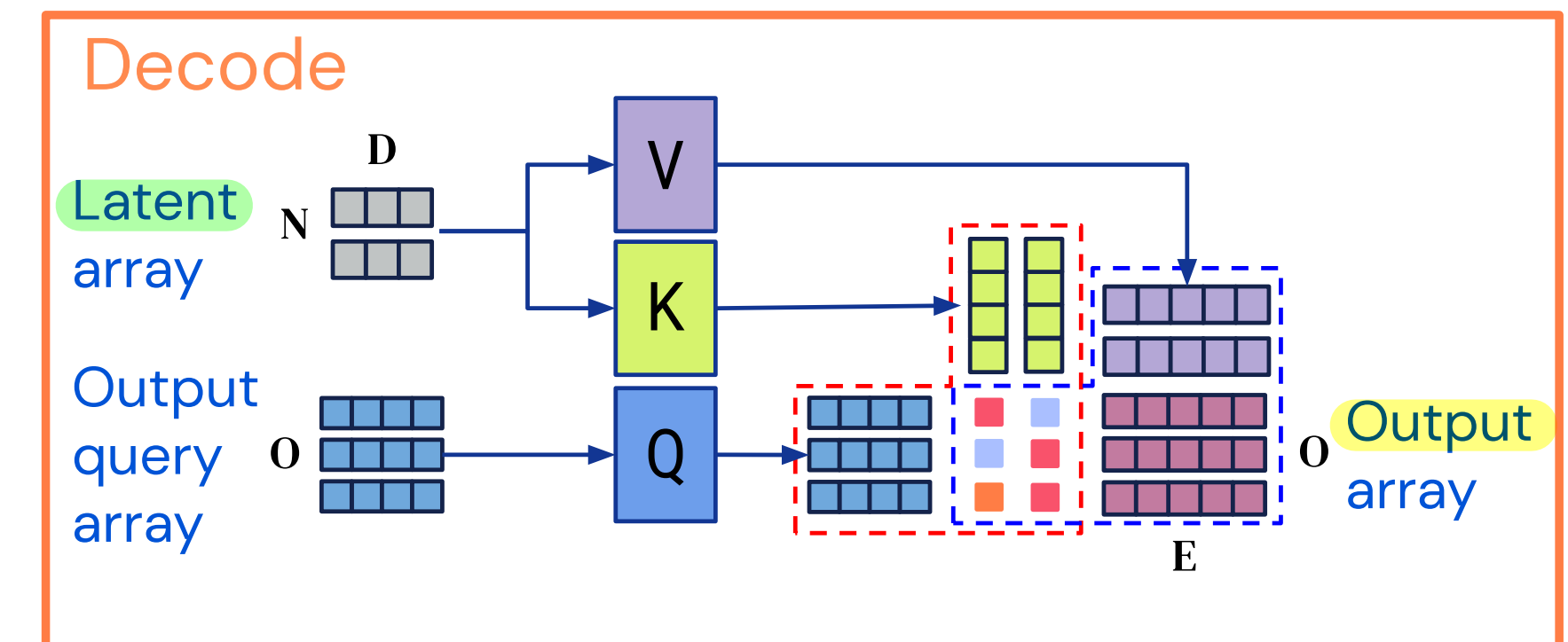
- Typically **cross-attention** with
Q = output queries, **K/V = latents / inputs**
- **Example: Perceiver IO**
(but **K/V** sequence may also be the input sequence)

3. (Optional) **Output token sequence may be further processed**

- E.g., applying a transformer encoder

4. **Prediction Heads**

- Apply general (e.g. token prediction) or **task-specific prediction** heads to it

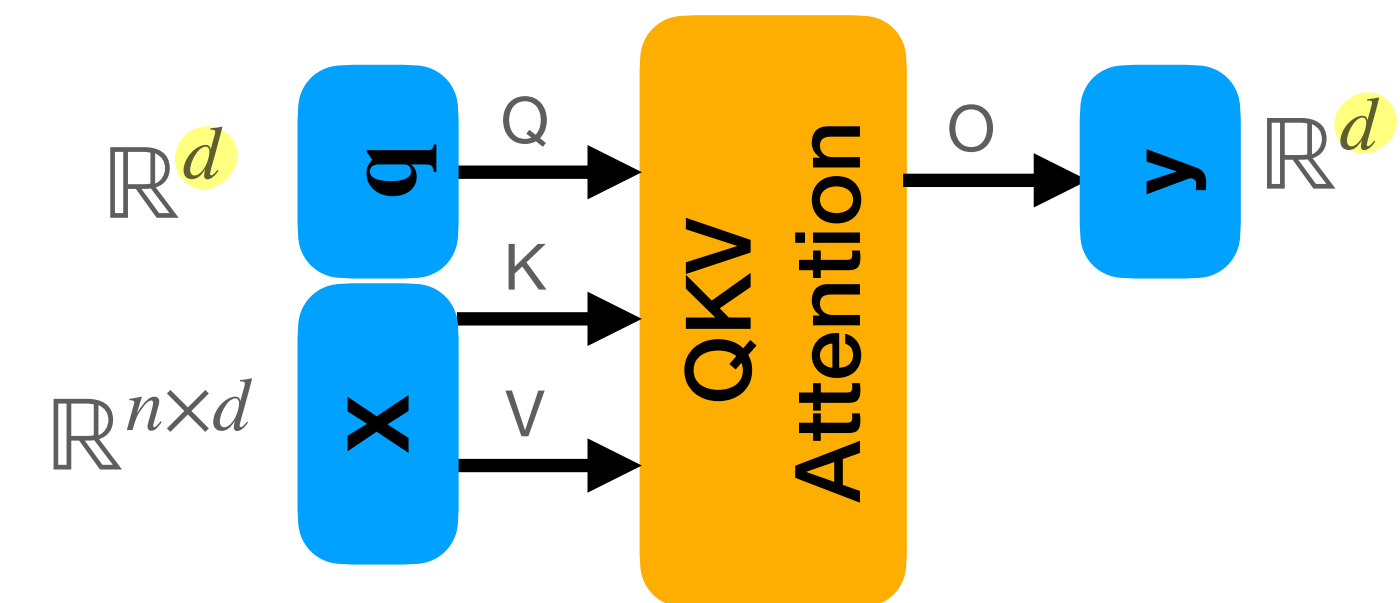


TRANSLATION: QUERY-TOKENS

Types of output query tokens

Single task token

- Single query token
 - ➔ Acts like attention pooling
- Learned individually per task
 - Shared for all samples with the same task
 - E.g. [CLS] token in ViT and similar models
- Used for classification or regression tasks

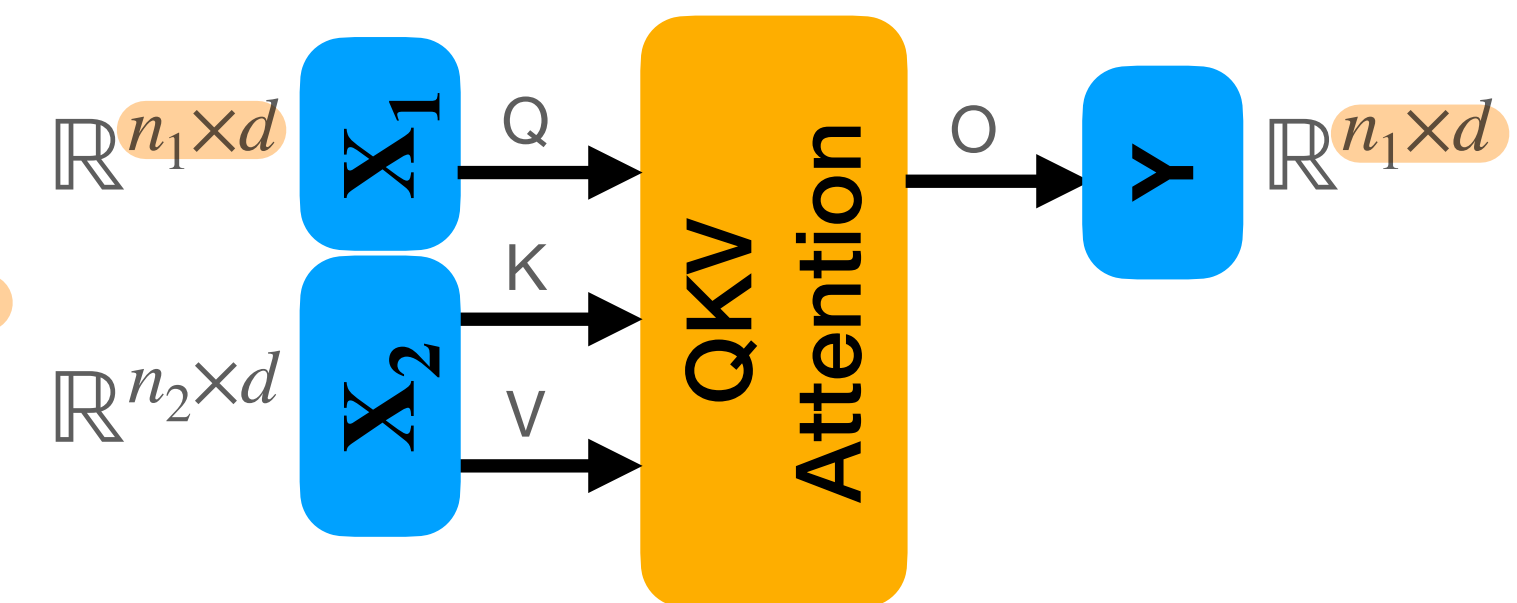


TRANSLATION: QUERY-TOKENS

Types of output query tokens

Multi-task / multi-class tokens

- Set of query tokens
 - ➔ All together used as “query sequence” in cross-attention
 - ➔ Multiple tasks can be predicted at the same time (**tabular / tuple outputs**)
- Learned query token per task or class
 - Each token corresponds to a predicted task / tabular column
 - **Alternatively:** 1 token per class in multiclass or multilabel tasks
- Used for multitask/multilabel classification/regression or tabular predictions

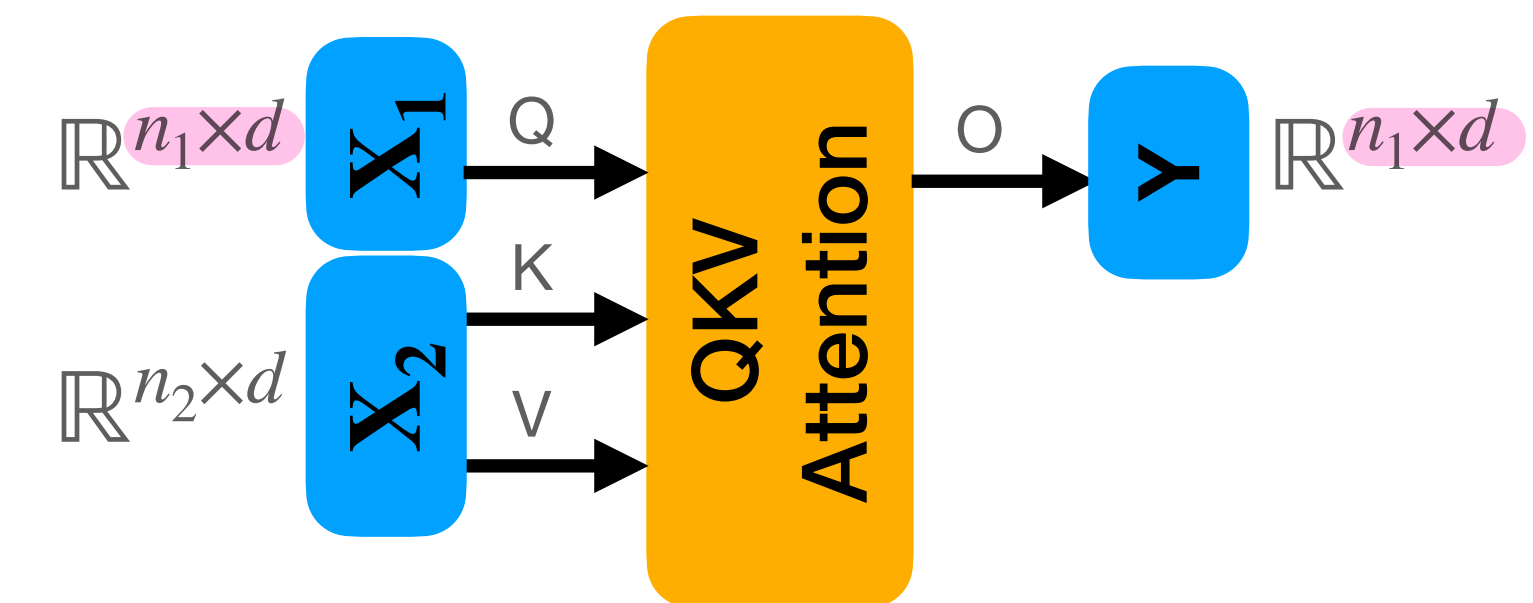


TRANSLATION: QUERY-TOKENS

Types of output query tokens

Positional Tokens

- Set of query tokens
 - ➔ All together used as “query sequence” in cross-attention
 - ➔ Multiple positions can be predicted at the same time
- Learned query token per position (spatial, time, ...)
 - Each token corresponds to a position to be queried
 - Set of queried positions needs to be known (could be input dependent)
- Examples:
 - **Masked data modeling:** position tokens for masked elements
 - **Time series forecasting:** time (t) tokens for forecasted time points (patches)
 - **Inpainting** (time series or image): t or x/y tokens for inpainted region (patches)
 - **Image generation** with known (patch) resolution: x/y tokens for target image



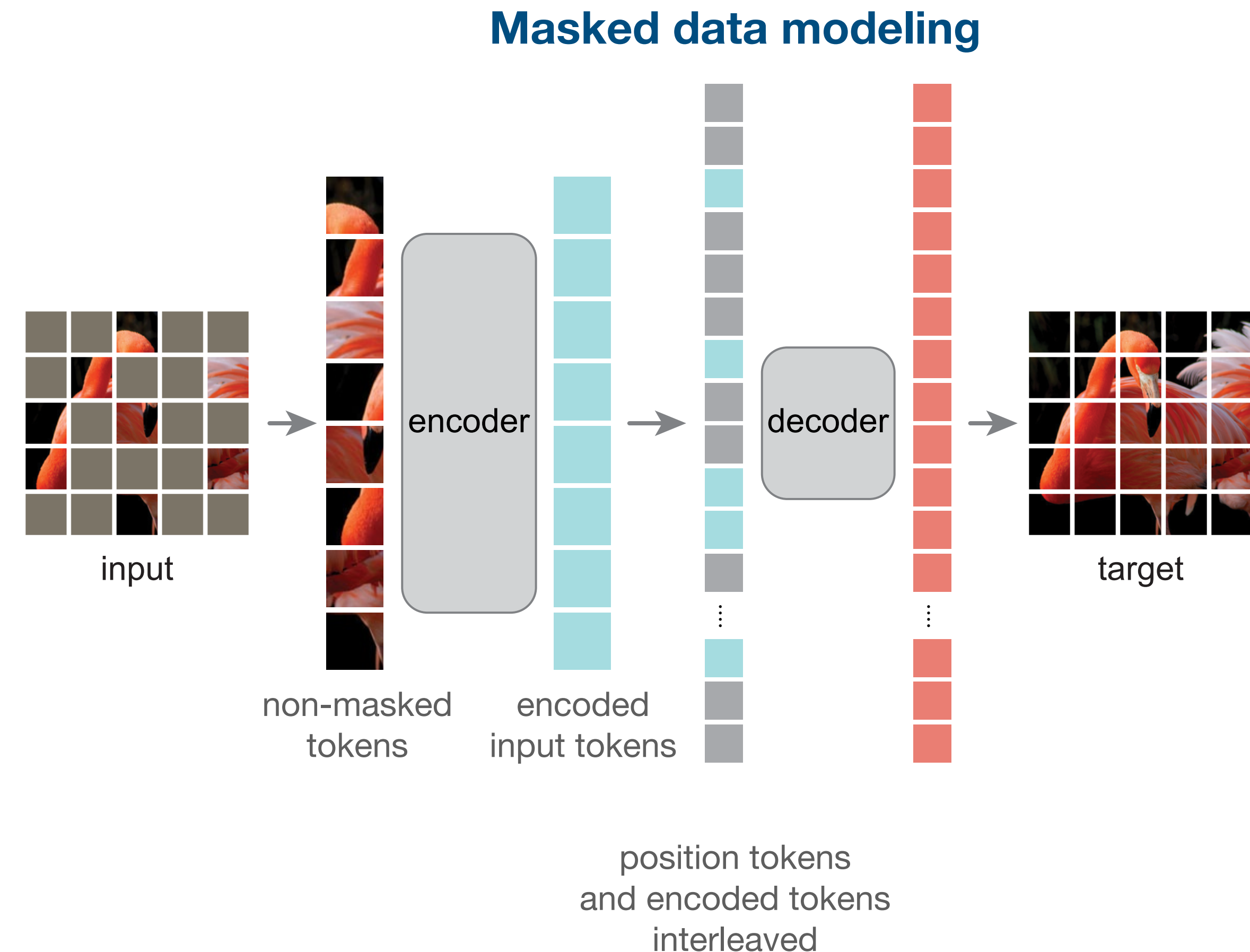
TRANSLATION: QUERY-TOKENS

Types of output query tokens

Positional Tokens

Tasks that extend the input structure:

- **Masked data modeling:**
position tokens for masked elements
- **Time series forecasting:**
time (t) tokens for forecasted time points (patches)
- **Inpainting** (time series or image):
t or x/y tokens for inpainted region (patches)
- **Image generation**
- **Alternative approach:**
Combine encoded input tokens and position tokens of “extended” regions
 - Position tokens fill in the masked/forecasted/inpainted points
 - Use **self-attention instead of cross-attention**

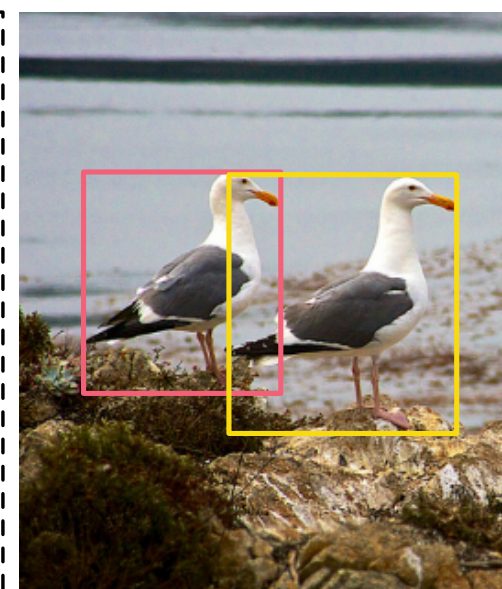
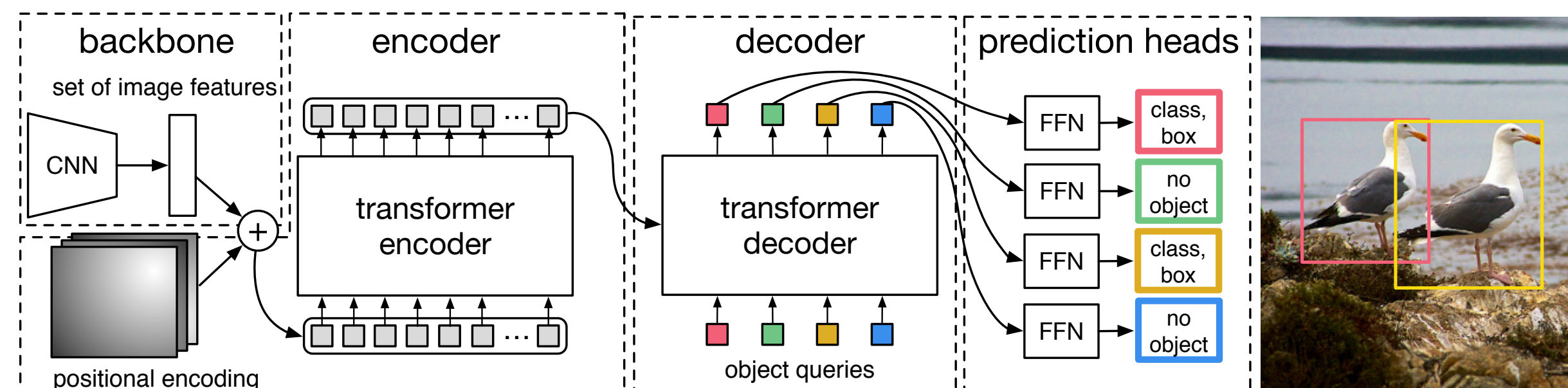
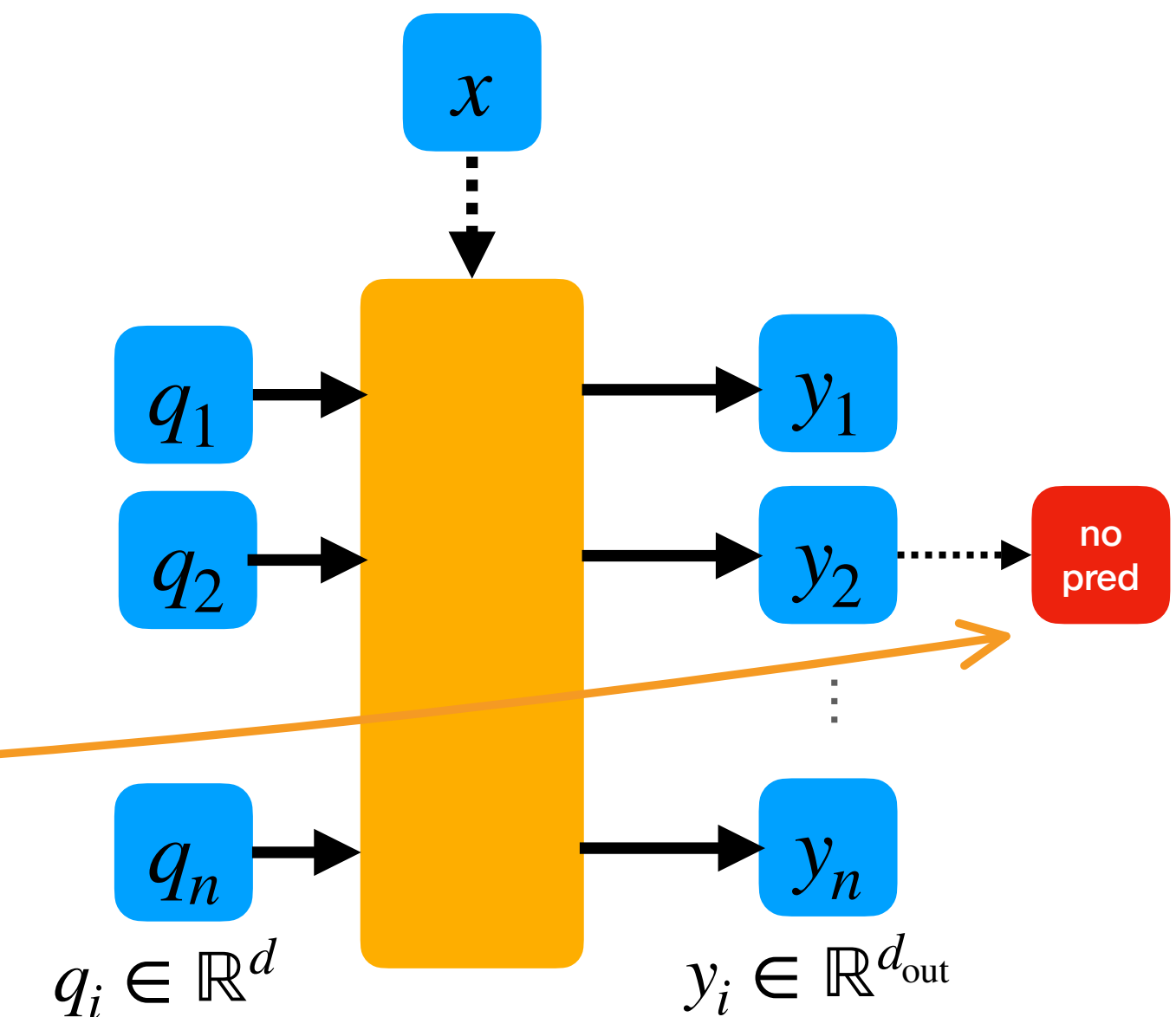


TRANSLATION: QUERY-TOKENS

Types of output query tokens

Dynamic Element Tokens (Set prediction)

- Set of query tokens
 - ➔ All together used as “query set” in cross-attention
 - ➔ Multiple elements can be predicted at the same time
- Learned query token per potential set element
 - Number of query tokens = maximum size of predicted set
 - Classifier (“no-pred”) to enable dynamic set sizes
- Examples:
 - Object detection (DETR)



TRANSLATION: AUTOREGRESSIVE

Autoregressive Generation

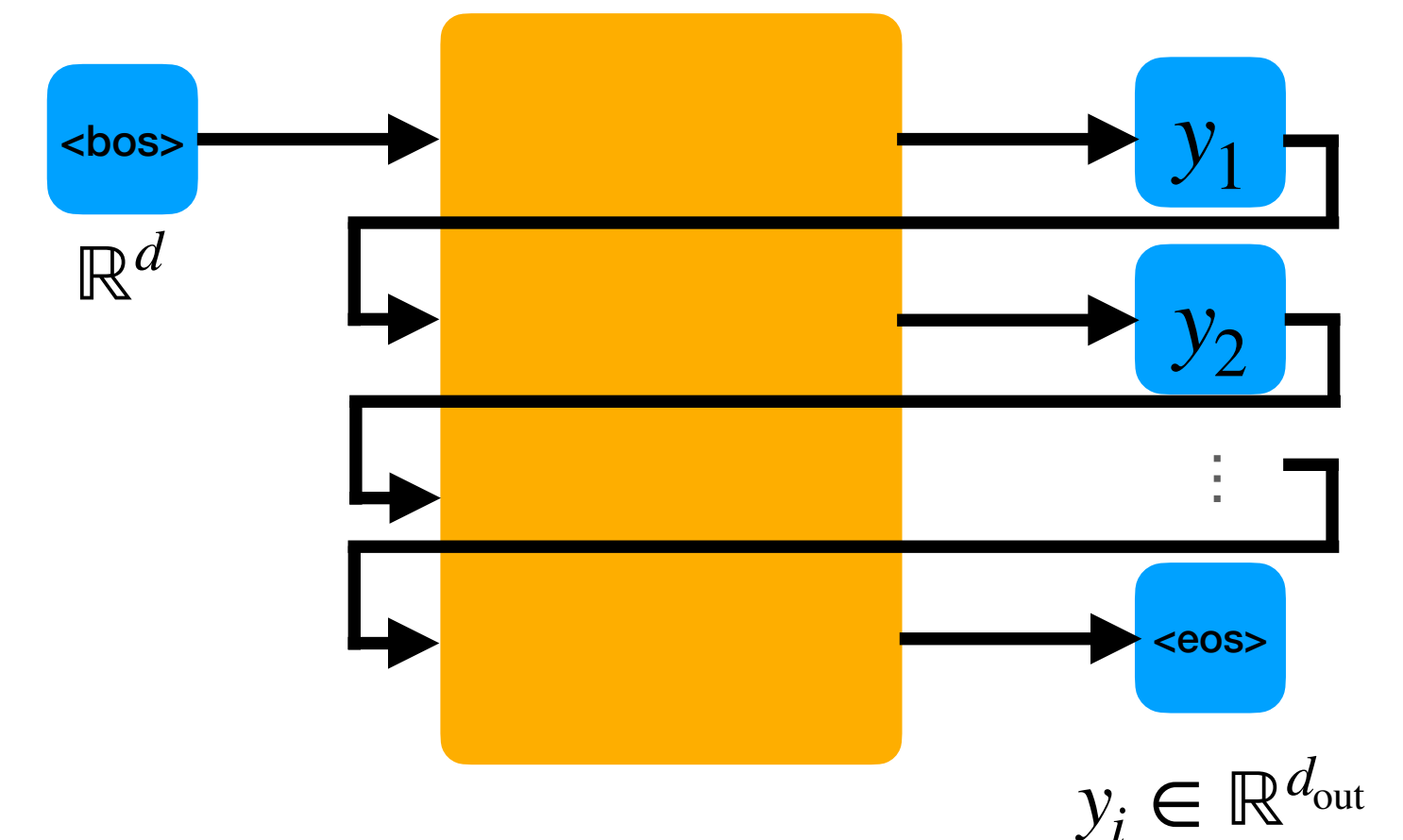
(see slide 111 in Set/Sequence/Tuples lecture)

Requirements:

- Canonical ordering of output structure (representations as a sequence)
- Unidirectional context (left-to-right or right-to-left)

Procedure: Generate step-by-step (Next Token Prediction)

1. Start from **<bos>** (begin of sequence) token
2. Predict next token conditioned on previous ones
 - **Causal decoder model required**
3. Stop generating when stopping criteria is reached
 - Stop when **<eos>** (end of sequence) token is predicted
 - Other criteria, e.g. max number of tokens



TRANSLATION: AUTOREGRESSIVE

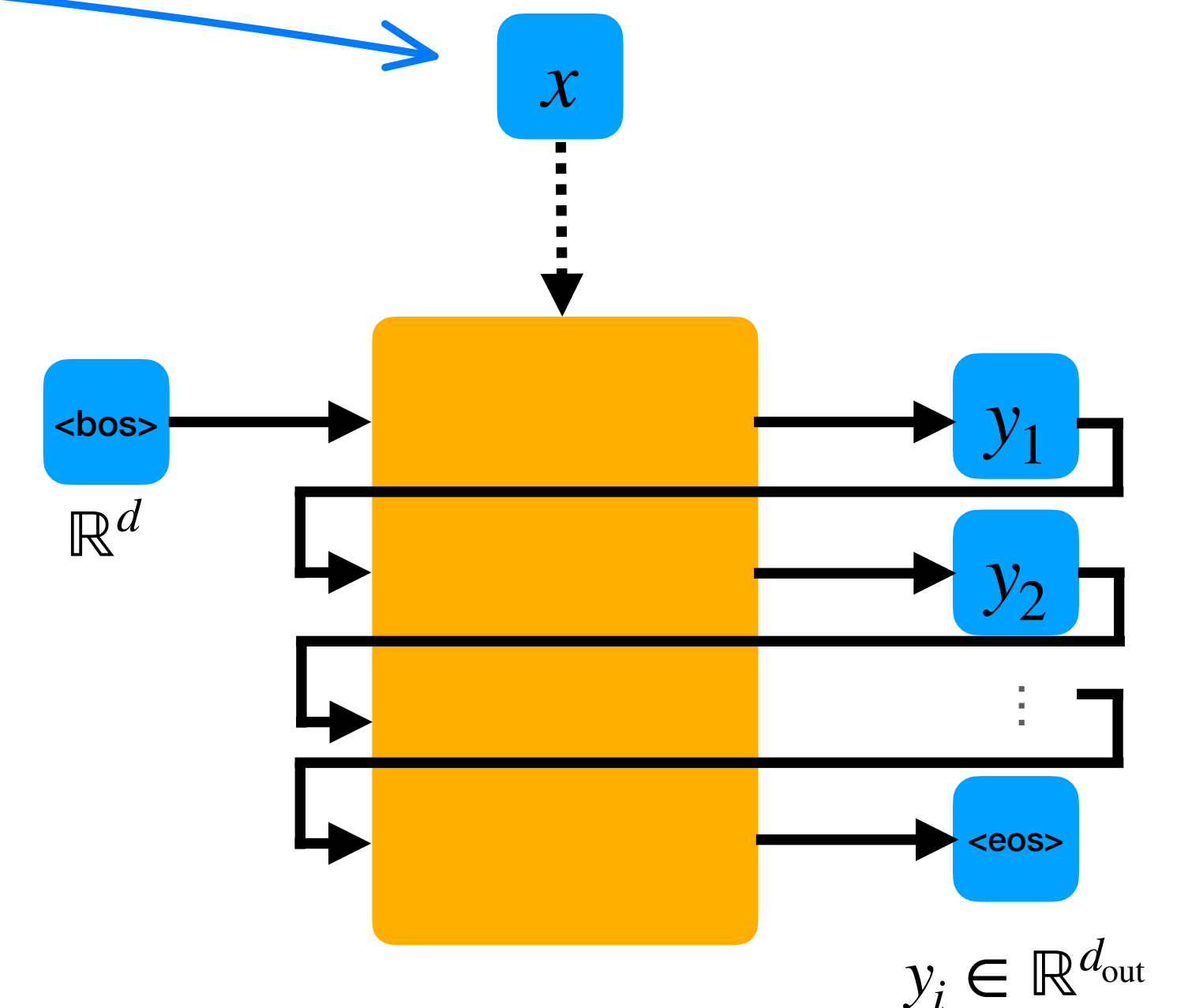
Autoregressive Generation

Model Conditioning:

Predict token y_i based on
all previous tokens $y_{<i}$ (and $\langle \text{bos} \rangle$)
and all encoded input tokens x

➔ Fusion of previous output sequence (the **prefix**)
with input sequence (the **context**)

- Prefix (i.e. the query/Q) is conditioned on the input context (e.g. K/V in attention)
- **Example:**
 - **transformer decoder** with cross-attention
 - but also multi-stream or other fusion methods possible



TRANSLATION: AUTOREGRESSIVE

Autoregressive Generation

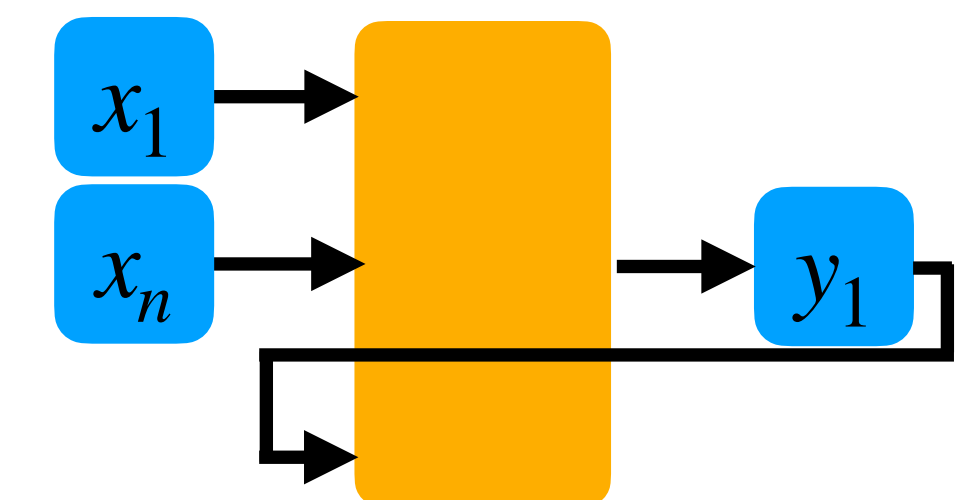
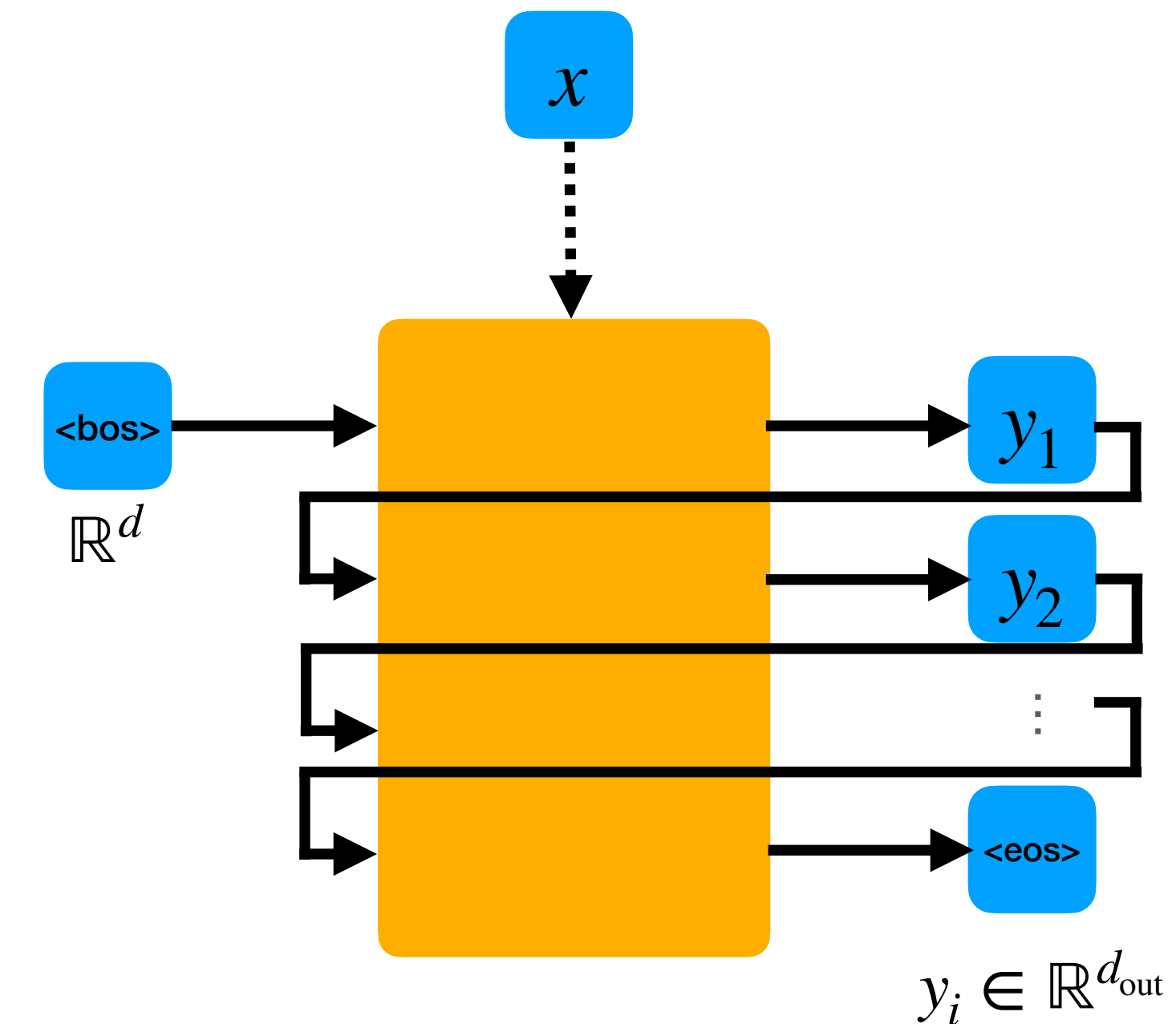
Model Conditioning:

Predict token y_i based on
all previous tokens $y_{<i}$ (and $\langle \text{bos} \rangle$)
and all encoded input tokens x

➔ Fusion of previous output sequence (the **prefix**)
with input sequence (the **context**)

- **Alternative: Prefix Conditioning**

- Input sequence concatenated with previous output sequence
- Both together form a single prefix
- **Self-attention-based, no cross-attention needed**
- Single-stream or multi-stream encoders possible

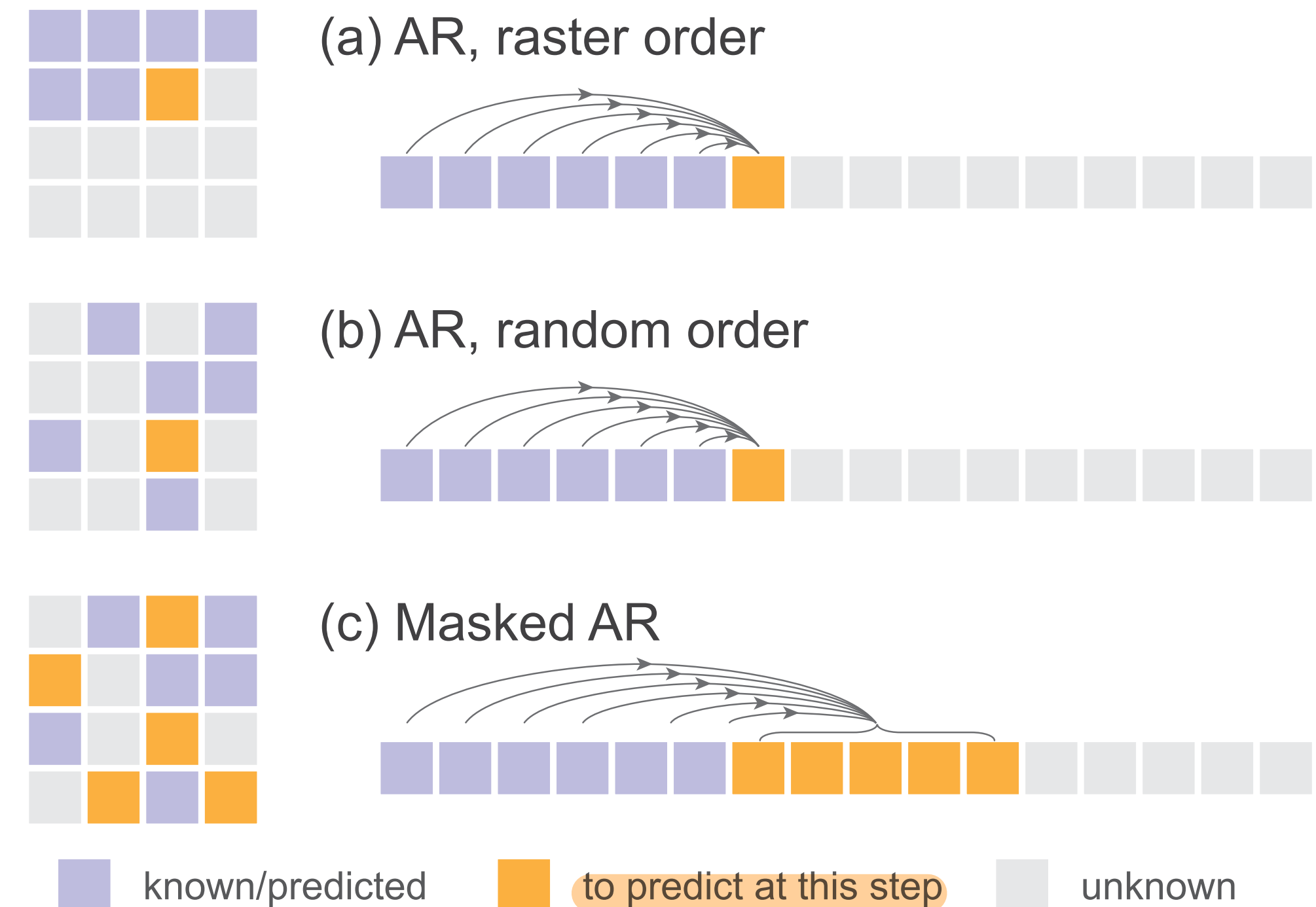


TRANSLATION: AUTOREGRESSIVE

Autoregressive Generation

Examples:

- Text generation / output
- Autoregressive time series forecasting
- **Autoregressive image generation**
ordering needs to be defined!
 - e.g. pixel-level prediction
 - or VQ-VAE token prediction

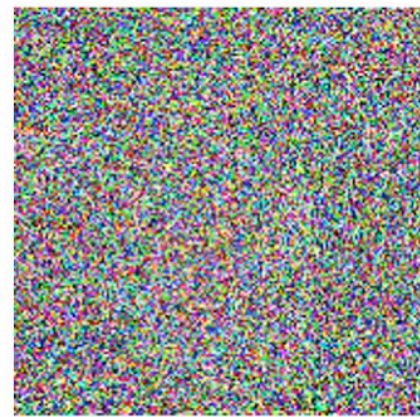


Li et al (2024)

TRANSLATION: NOISY-BASED

Diffusion Models: Generating image conditioned on other modalities

- Start with a noise image

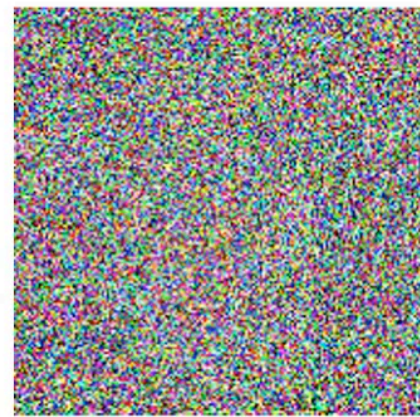


- **This is the (initial) query !**
(i.e. the query has a grid structure)

TRANSLATION: NOISY-BASED

Diffusion Models: Generating image conditioned on other modalities

- Start with a noise image



- **This is the (initial) query !**
(i.e. the query has a grid structure)

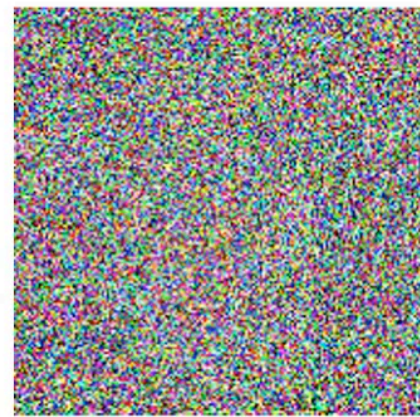
- Denoise the image in multiple steps



TRANSLATION: NOISY-BASED

Diffusion Models: Generating image conditioned on other modalities

- Start with a noise image



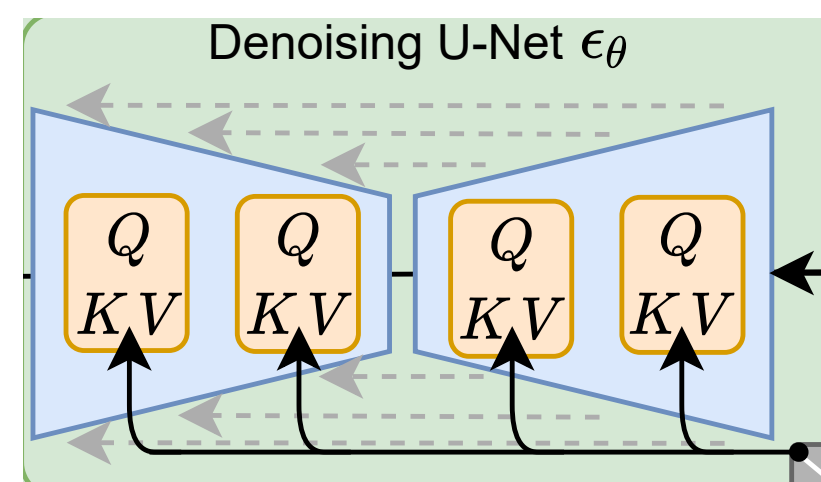
- **This is the (initial) query !**
(i.e. the query has a grid structure)

- Denoise the image in multiple steps



➔ **More on Diffusion Models in the guest lecture**

- Each denoising step is conditioned on the (encoded) input modality



- After the first step, the **previously denoised image is the new query!**
- **Injection fusion into primary structure (the query grid/image)**
E.g. using cross-attention in conditioned U-Nets
Q=image, K/V=input modality
(see structural fusion)

TRANSLATION

**Typically,
translation is the fusion between
the input modality
and an output query**

(e.g. query tokens, autoregressive prefix, noise)

Multimodal ALIGNMENT (How to train)

An abstract geometric background featuring a cluster of 3D cubes. The cubes are rendered in a gradient of colors, ranging from deep purple to bright pink, with some faces appearing more illuminated than others, creating a sense of depth and perspective. The cubes are arranged in a way that they seem to be part of a larger, partially visible structure.

MULTI-MODAL DEEP LEARNING PIPELINES

Multimodal Frameworks
combine different multimodal pipelines

Fusion

to combine multiple inputs or inputs with output queries

Translation

to generate the final outputs

Alignment

to pretrain individual components
to make the components work together
to (pre-)train the full framework

ALIGNMENT

How can we train multimodal models

Given

- Paired multimodal data, i.e. samples with multiple modalities
- (Optional) unimodal data
- (Optional) pretrained encoder/decoder models (frozen or fine-tuned)

ALIGNMENT: TYPES OF OBJECTIVES

Disciminative Representation Learning

Reconstruction/Generative Representation Learning

Supervised / Multi-task Training

} SSL

DISCRIMINATIVE ALIGNMENT: DUAL ENCODER

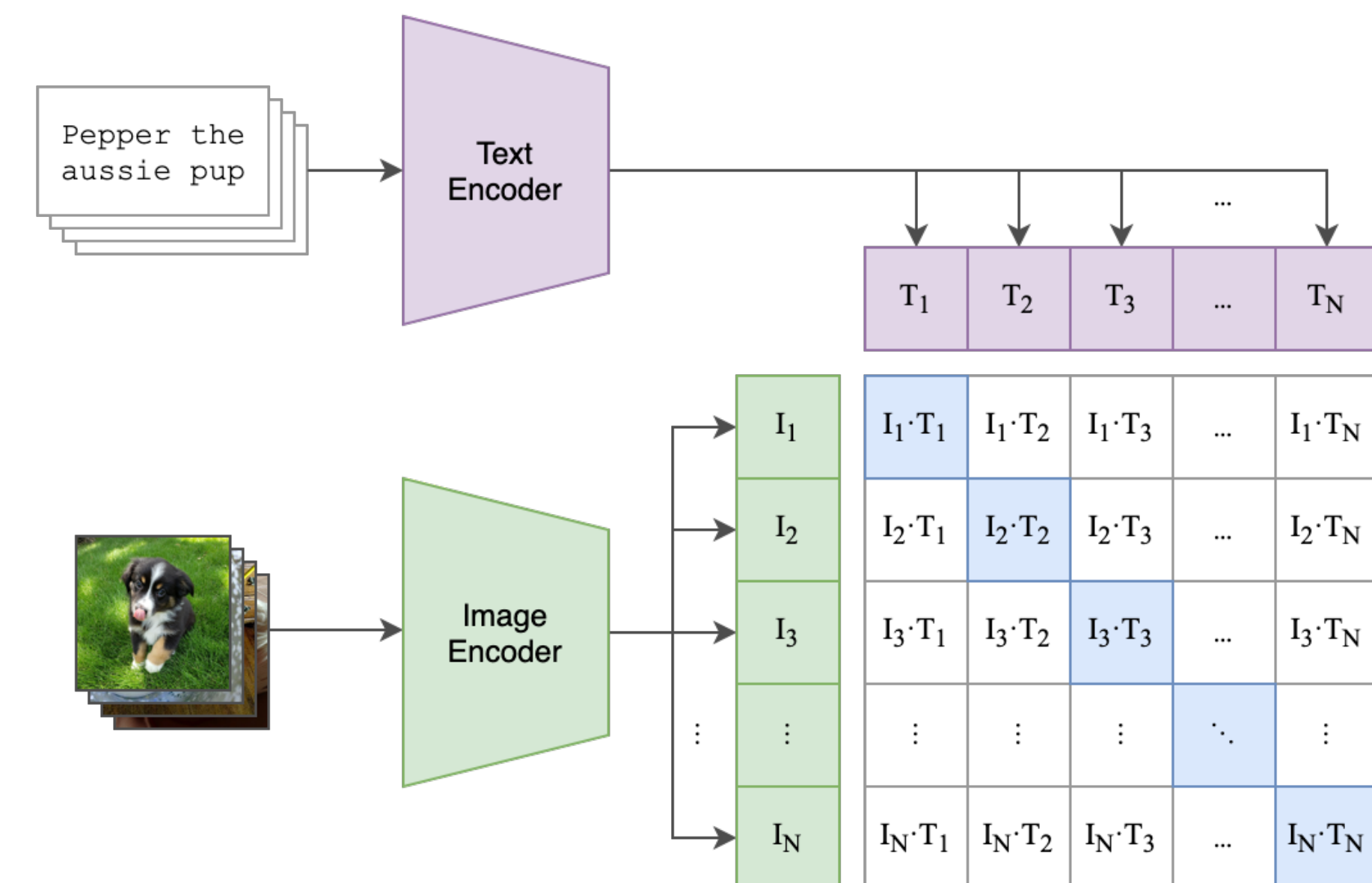
Reminder: Multimodal Contrastive Learning (CLIP)

(see Losses lecture slide 60)

Learn multimodal representations
by predicting which modality pairs are correct.

Dual Encoder:

Separate encoders for both modalities



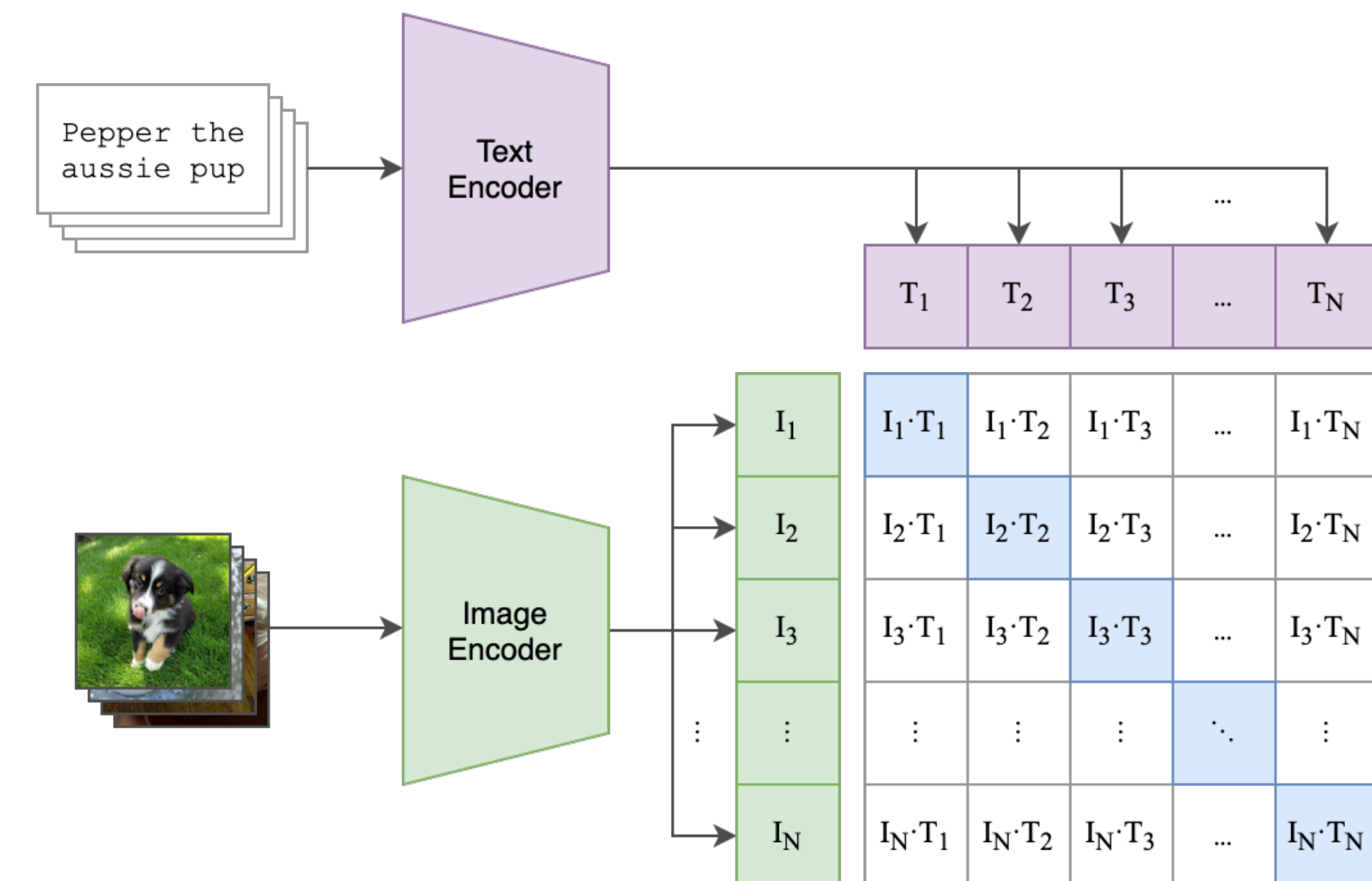
$$L_{CLIP_i} = (L_{I_i} + L_{T_i})/2,$$

$$\text{with } L_{I_i} = -\log \frac{\exp(\text{sim}(I_i, T_i)/\tau)}{\sum_{k=0}^{2N-1} \mathbf{1}_{k \neq i} \exp(\text{sim}(I_i, T_k)/\tau)} \text{ and } L_{T_i} = -\log \frac{\exp(\text{sim}(I_i, T_i)/\tau)}{\sum_{k=0}^{2N-1} \mathbf{1}_{k \neq i} \exp(\text{sim}(I_k, T_i)/\tau)}$$

DISCRIMINATIVE ALIGNMENT: DUAL ENCODER

Limitation of CLIP: No interactions between modalities

- Independent encoders! (dual encoder)
- Features never fuse/interact
 - alignment only on final features
 - general feature extraction independent of other modality

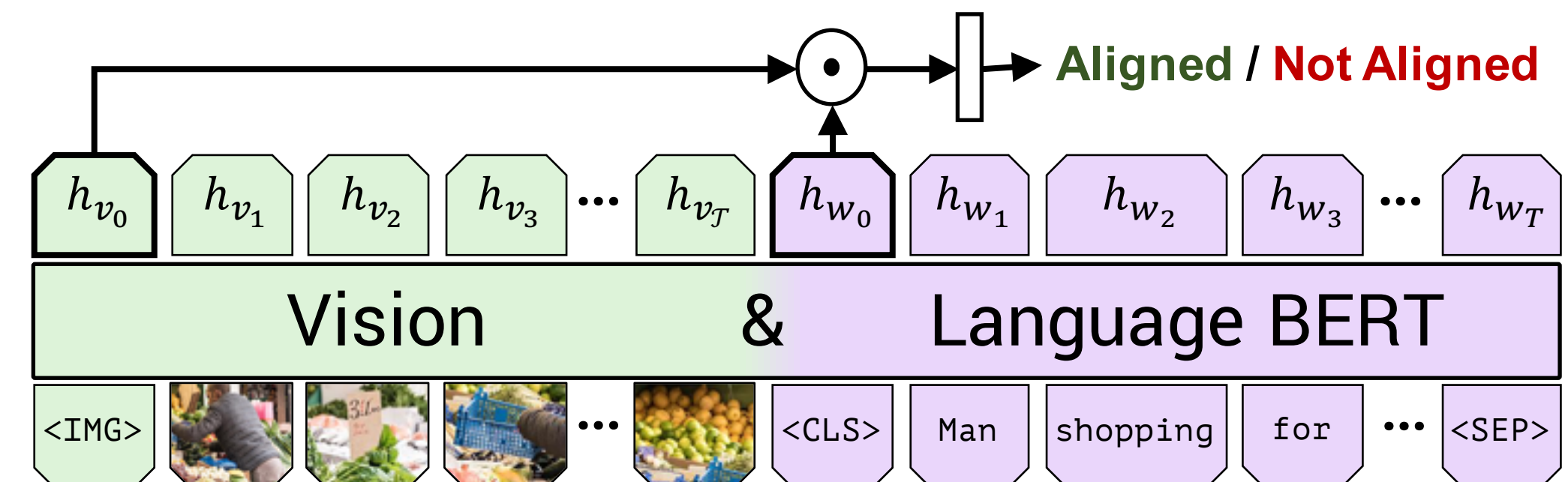


DISCRIMINATIVE ALIGNMENT: JOINT ENCODER

Solution: Joint Encoder

“Multimodal Matching” or “Multimodal Alignment Prediction”

- **Joint multimodal encoder (fusion)**
 - Features are dependent on both modalities
 - E.g. multisteam fusion
- Binary matching prediction based on joint features
 - E.g. cosine similarity btw. stream features (each stream feature is influence by both modalities)
 - Or binary prediction head on “multimodal” features
- **Training:** Either matched (positive) or unmatched (negative) pair
 - ➡ Apply BCE loss



DISCRIMINATIVE ALIGNMENT: JOINT ENCODER

Dual-Encoder vs. Joint Encoder

	Dual Encoder (CLIP)	Joint Encoder (Multimodal Matching)
Interactions	No interaction btw modalities	Interaction btw modalities
Complexity (matching N with M samples)	$N + M$ passes	$N \times M$ passes
Sensitivity to Negatives	Large (many negatives required)	Small (1 negative per positive)
Benefits and Use	Efficient similarity computation of many pairs	Nuanced matching of pre-selected pairs

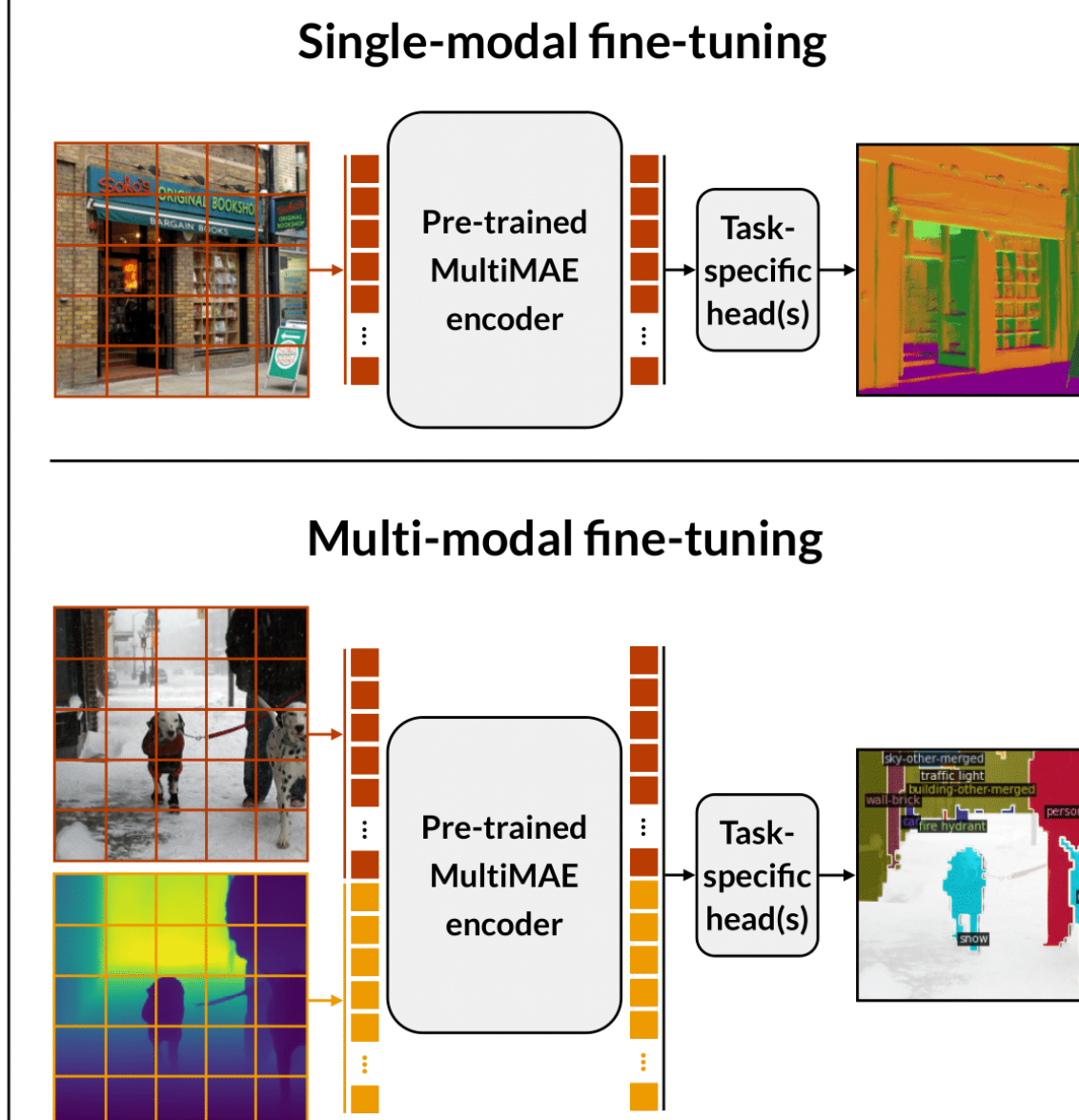
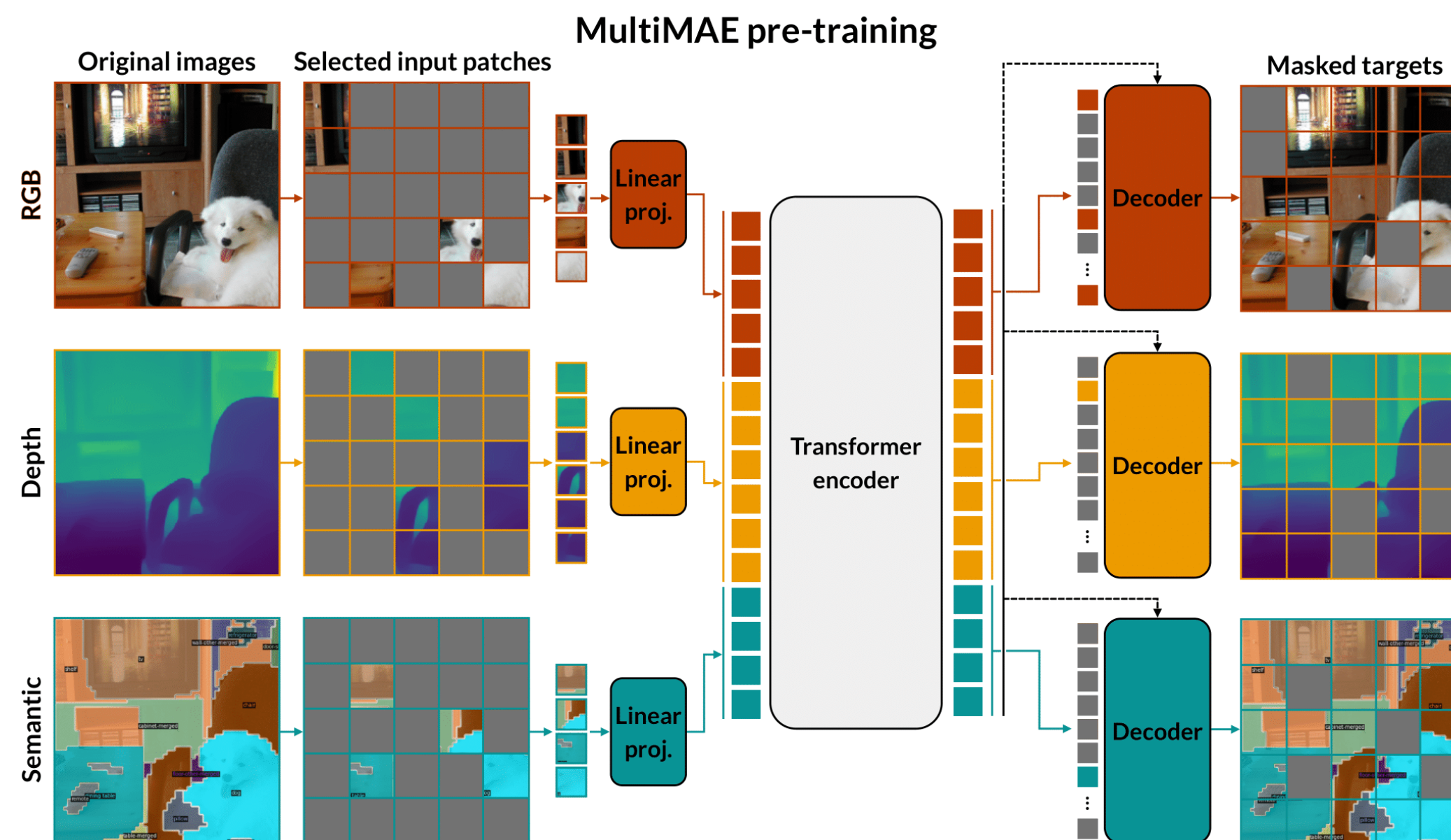
RECONSTRUCTION-BASED ALIGNMENT: MASKING

Reminder: **Multimodal Masked Autoencoder (MultiMAE)**

“**Multimodal Masked (Data) Modeling**”

(see Losses lecture slide 68)

Joint Masking: Randomly masks out portions of *all* input modalities.



RECONSTRUCTION-BASED ALIGNMENT: MASKING

Reminder: **Multimodal Masked Autoencoder (MultiMAE)**

“Multimodal Masked (Data) Modeling”

(see Losses lecture slide 68)

Encoder:

- **Shared encoder**, e.g. multistream / modality experts

Decoder Variants:

- **One decoder per modality**
e.g. **with cross-entropy or pixel MSE loss**
(MultiMAE model)
- **Token prediction head with cross-entropy loss** on shared vocabulary,
e.g. with VQ-VAE tokens
(BeiT-3 and 4M model)
- **Additional labels as targets**,
e.g. “object classes” for image regions
(ViBERT)

RECONSTRUCTION-BASED ALIGNMENT: AUTOREGRESSIVE

Reminder: Multimodal Autoregressive (Data) Modeling

(see Losses lecture slide 70)

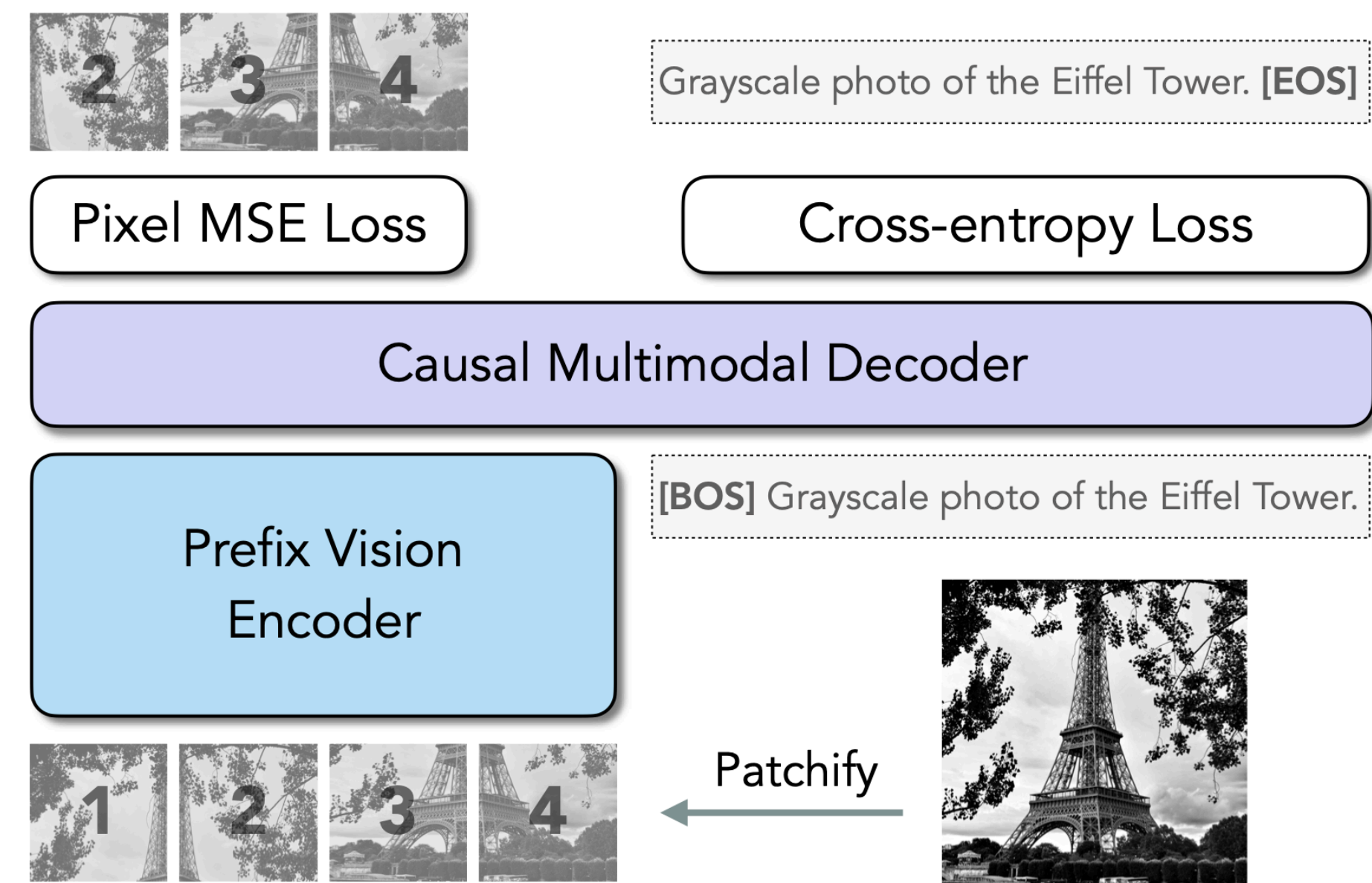
Predict next tokens conditioned on previous using causal masking

Types of loss function

- Crosse-entropy (for discrete tokens)
- MSE loss (for pixel-level or embedding preds)

Variants

- Fully autoregressive with or without interleaved modalities
- Prefix-based
e.g. condition text generation on image tokens



RECONSTRUCTION-BASED ALIGNMENT: UNIMODAL

Uni-modal reconstruction (masked or autoregressive) may be used as supplementary losses

- Apply to unimodal inputs given to (shared) multimodal encoder
- ➔ Unimodal “understanding” is also important for multimodal “understanding”
- ➔ Shared components (e.g. in multimodal experts) are also trained with unimodal training
- ➔ **There is much more unimodal than paired data**

SUPERVISED ALIGNMENT: MULTIMODAL MULTITASK TRAINING

Multimodal Supervised Training

Examples:

- Visual Question Answering
- Image Classification as Textual Answers
- Multimodal classification / regression / segmentation / ...

“Hybrid Approach”: Instruction Tuning

Autoregressive / MAE training
based on supervised multi-task data
arranged as sequence-to-sequence tasks

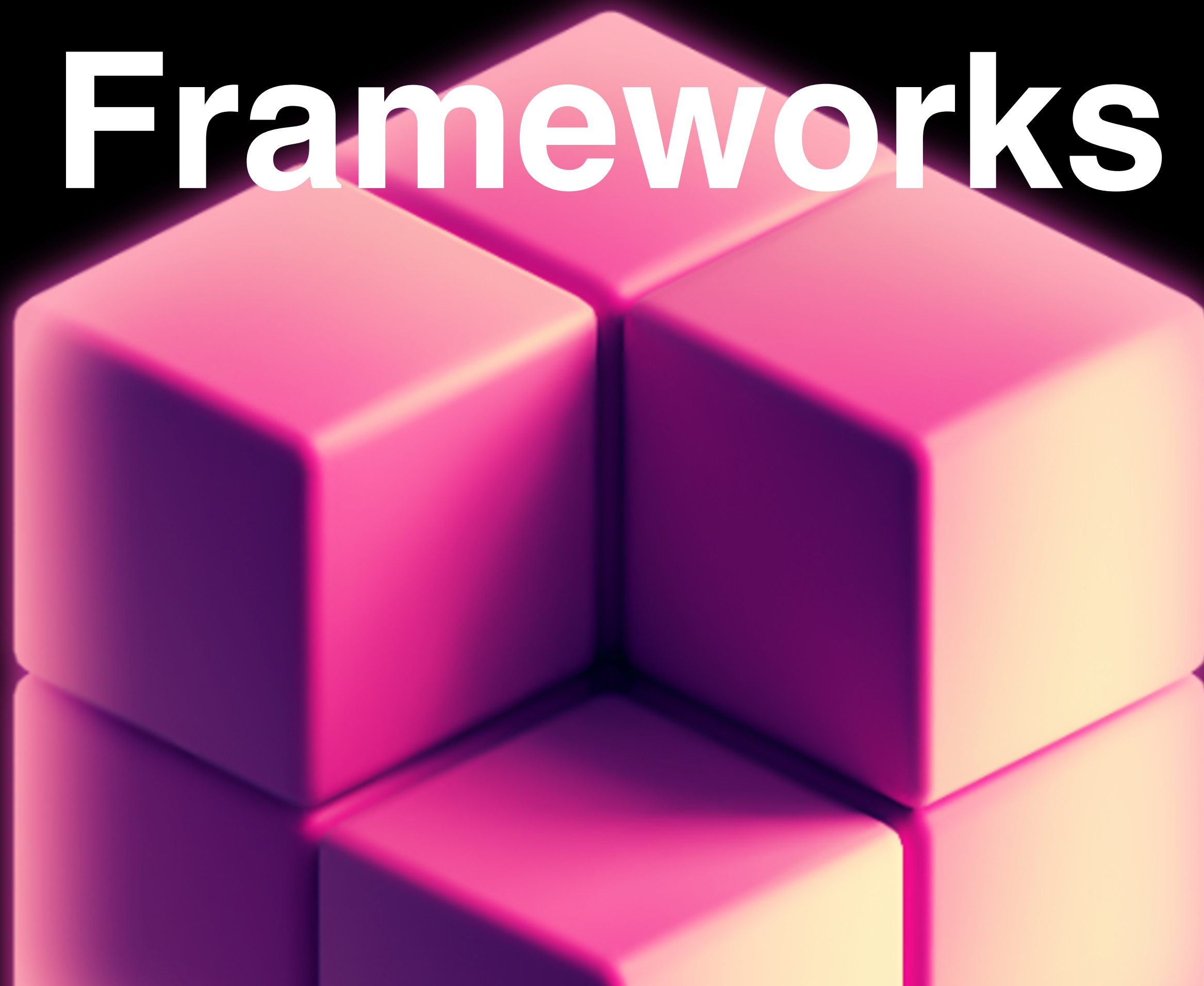
ALIGNMENT IN PRACTICE

Often a combination of different objectives is used

Often pre-trained components are used

Often multiple training stages are used

Multimodal Frameworks



MULTIMODAL FRAMEWORKS

Multimodal Frameworks
combine different multimodal pipelines

Fusion

to combine multiple inputs or inputs with output queries

Translation

to generate the final outputs

Alignment

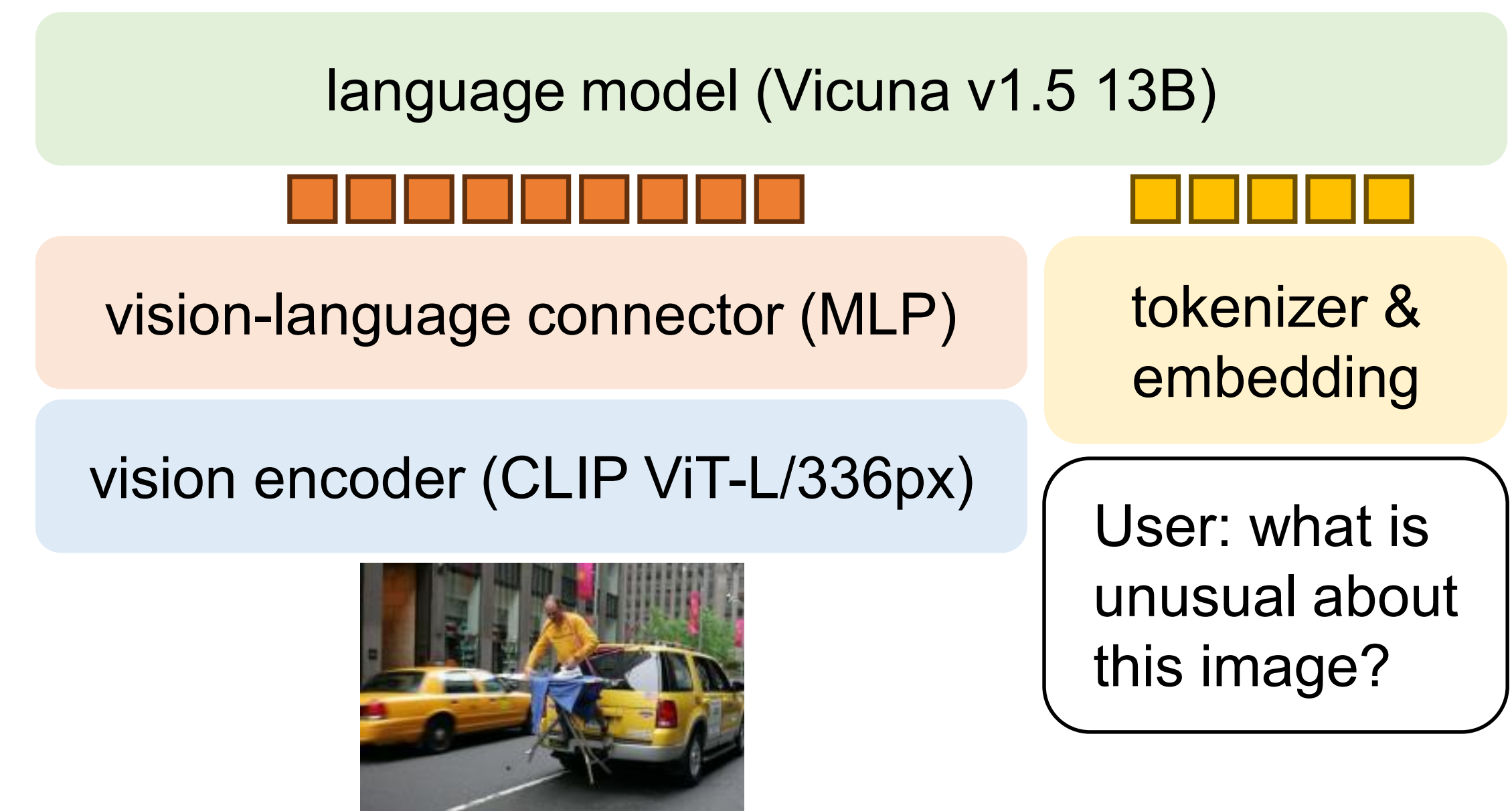
to pretrain individual components
to make the components work together
to (pre-)train the full framework

THE LLAVA FRAMEWORK

Autoregressive Vision Language Chat Bot

Approach

- Low-resource, open data training
 - ~1 day on 8 A100s (Llava 1.5)
 - ~1.2 M public vision-language samples
- Pre-trained image encoder and LLM
- MLP projection and sequence concatenation
- Autoregressive training
 - on visual instruction tuning data



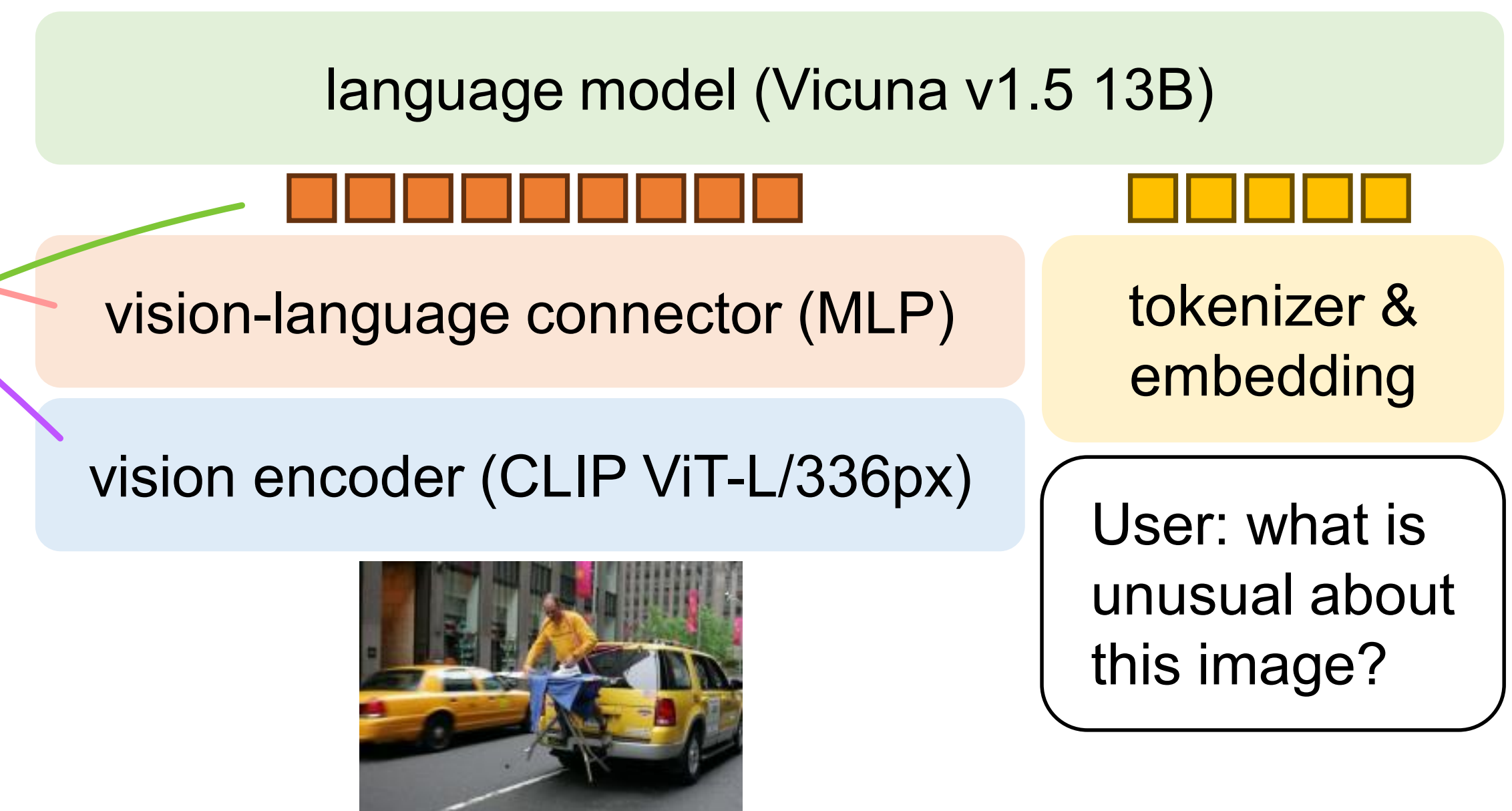
[Liu et al \(2023a\)](#)

[Liu et al \(2023b\)](#)

THE LLAVA FRAMEWORK

Architecture

- **Image encoder**
 - Pre-trained and frozen
 - Feature vector per patch
- **MLP Projection**
 - Trainable (these are the main parameters)
 - Image encoder dim $\rightarrow$ LLM dim
 - Projects each patch token independently
- **LLM**
 - Pre-trained and (sometimes) frozen
- **Sequence concatenation**
 - Flatten projected image patches
 - Concatenate with (embedded) text tokens
 - Add positional embeddings afterwards



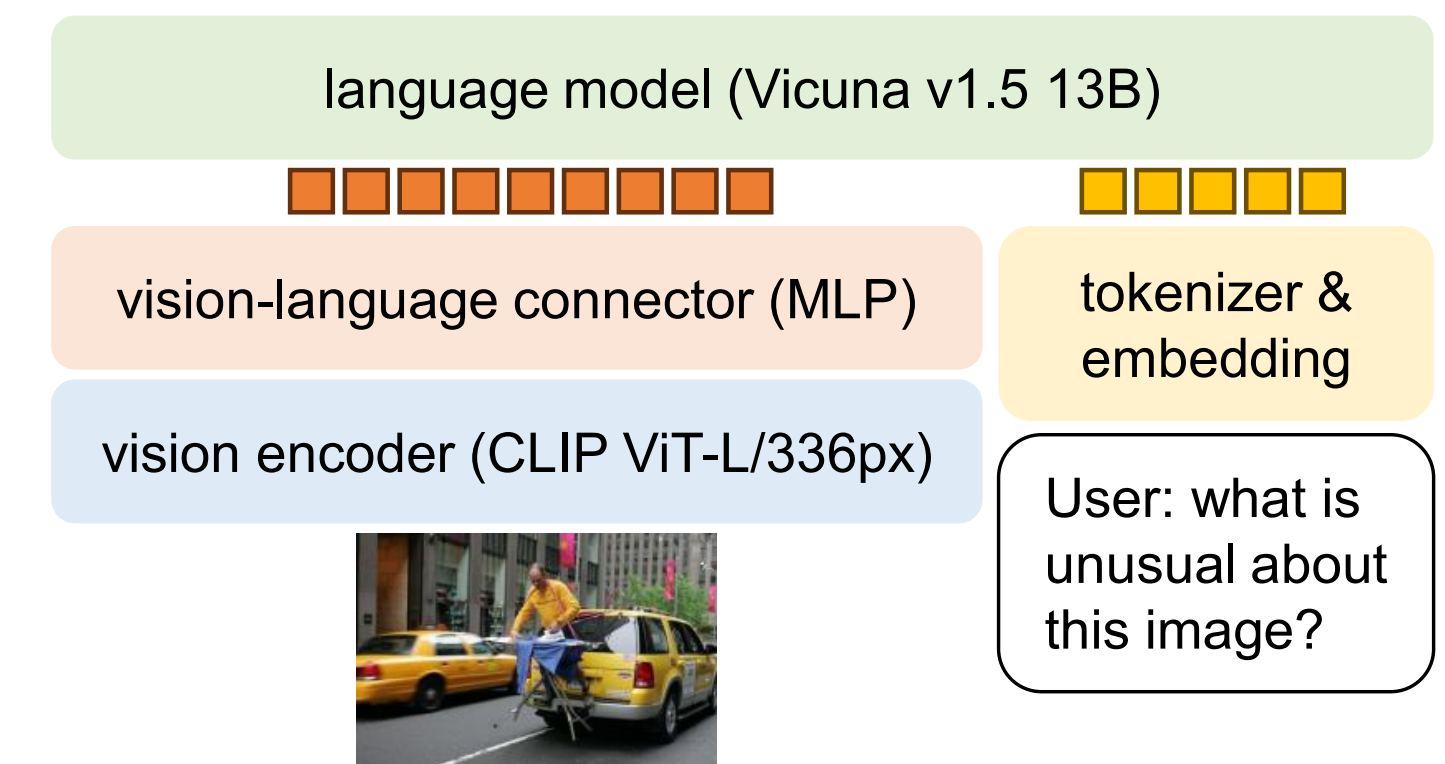
THE LLAVA FRAMEWORK

Training Stage 1: Vision-language Pre-Training

- Autoregressive **image captioning** (prefix-based)
 - **Only projector is trained** (LLM frozen)

Training Stage 2: Instruction Fine-Tuning

- Autoregressive **multitask training**
 - Formulate all tasks as textual answers, **only train on answer tokens**
 - **Projector + LLM trained**



Context type 1: Captions

A group of people standing outside of a black vehicle with various luggage.

Luggage surrounds a vehicle in an underground parking area

People try to fit all of their luggage in an SUV.

The sport utility vehicle is parked in the public garage, being packed for a trip

Some people with luggage near a van that is transporting it.

Context type 2: Boxes

person: [0.681, 0.242, 0.774, 0.694], backpack: [0.384, 0.696, 0.485, 0.914], suitcase: ...<omitted>

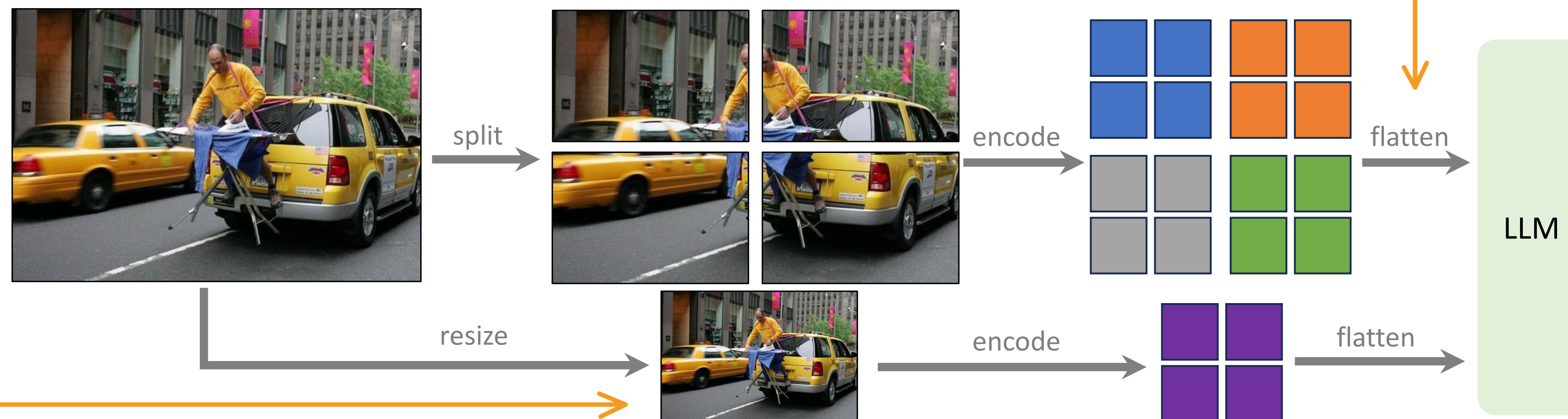


THE LLAVA FRAMEWORK

Handling high-resolution images (Llava 1.5)

Image encoder supports only 336x336 pixels (14x14 pixels per token)

1. **Resize to image encoder resolution and encode**
 - Results in $24 \times 24 = 576$ patch tokens
2. **Split full-resolution image into ~336x336-sized sub-images** and encode each of them
 - E.g. a 672x672 image would be split in 4 sub-images, resulting in $4 \times 576 = 2304$
3. **Flatten and concatenate all patch tokens**

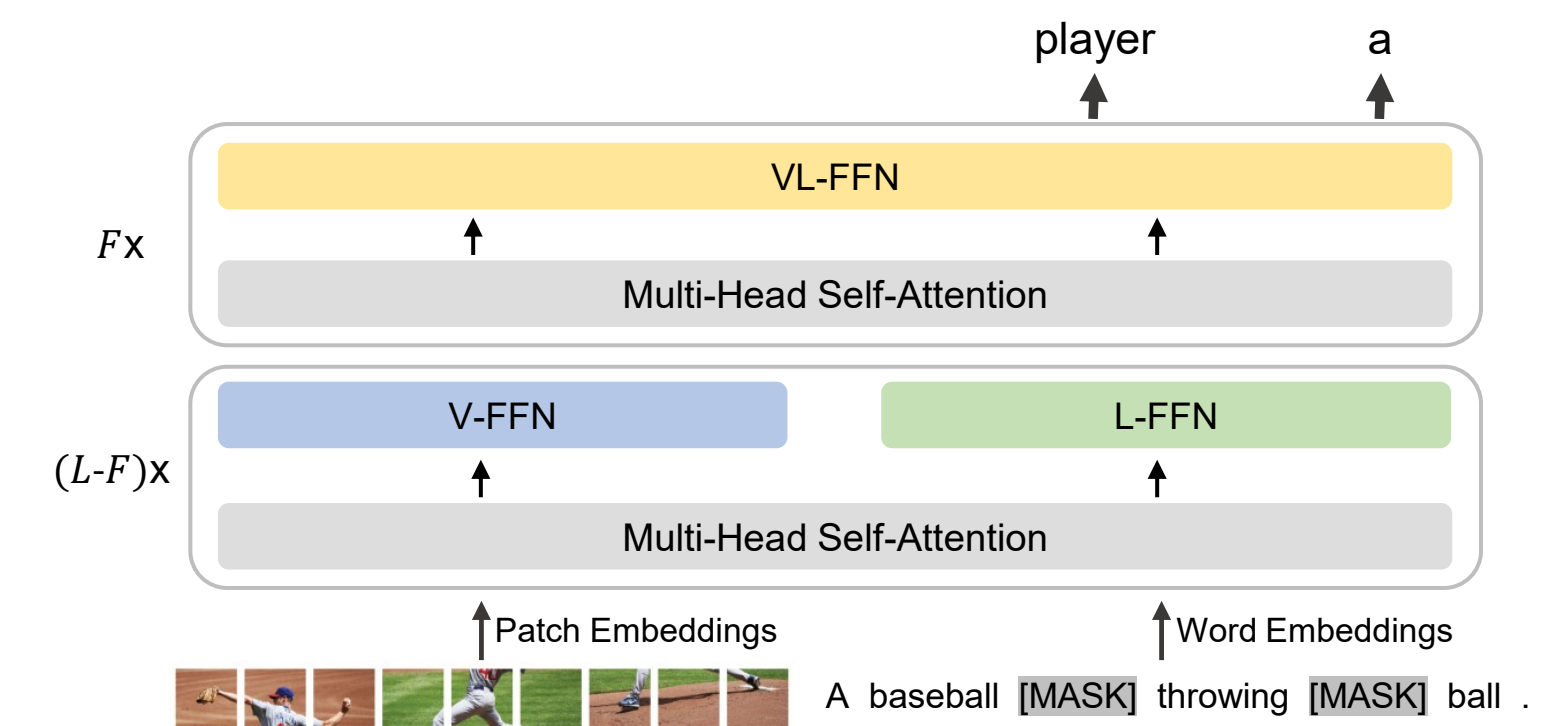


THE BEIT-3 FRAMEWORK

Vision Language Modeling with Modality Experts

Approach

- Uniform model for vision and text
 - Shared parameters for vision-only, text-only and image-text tasks
 - Single sequence for vision and/or text tokens
 - Experts for different modalities
 - Simple uniform training with single- and multimodal data



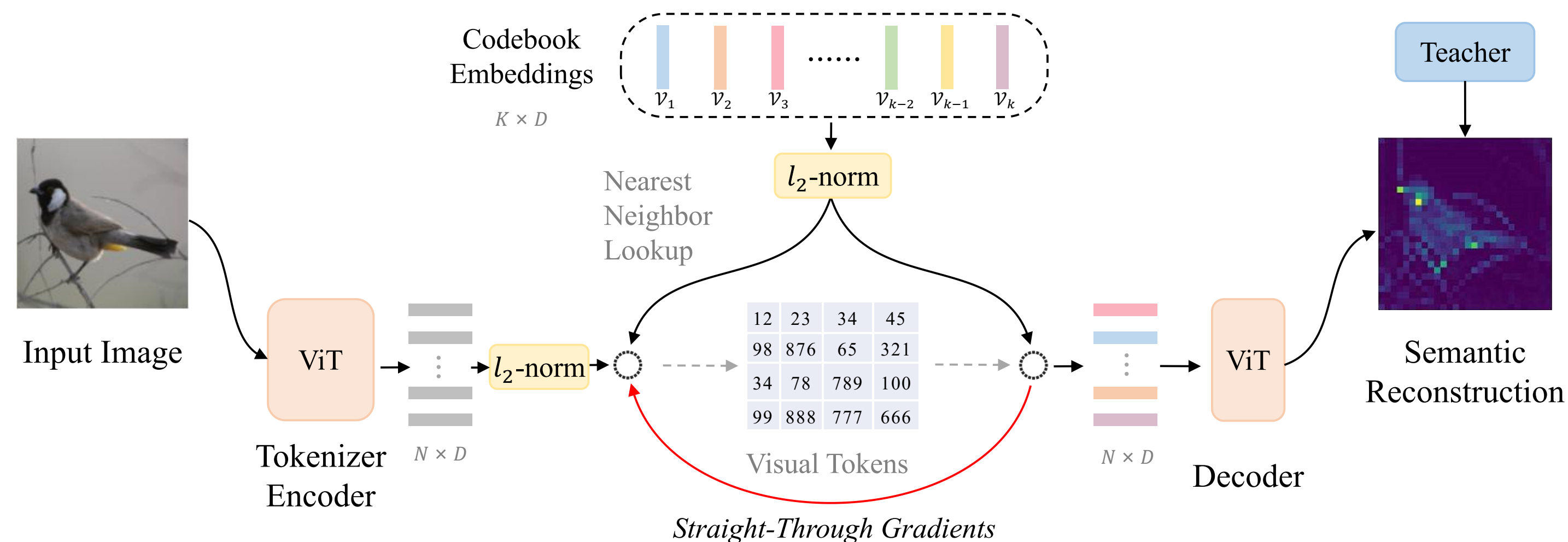
Wang et al (2022)

THE BEIT-3 FRAMEWORK

Images Tokenizer

Discrete Image Tokens (Codebook-based)

- Similar to VQ-VAE / VQ-GAN tokenizer, but:
 - Based on ViT encoder (no CNN)
 - Decoding into embeddings instead of pixel space
 - Targets for embeddings from frozen teacher model (an existing pre-trained model like DINO or CLIP)



THE BEIT-3 FRAMEWORK

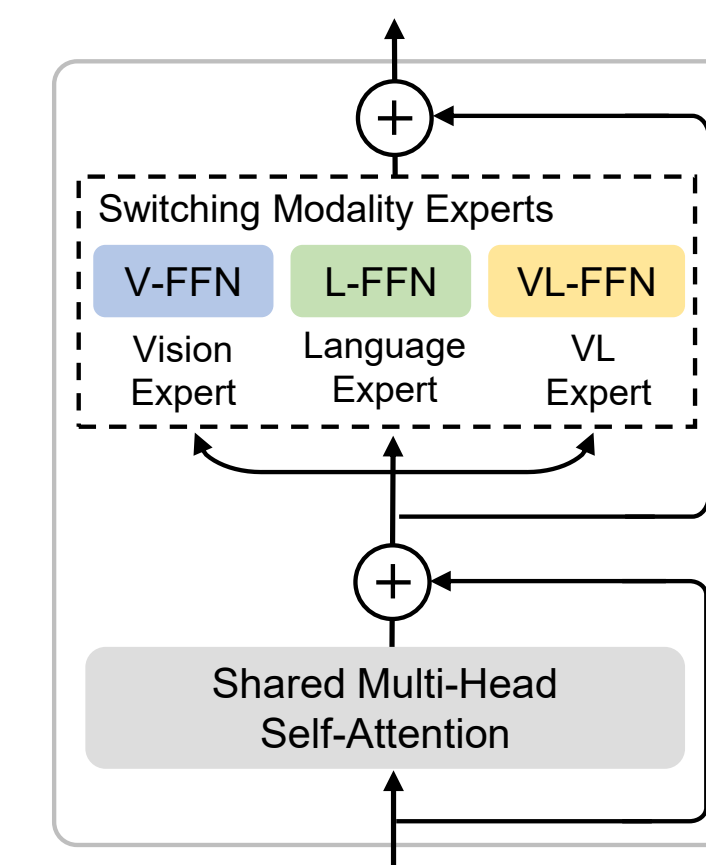
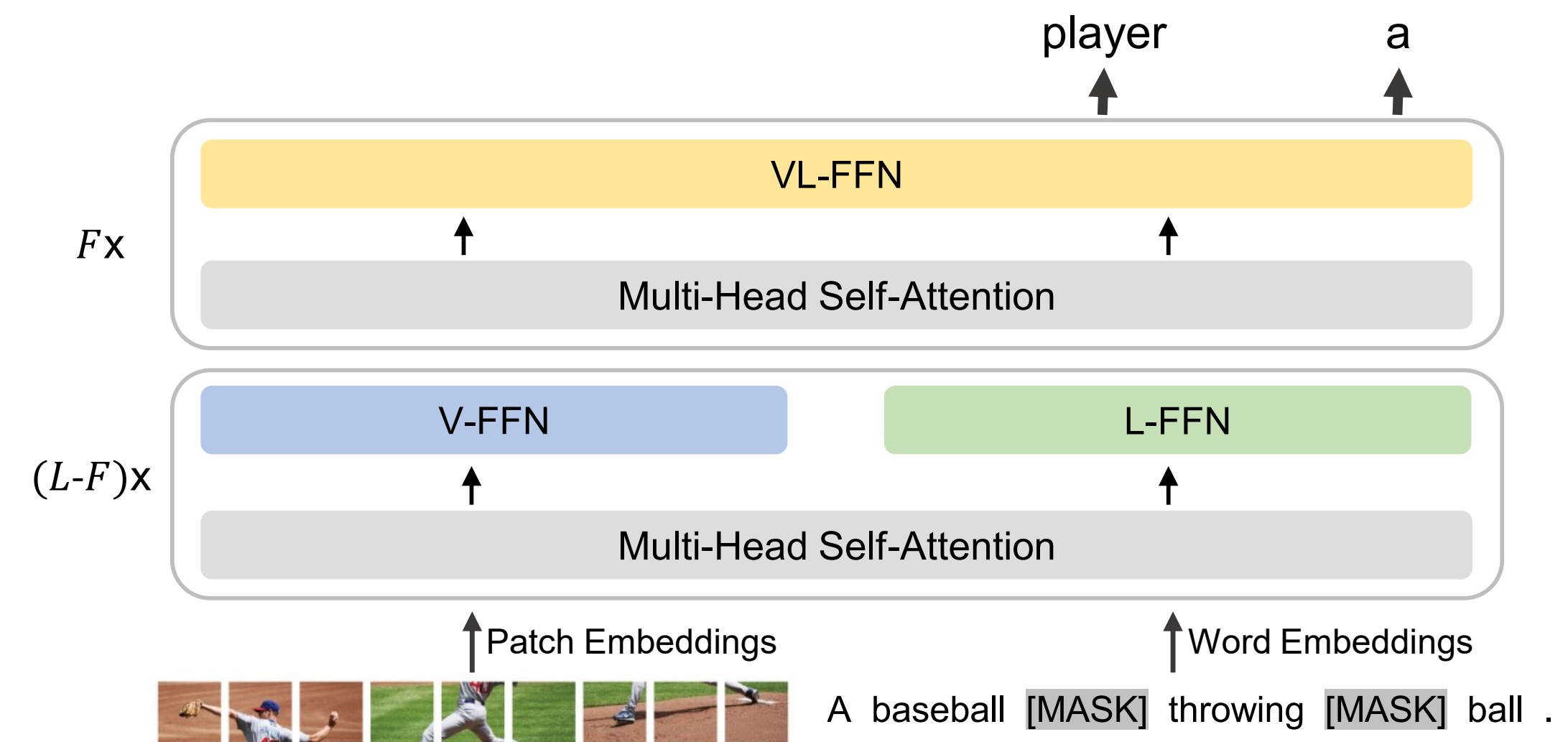
Architecture

Multimodal Experts (parameter-sets for FFNs)

- **First layers:**
Vision (V) or **Language (T)** expert
(based on type of token)
- **Last layers for vision language tasks**
only **Vision-Language (VL)** expert
- **Last layers for unimodal tasks**
Vision (V) or **Language (T)** expert
(based on type of token)

Shared Multi-head self attention

- Shared QKVO projection layers
- Full intra- and inter-modality attention

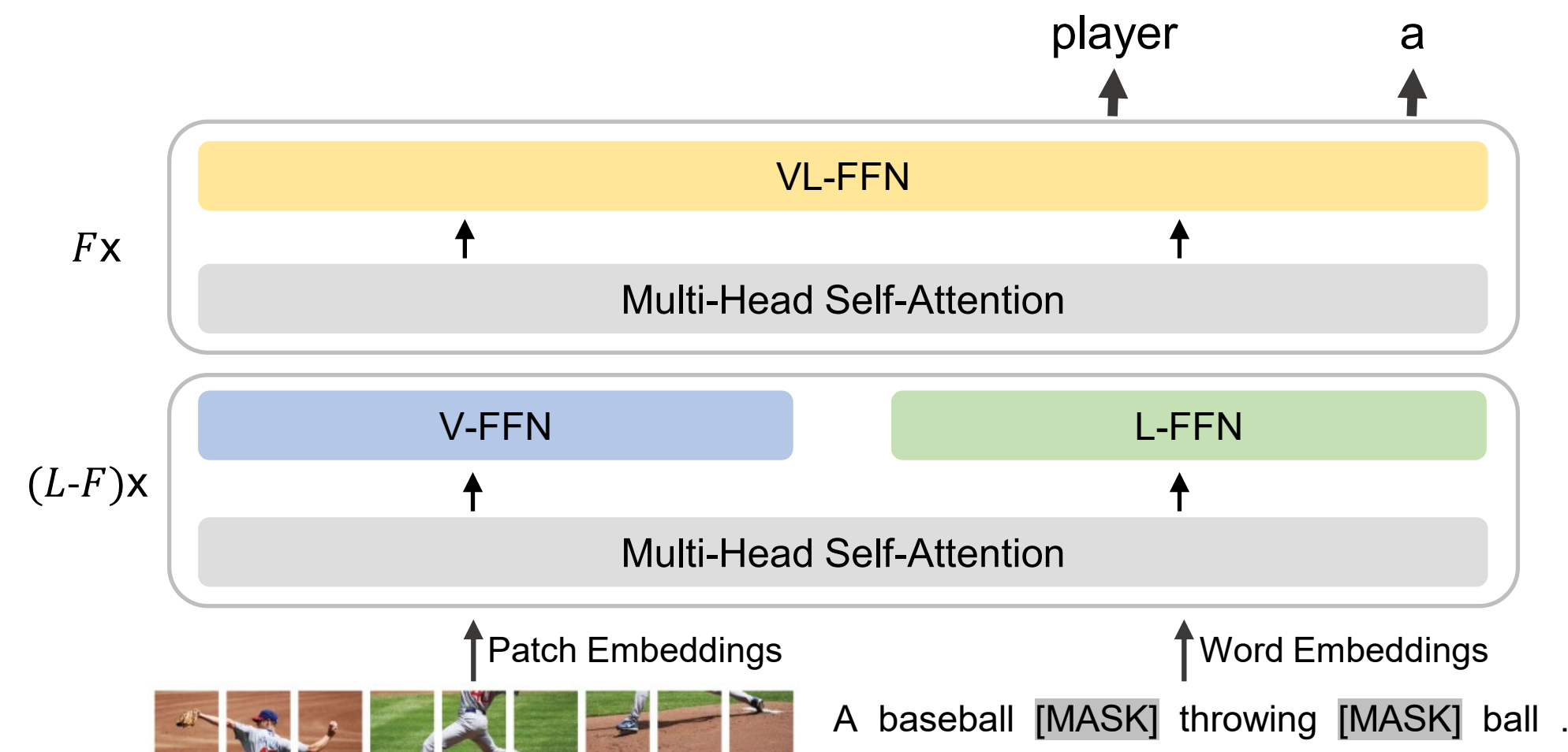


THE BEIT-3 FRAMEWORK

Training:

Unimodal and Multimodal Masked Modeling

- Trained on text-only, image-only, and image-text data (activating experts based on task and token)
- Masking 40% of image patches
- Masking 15% of unimodal or 50% of multimodal text tokens



THE BEIT-3 FRAMEWORK

Inference

Add task-specific prediction heads or fine-tune token heads

(a) Image-only tasks

- Only V experts (in all L layers)

(b) Text-only tasks

- Only L experts (in all L layers)

(c) Vision-Language tasks

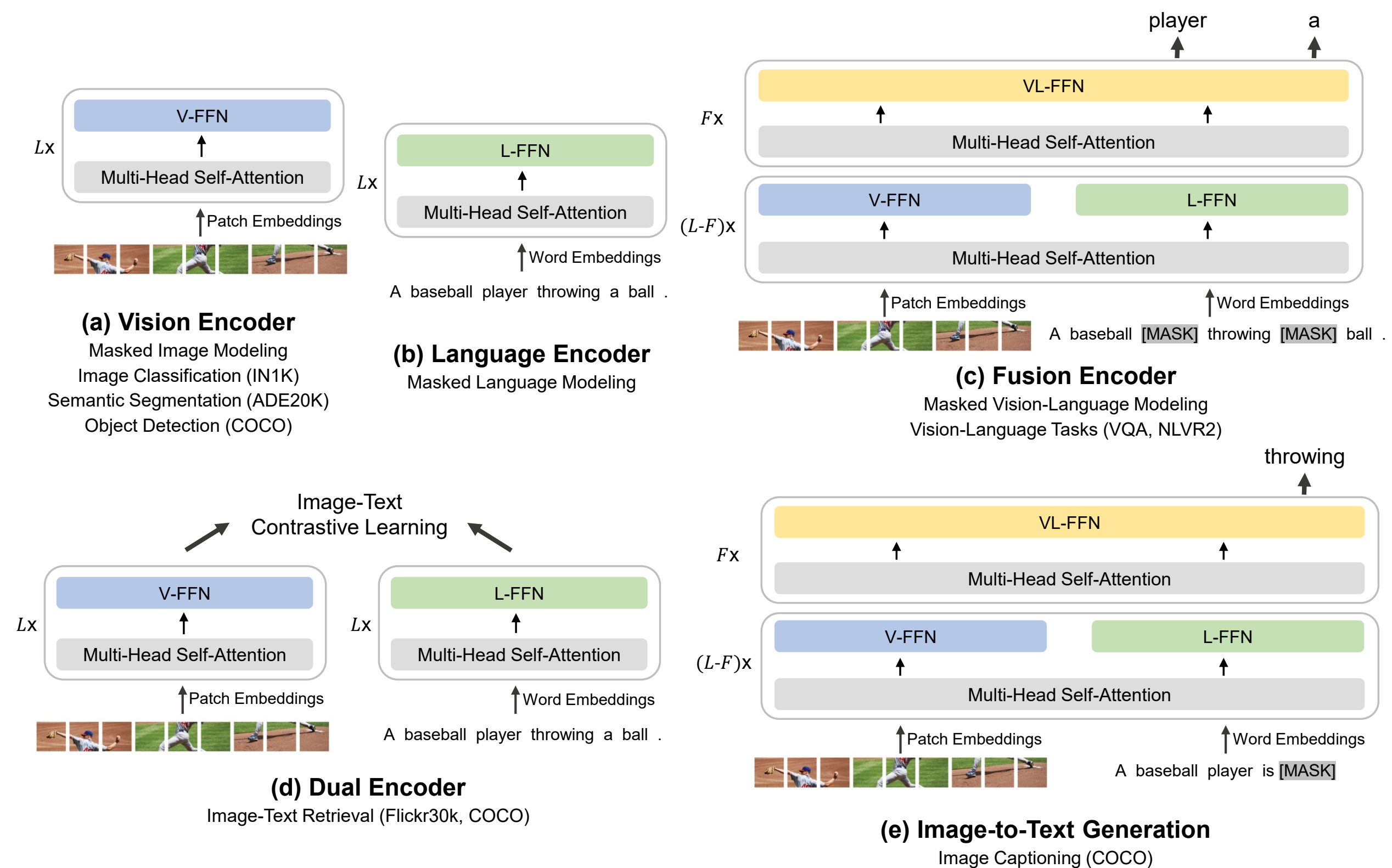
- VL experts in last F layers
- V or L experts in other layers, with full attention

(d) Image-Text Retrieval

- V or L experts, no attention between them
- Can also be used for zero-shot image classification

(e) Image-to-Text Generation

- Like vision-language tasks (c)
- Next token prediction using [MASK]

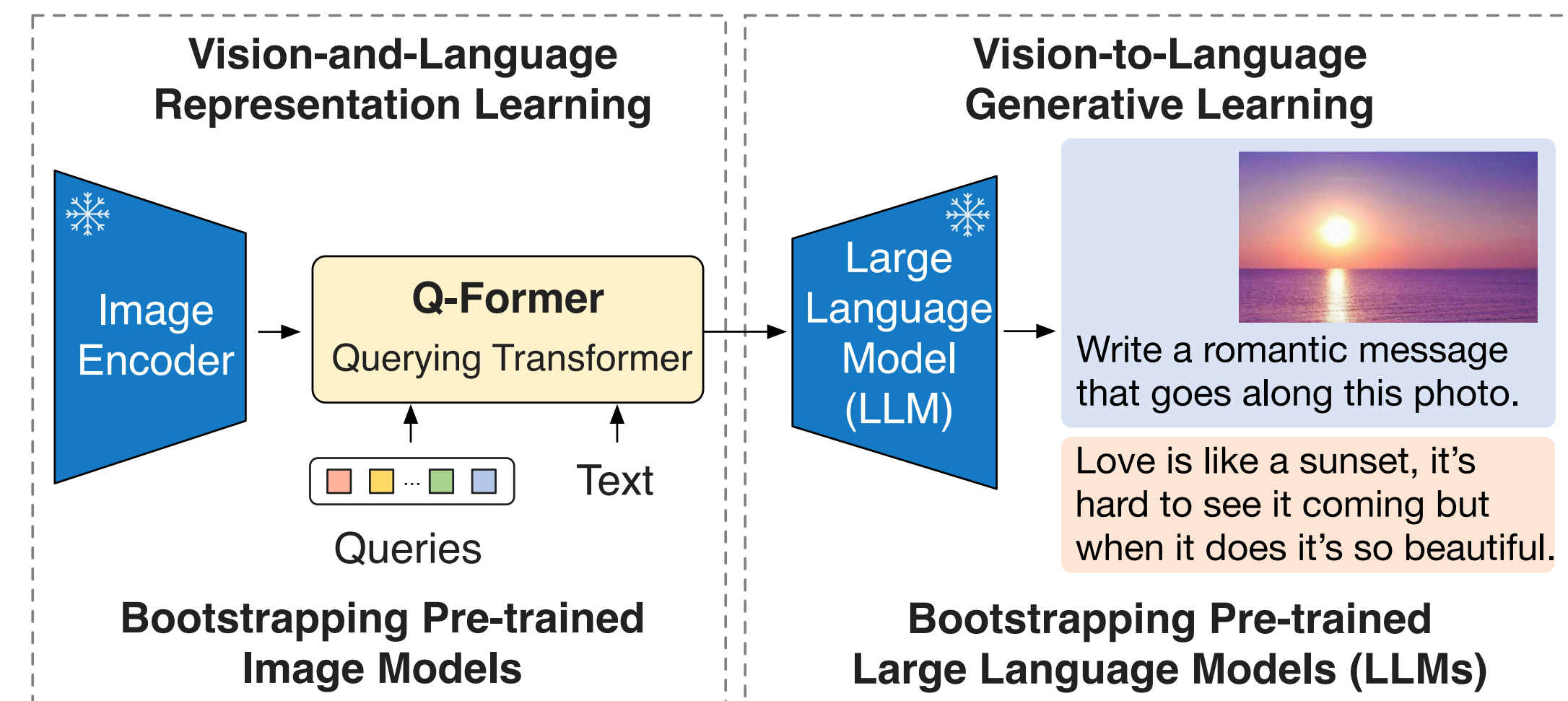


THE BLIP-2 FRAMEWORK

Vision Language Modeling with Latent Image Queries

Approach

- Efficient vision-language representation learning
 - Frozen image encoder
 - Latent queries (Q-former) for vision-language features
- Alignment with LLM for text generation
 - Frozen LLM
- Multi-stage, multi-objective training
 - Focus on training the Q-former

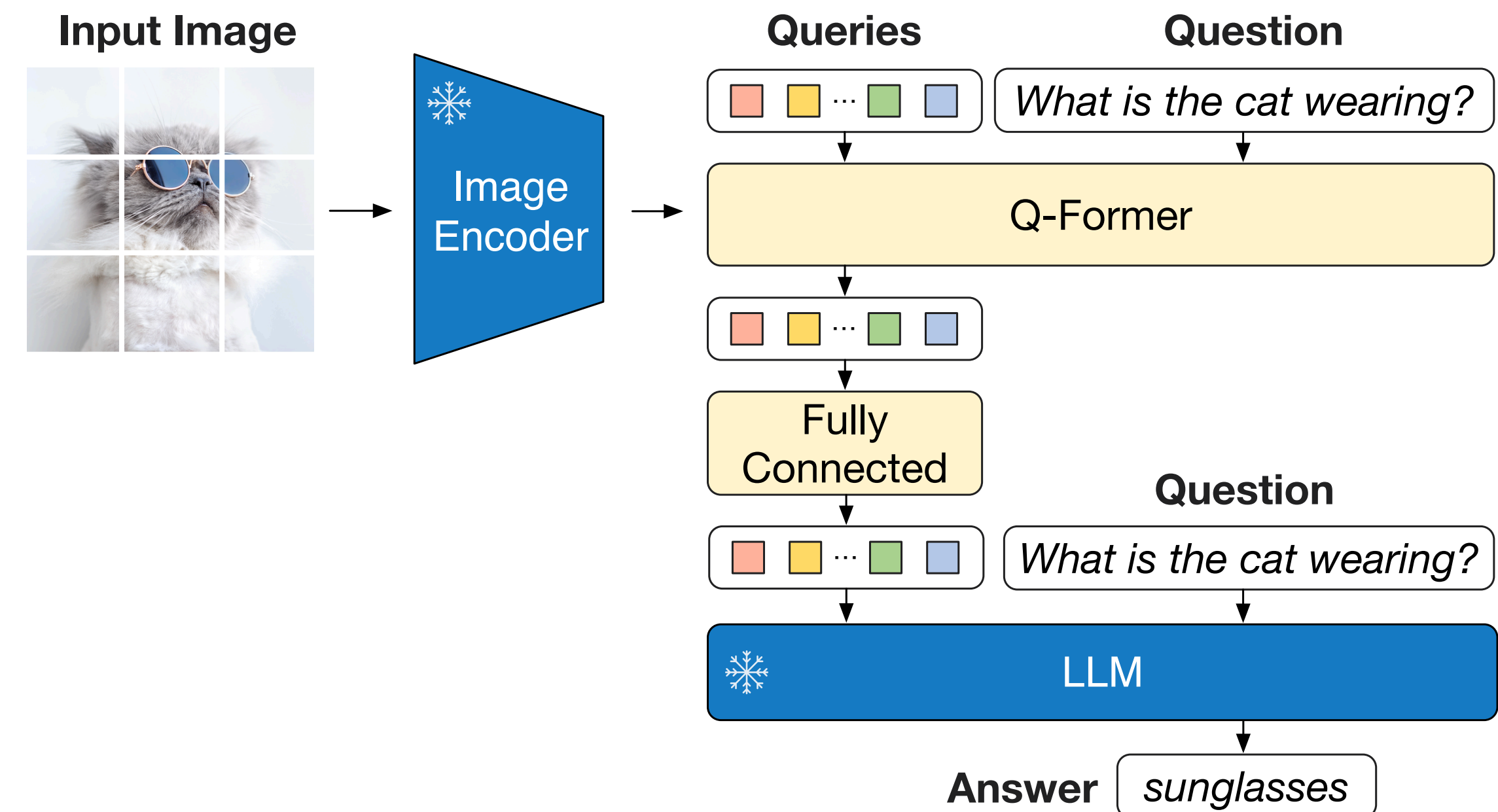


Li et al (2023)

THE BLIP-2 FRAMEWORK

Architecture

- Q-Former
 - Latent queries
 - Conditioned on image features
 - Conditioned on text input (question)
- MLP Projection
 - Projects query output features to LLM
- LLM
 - Concatenate query tokens with (embedded) text tokens (question)

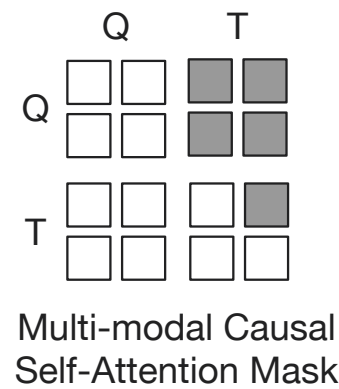
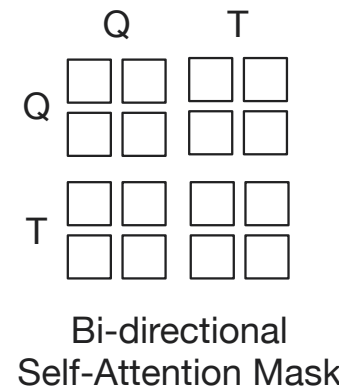
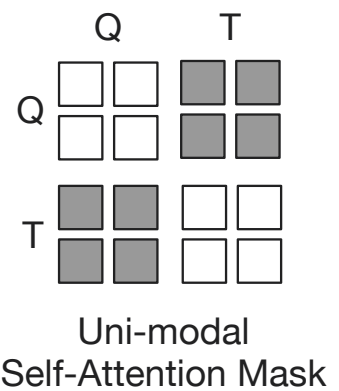


THE BLIP-2 FRAMEWORK

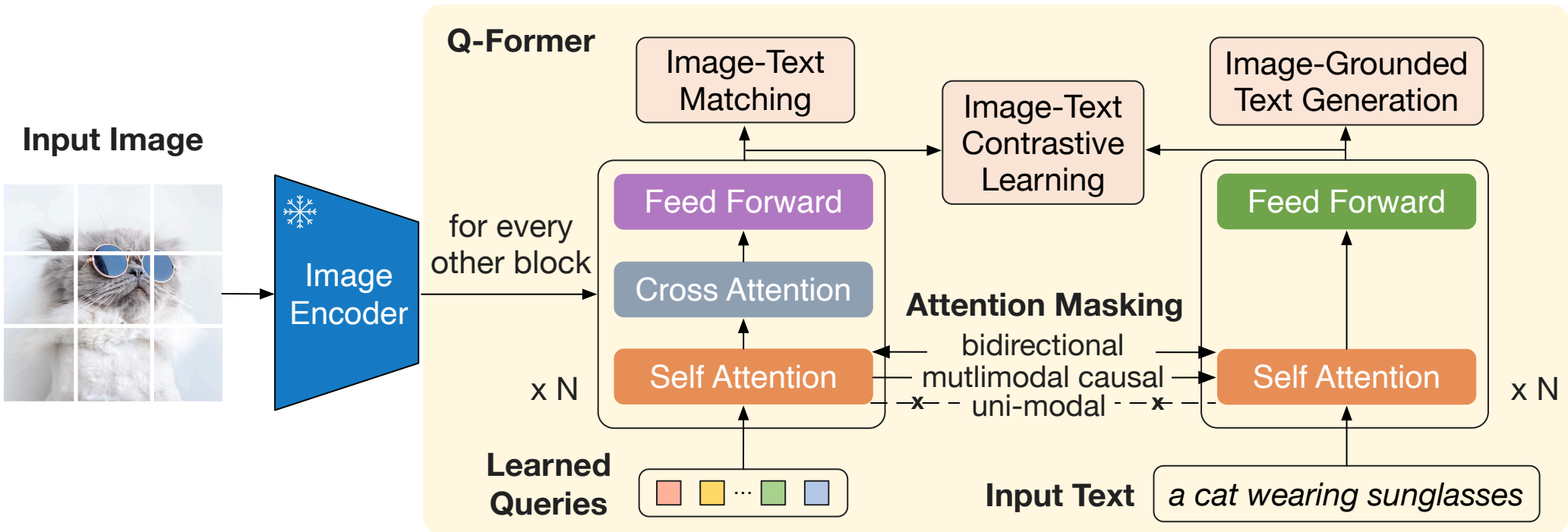
Training Stage 1: Vision-Language Representation Learning (Q-Former Training)

Reminder: Shared self-attention for query and text tokens with different masking strategies

- **Image-Text Contrastive Learning (ITC) — Dual encoder**
 - CLIP loss on query-token features vs. text token features
 - **Mask:** No interaction btw queries vs. text
- **Image-Text Matching (ITM) — Joint encoder**
 - Binary predictor whether image-text pair is from the same sample
 - **Mask:** Full attention (interaction btw. queries and text)
- **Image-Grounded Text Generation — Encoder-decoder**
 - Autoregressive text generation (image captioning)
 - **Mask:** causal within text, full attention text → queries and within queries



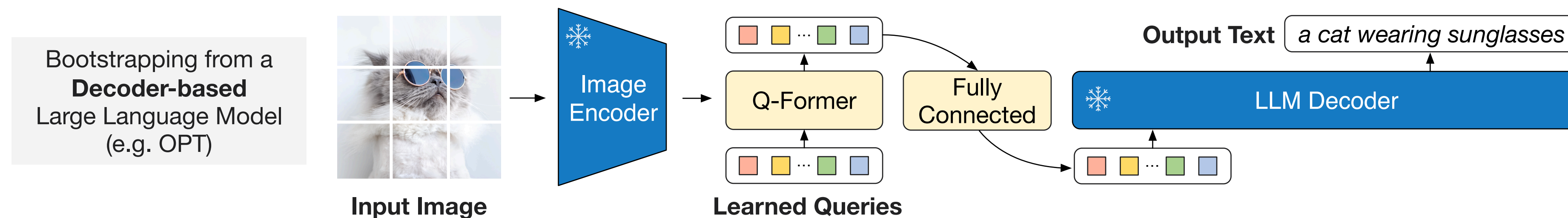
Q: query token positions; T: text token positions.
■ masked □ unmasked



THE BLIP-2 FRAMEWORK

Training Stage 2: Alignment with LLM for Generation

- **Autoregressive image captioning (frozen LLM)**
 - Q-Former extracts visual features through queries (no text used for captioning task)
 - Query tokens are projected to LLM space
 - Projected query tokens are concatenated with target text sequence
 - Autoregressive training (for target text sequence)

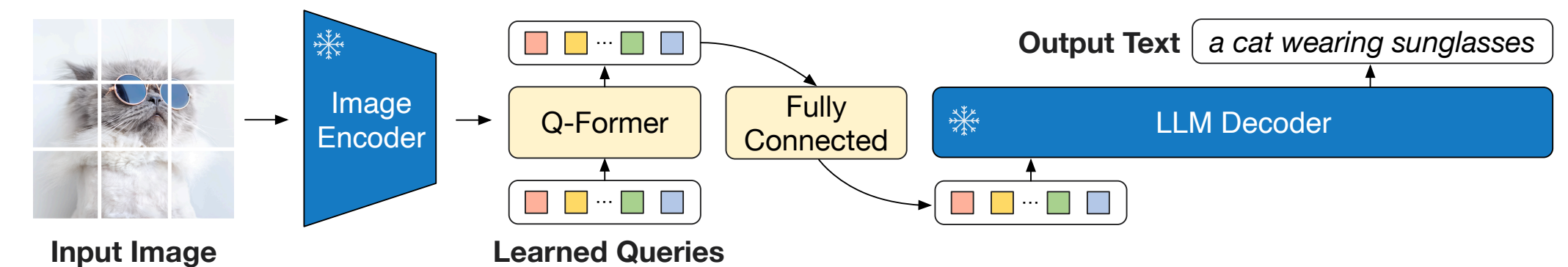


THE BLIP-2 FRAMEWORK

Inference

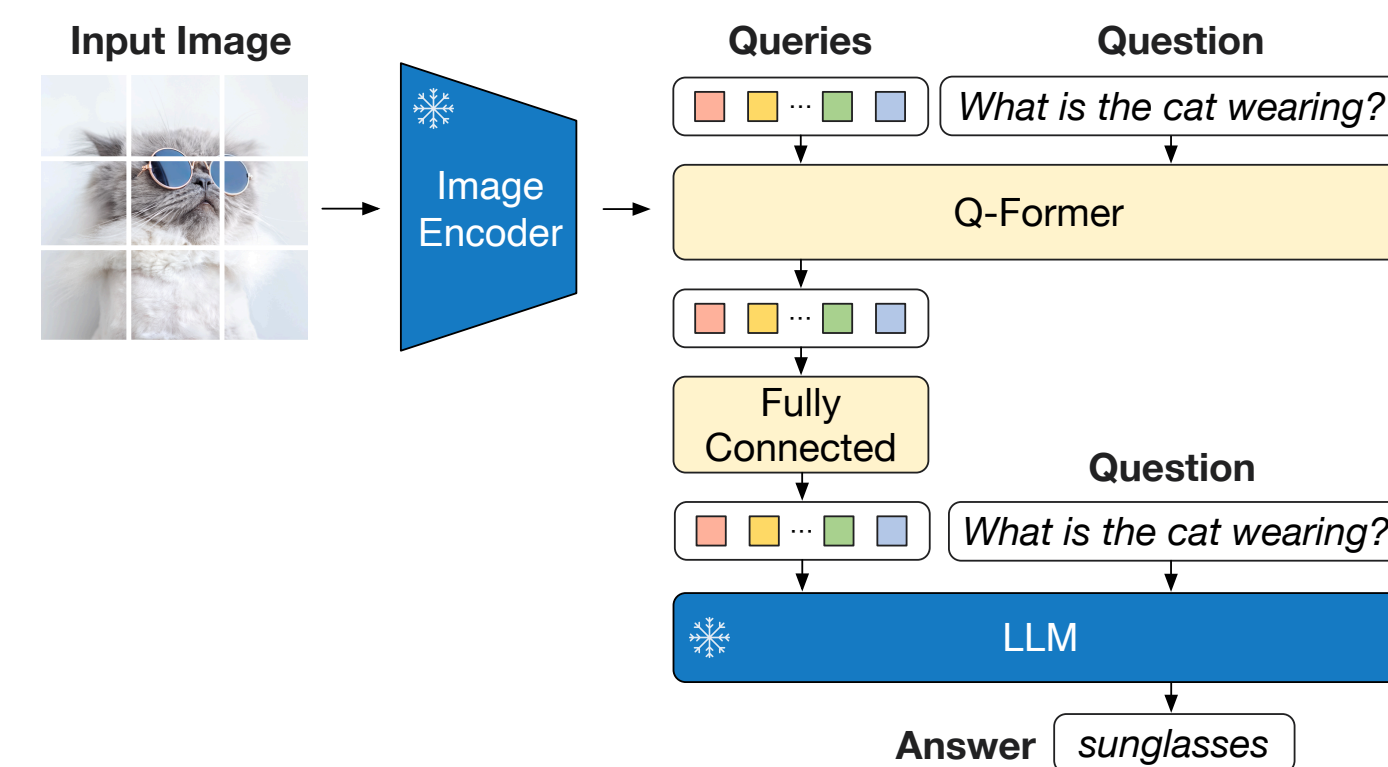
- **Image captioning**

- Condition LLM on query tokens (like pre-training)
- No text tokens in Q-former



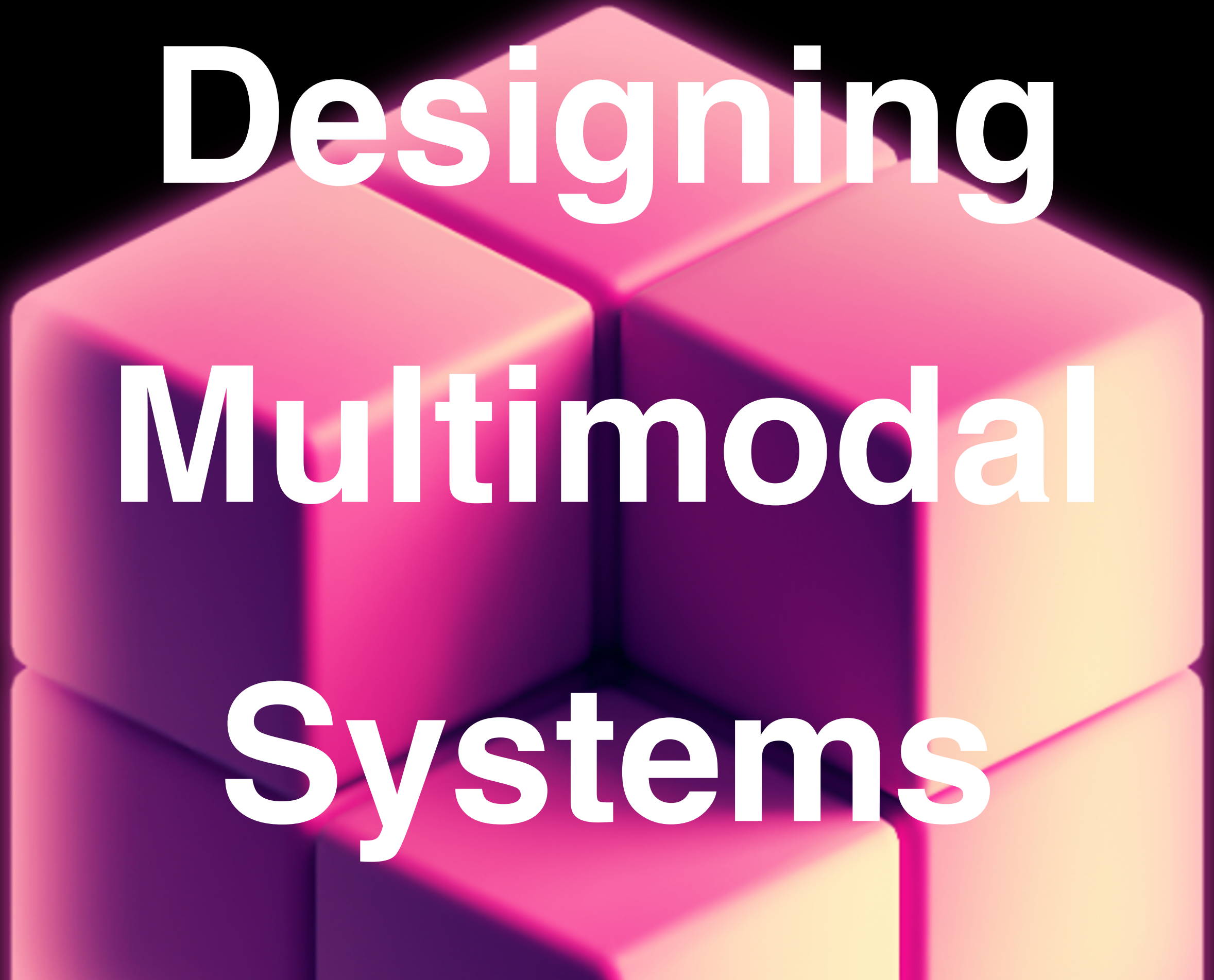
- **Visual question-answering**

- Encode query tokens with question (full visibility)
- Condition LLM on query tokens and question



- **Image-Text Retrieval**

- Q-former only (no LLM needed)
1. Encode query and text tokens individually (like in ITC) to find candidates
 2. Encode query and text tokens jointly (like ITM) on candidates

The background features a cluster of translucent, 3D cubes in shades of pink, magenta, and purple. The cubes are arranged in a way that creates a sense of depth and perspective, with some cubes appearing to be in front of others. The lighting is soft, creating a glowing effect around the cubes.

HOWTO: Designing Multimodal Systems

1. KNOW YOUR DATA AND TASKS

1a. Downstream Task(s)

- Which input modalities are available during inference?
 - Are they paired, alternative, or may some be missing?
- What are my target/evaluation tasks?
 - What should the model predict / what are my output modalities?
 - Should this be a specialist (single/few tasks) or generalist (multitask) model?
 - **Which evaluation metrics to use?**

1b. Training Data

- Which input/output modalities are available during training?
 - How many unimodal samples? How many paired / multimodal samples?
 - How closely are the modalities paired (do acquisition time points align)?
 - Which annotations are available? What is their quality?

2. KNOW YOUR MODALITIES

2a. What structure do your modalities have?

- Which encoder/decoder architectures can handle these structures?
- Is the structure (partially) shared between modalities?
- Are different modalities registered to each other?
- Are there “stronger” and “weaker” modalities
 - Having more/easier to extract or less/harder to extract information
- Do you want to simplify the structures (e.g. to sets/sequences)?
- How computationally expensive are the structures?

2b. How do you expect the modalities to interact?

- Is there a primary modality?
- Are there subsets of modalities that are expected to interact differently?
- At which level (input, features, predictions) do you expect the modalities to interact?
- Would an intermediate multimodal structure (set/sequences/graph) make sense?

3. FIND PRE-TRAINED MODELS

3a. Pre-Trained Encoder/Decoder Models

- What are their input/output structures? Or do they produce/expect global vectors?
- Can you freeze them or should they be fine-tuned?
- How expensive are they?
- Is there a model you want to integrate other models into (e.g. an LLM)?

3b. Pre-Trained Multimodal Models

- Do they already (partially) solve your tasks?
- Could you use components of them?

3c. Pre-Trained Supplementary Models for Transfer

- Can you utilize them as a training signal?
 - Teacher/Student
 - Generating targets

4. DEFINE MULTIMODAL ARCHITECTURE AND TRAINING OBJECTIVES

4a. Build your architecture

- Decide on a multimodal structure/space
 - Could be based on one primary modality (e.g. image, sequence/text) or a general structure (set/sequence)
 - Several different structures/spaces possible at different places in the model
 - **Consider:** Target task, available modalities, **existing works (literature), compute**
- Decide on fusion / translation techniques
 - Mixture of techniques is possible, different techniques for different modality pairs is possible
 - **Consider:** Properties of different modalities, available encoders/decoders, available pre-trained multimodal models, **existing works (literature), compute**
- Decide on encoder / decoder models
 - Decide on their architectures or pick pre-trained ones
 - **Consider:** Available encoders/decoders and their output/input structures, used fusion/translation techniques, **existing works (literature), compute**

4. DEFINE MULTIMODAL ARCHITECTURE AND TRAINING OBJECTIVES

4b. Build your training pipeline

- Select possible objectives / loss functions
 - Self-supervised: Contrastive or reconstruction-based?
 - Generative: Autoregressive, diffusion, or other?
 - Unimodal and multimodal objectives?
 - Task-specific targets? Supplementary targets?
 - **Consider:** Available targets / training data, supplementary models, properties of modality pairings, **existing works (literature), compute**
- Pre-Alignment of Components
 - Some components may be pre-aligned (e.g. CLIP)
 - **Consider:** Properties of different modalities, available encoders/decoders, training data, **existing works (literature), compute**
- Training Stages
 - The training may be split into several stages
 - **Consider:** Available pre-trained models, training data, architecture / sub-components, **compute!!**, evaluation of individual stages, **existing works (literature)**

4. DEFINE MULTIMODAL ARCHITECTURE AND TRAINING OBJECTIVES

4b. Design your inference

- Decide how to use the trained components during inference
 - Which components can you drop, which do you keep after training
- Choose inference-time adaptations if needed
 - Preprocessing, e.g. placeholders for missing data
 - Post-processing, e.g. thresholding or filtering predictions
 - Sampling techniques for generative models
 - Prompting
 - Combining different outputs
- Inference Optimization: Can you speed up inference?
 - KV-caching in autoregressive transformer models
 - Quantization
 - Model Distillation or Pruning
 - ...

4. DEFINE MULTIMODAL ARCHITECTURE AND TRAINING OBJECTIVES

4a, 4b, and 4c depend on each other!

- You may need to consider all of them for your decisions
- Always look for existing literature or models that cover similar use cases to yours
 - Sometimes you can follow a similar architecture, but just change the encoders/decoders to match your modalities

During design, implementation, training, or evaluation, you may learn more about your problem:

- Characteristics of your task, modalities, and data (see steps 1-3)
- Problems or chances in your design choices or in components
 - e.g., pre-trained models may not work as expected

➡ **Use these insights to refine your understanding and design**

➡ **Iterate!**

- **Important:** Always evaluate on your target task (if possible) or use preliminary tasks to evaluate intermediate stages

HOWTO: DESIGNING MULTIMODAL SYSTEMS

Multimodal systems are complex and often very problem-specific:

There is no standard scheme to design them all!

Instead, try to start simple and based on “battle-proven” solutions

Summary

and

Recap



SUMMARY OF PREVIOUS LECTURES

1. Data types, representations, and feature spaces

- Modalities and their structure
- Encodings, representations, feature spaces (projections/distances)

2. Architectures for different modalities

- Sets, Sequences, Tuples
- Groups and Equivariant Deep Learning
- Graphs
- Implicit neural function (INRs)

3. Training signals

- Loss functions, Self-supervision, Reinforcement Learning
- Parameter-efficient fine-tuning (PEFT)

MULTIMODAL PIPELINES AND FRAMEWORKS

- **Fusion: Combining Modalities**
 - Prediction (Late) Fusion + Agents
 - Representation (Early) Fusion
 - Input (Very early) Fusion + Excuse into tokenization/embedding
 - Structural Fusion and Multimodal Structures
- **Translation: Converting btw Modalities**
 - Query tokens, autoregressive, noise-based
- **Alignment: Training Objectives**
 - Discriminative, Reconstruction-based, Multitask
- **Multimodal Frameworks**
 - Llava, BEiT-3, BLIP-2
- **How-To: Designing Multimodal Systems**

Open Research Questions

An abstract geometric background featuring a cluster of 3D cubes. The cubes are rendered in a gradient of colors, ranging from deep purple to bright pink, with some faces appearing more illuminated than others, creating a sense of depth and volume. The cubes are arranged in a way that they appear to be stacked or interlocking, with some faces visible in the foreground and others receding into the background.

OPEN RESEARCH QUESTIONS

Multimodal Agents and Decision Making

planning and acting in multimodal environments
e.g. robotics or clinical decision making

Scaling the number of modalities

currently often limited to 2-3
how to generalize this to 10s or 100s?
or the “full reality”

Efficient alignment

e.g. less paired data, or more effective use of it
may be an requirement for modality scaling
information compression vs. invariance

Compute efficiency and context lengths

quadratic complexity of transformers...
may be an requirement for modality scaling

Low data domains

e.g. industrial process data
or some medical modalities like MRIs

Announcements



NEXT WEEK: INVITED GUEST LECTURE

Guest lecture by

Georgios Kaissis

senior research scientist at Google DeepMind

**Thinking in Vectors, Dreaming in Frames:
AI Video Generation and Multimodal Reasoning**

In two weeks: no lecture!

Exam



EXAM Q&A

- 90 minutes written exam
- similar to homework assignments
- understanding not remembering, e.g.
 - NOT: draw the remembered architecture
 - INSTEAD: why do you need this component / what would happen without itor
 - NOT: list all possible algorithms for ...
 - INSTEAD: you are given this problem, which of A or B would you use and why

Further Readings



FURTHER READINGS

General

- Liang et al (2022) “Tutorial on Multimodal Machine Learning”
<https://cmu-multicomp-lab.github.io/mmml-tutorial/schedule/>

Agents

- Anthropic (2025) “How we built our multi-agent research system”
<https://www.anthropic.com/engineering/built-multi-agent-research-system>
- Cognition (2025) “Don’t Build Multi-Agents”
<https://cognition.ai/blog/dont-build-multi-agents>

Fusion

- Zadeh et al (2017) “Tensor Fusion Network for Multimodal Sentiment Analysis”
<https://aclanthology.org/D17-1115.pdf>
- Ziegler et al (2019) “Encoder-Agnostic Adaptation for Conditional Language Generation”
<https://arxiv.org/abs/1908.06938>
- Peebles et al (2022) “Scalable Diffusion Models with Transformers”
<https://arxiv.org/abs/2212.09748>

FURTHER READINGS

Multimodal Models (1/2)

- **Gato:** Reed et al (2022) “A Generalist Agent”
<https://arxiv.org/abs/2205.06175>
- **4M:** Mizrahi et al (2023) “4M: Massively Multimodal Masked Modeling”
<https://arxiv.org/abs/2312.06647>
- **Llava:** Liu et al (2023) “Visual Instruction Tuning”
<https://arxiv.org/abs/2304.08485>
- **Llava 1.5:** Liu et al (2023) “Improved Baselines with Visual Instruction Tuning”
<https://arxiv.org/abs/2310.03744>
- **ViLBERT:** Lu et al (2019) “ViLBERT: Pretraining Task-Agnostic Visiolinguistic Representations for Vision-and-Language Tasks”
<https://arxiv.org/abs/1908.02265>

FURTHER READINGS

Multimodal Models (2/2)

- **BEiT-2:** Peng et al (2022) “BEiT v2: Masked Image Modeling with Vector-Quantized Visual Tokenizers”
<https://arxiv.org/abs/2208.06366>
- **BEiT-3:** Wang et al (2022) “Image as a Foreign Language: BEiT Pretraining for All Vision and Vision-Language Tasks”
<https://arxiv.org/abs/2208.10442>
- **Perceiver IO:** Jaegle et al (2021) “Perceiver IO: A General Architecture for Structured Inputs & Outputs”
<https://arxiv.org/abs/2107.14795>
- **BLIP-2:** Li et al (2023) “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models”
<https://arxiv.org/abs/2301.12597>